

Greedy Algorithms & Binary Search

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Greedy Algorithms	1
1.1	What Is a Greedy Algorithm?	1
1.2	Assign Cookies	3
1.3	Fractional Knapsack Problem	6
1.4	Greedy Algorithm to Find Minimum Number of Coins	10
1.5	Lemonade Change	13
1.6	Valid Parenthesis String	17
1.7	N Meetings in One Room	20
1.8	Jump Game	23
1.9	Jump Game II	25
1.10	Minimum Number of Platforms Required for a Railway	28
1.11	Job Sequencing Problem	31
1.12	Candy	35
1.13	Shortest Job First (SJF) CPU Scheduling	38
1.14	Insert Interval	41
1.15	Merge Intervals	45
1.16	Non-overlapping Intervals	47
2	Binary Search	51
2.1	What Is Binary Search, Really?	51
2.2	Binary Search to Find X in a Sorted Array	53
2.3	Implement Lower Bound	55
2.4	Implement Upper Bound	58
2.5	Search Insert Position	60
2.6	Floor and Ceil in a Sorted Array	62
2.7	Find the First and Last Occurrence of an Element	64
2.8	Count Occurrences of an Element in a Sorted Array	67
2.9	Search in a Rotated Sorted Array I	69
2.10	Search in a Rotated Sorted Array II	72
2.11	Find Minimum in a Rotated Sorted Array	75
2.12	Find How Many Times an Array Has Been Rotated	78
2.13	Single Element in a Sorted Array	79
2.14	Find Peak Element	82
2.15	Find Square Root of a Number using Binary Search	84
2.16	Find the Nth Root of a Number using Binary Search	87
2.17	Koko Eating Bananas	89
2.18	Minimum Number of Days to Make M Bouquets	92
2.19	Find the Smallest Divisor Given a Threshold	95
2.20	Capacity to Ship Packages within D Days	97
2.21	Kth Missing Positive Number	100
2.22	Aggressive Cows Problem	102
2.23	Book Allocation Problem	105
2.24	Split Array – Largest Sum	108
2.25	Painter's Partition Problem	110
2.26	Minimize the Maximum Distance Between Gas Stations	112
2.27	Median of Two Sorted Arrays	114

2.28 Kth Element of Two Sorted Arrays	117
2.29 Row with Maximum Number of 1s	120
2.30 Search in a 2D Matrix	123
2.31 Search in a 2D Matrix II	125
2.32 Find a Peak Element in a 2D Matrix	128
2.33 Median of a Row-wise Sorted Matrix	131

Chapter 1

Greedy Algorithms

A greedy algorithm never looks back. At every step it makes the choice that looks best at that instant, and trusts – or, better, proves – that this sequence of locally best choices assembles itself into a globally best solution.

1.1 What Is a Greedy Algorithm?

A **greedy algorithm** builds up a solution piece by piece, and at every step it commits, without revision, to whichever available choice looks most favourable *right now*. It never reconsiders an earlier decision, and it never explores alternative branches the way recursion or dynamic programming would. This makes greedy algorithms extremely fast and extremely simple to code – usually a single pass over the input, sometimes after a sort – but it also means that greedy is dangerous to apply carelessly. A locally best choice is only safe if it can be *proved* never to block a globally best solution. When that proof exists, greedy is the right tool, and it is almost always the most elegant one. When that proof does not exist, a greedy-looking shortcut will quietly produce wrong answers on some hidden test case, while looking completely correct on the examples you tried by hand.

When Is Greedy Safe to Use?

A problem is a good candidate for a greedy algorithm when it has two properties.

1. **Greedy choice property.** There is a way to pick, at each step, a single locally optimal choice such that some globally optimal solution is consistent with that choice – i.e. making the greedy choice never rules out the best possible final answer.
2. **Optimal substructure.** Once the greedy choice is made, what remains is a smaller version of the same problem, and an optimal solution to the whole problem is built from the greedy choice plus an optimal solution to this smaller remaining problem.

Contrast this with dynamic programming, which is needed precisely when the greedy choice property *fails* – when the locally best choice at a step can depend on, or conflict with, choices made elsewhere, forcing us to compare many overlapping subproblems instead of committing to one path.

How to Recognise a Greedy Problem from Its Wording

Across the problems in this chapter, certain words and phrases recur whenever greedy is the intended technique. Training yourself to notice them is the fastest way to identify the right pattern within seconds of reading a problem statement.

Recurring Greedy Keywords

- “Maximum number of non-overlapping ...” / “minimum number of ... to remove” – classic interval scheduling; sort by end time or by a similar criterion and sweep once.

- **“Assign / allocate / distribute to satisfy as many as possible”** – two-pointer style matching after sorting both sides (e.g. Assign Cookies).
- **“In order” / “queue” / “one at a time, immediately”** – signals that decisions must be made online, in a fixed order, with no ability to rearrange (e.g. Lemonade Change).
- **“Maximize profit/value subject to a capacity/weight/time limit” with divisible items** – Fractional Knapsack: take by best ratio of value to cost.
- **“Reach the last index” / “minimum jumps”** – range/horizon extension greedy (Jump Game family).
- **“Schedule jobs with deadlines and profits”** – Job Sequencing: sort by profit, place as late as possible before the deadline.
- **“Neighbouring elements must be strictly greater/smaller”** – local comparison greedy done in two directional passes (Candy).

Whenever you see sorting followed by a single linear sweep with no backtracking, and the problem asks for an optimum count, profit, or feasibility, greedy should be the first technique you test – and you should immediately try to articulate *why* the locally best choice cannot hurt the global answer, since that argument is the real content of a greedy solution, not just the code.

Each section in this chapter follows the same five-part structure: first we restate the problem precisely and identify the keywords that flag it as greedy; then we reason our way, case by case, to the greedy choice and argue informally why it is safe; then we write the algorithm as a numbered procedure and as formal pseudocode; then we trace the algorithm by hand on a concrete example; and finally we give complete, compilable C++ code together with a careful time and space complexity analysis.

1.2 Assign Cookies

Problem Statement. We are given two arrays. The array g of length n holds the *greed factor* of each child: child i will be content only if it receives a cookie of size at least $g[i]$. The array s of length m holds the *size* of each available cookie. Each cookie may be given to at most one child, and each child may receive at most one cookie. We must hand out cookies so as to **maximize the number of content children**, and return that maximum count.

Pattern Recognition

The phrase “*maximize the number of children you can satisfy*” together with two independent arrays that must be *matched* against each other is the signature of a two-pointer greedy assignment problem. Whenever a problem gives you two collections, a one-to-one matching rule of the form “resource j can serve request i if resource j is large enough”, and asks for the maximum number of satisfied requests, the standard reflex is: *sort both arrays, then match the smallest unsatisfied request with the smallest resource capable of satisfying it*. No recursion, no DP table, and no exhaustive matching is needed – sorting already throws away every irrelevant ordering and leaves only a single greedy sweep.

Understanding the problem carefully

Strip away the cookie/child story and the problem is really about two multisets of numbers, g and s . We are allowed to pair an element of g with an element of s exactly when

$$s[j] \geq g[i],$$

and each element of either array can be used in at most one pair. We want the largest possible number of valid pairs.

Notice two things immediately.

1. The actual *order* in which children or cookies are listed in the input is irrelevant to the answer – only the multiset of greed factors and the multiset of cookie sizes matter. This is a strong hint that we are free to reorder (sort) the input without losing anything, which is exactly the kind of freedom a greedy algorithm wants to exploit.
2. A large cookie can always satisfy any child that a smaller cookie could satisfy. So if we are ever tempted to “save” a large cookie for later while a smaller cookie sitting right next to it could have done the same job, we are never making things better, and sometimes we are making things strictly worse, because the large cookie might have been the *only* cookie capable of satisfying some greedier child later on.

Why sorting is the first move

Greedy Choice

Sort g and s in increasing order. Walk through the children from the least greedy to the most greedy. For the current least-greedy unsatisfied child, give them the *smallest cookie that is still large enough* to satisfy them. If no remaining cookie is large enough, this child – and every child after them, since they are all at least as greedy – cannot be the next one satisfied with the cookies left, so we move on without giving up a cookie unnecessarily.

Why does always matching the least-greedy child with the smallest sufficient cookie never hurt? Suppose, for contradiction, that some optimal assignment does *not* give the least-greedy child c the smallest sufficient cookie k available, but instead gives c some larger sufficient cookie

k' , while k is given to a different child c' (or not used at all). Since c is the least greedy child being considered, and k is already known to satisfy c (it is sufficient), and k is the smallest such cookie, swapping k and k' between c and c' keeps c satisfied (since k is still sufficient for c by definition) and does not break c' 's satisfaction either, because we only handed c' a *larger* cookie k' than before – and a larger cookie satisfies anyone a smaller one would. So the swap never decreases the number of satisfied children. This is a standard *exchange argument*: any optimal solution can be rearranged, one swap at a time, into the greedy solution without ever lowering the count of satisfied children. That is precisely what makes the greedy choice safe.

Why two pointers are enough

Once both arrays are sorted, we never need to look backwards. We keep one pointer i over children (starting at the least greedy) and one pointer j over cookies (starting at the smallest cookie).

- If the current cookie $s[j]$ is large enough for the current child $g[i]$, this is the cheapest cookie that can possibly satisfy this child (every cookie before j was too small, by sorted order), so we commit: give it to the child, and advance both i and j .
- If $s[j]$ is too small for $g[i]$, this cookie is too small for *every* remaining child too (they are all at least as greedy as $g[i]$), so it can never be used – discard it by advancing j alone.

Each pointer only ever moves forward, and each comparison either satisfies a child or permanently discards a cookie. There is no benefit to revisiting a discarded cookie or an already-satisfied child, so a single linear sweep after sorting is provably sufficient – no recursion or dynamic programming is needed because there is no overlapping subproblem to memoize; every decision is final the moment it is made.

Algorithm

1. Sort g in increasing order; sort s in increasing order.
2. Set $i \leftarrow 0$ (pointer into g), $j \leftarrow 0$ (pointer into s), $count \leftarrow 0$.
3. While $i < n$ and $j < m$:
 - (a) If $s[j] \geq g[i]$: this cookie satisfies this child. Increment $count$, and advance both $i \leftarrow i + 1$ and $j \leftarrow j + 1$.
 - (b) Otherwise: this cookie is too small for every remaining child. Advance $j \leftarrow j + 1$ only.
4. Return $count$.

Algorithm 1 Assign Cookies

```

1: procedure FINDCONTENTCHILDREN( $g, s$ )
2:   sort  $g$  in increasing order
3:   sort  $s$  in increasing order
4:    $i \leftarrow 0, j \leftarrow 0, count \leftarrow 0$ 
5:   while  $i < \text{length}(g)$  and  $j < \text{length}(s)$  do
6:     if  $s[j] \geq g[i]$  then
7:        $count \leftarrow count + 1$ 
8:        $i \leftarrow i + 1$ 
9:        $j \leftarrow j + 1$ 
10:    else
11:       $j \leftarrow j + 1$ 
12:    end if
13:  end while
14:  return  $count$ 
15: end procedure

```

Dry run

Dry Run. Take $g = [1, 2, 3]$ and $s = [1, 1]$.

Both arrays are already sorted. We initialize $i = 0, j = 0, count = 0$.

- $i = 0, j = 0$: compare $s[0] = 1$ with $g[0] = 1$. Since $1 \geq 1$, the child is satisfied. $count = 1, i \rightarrow 1, j \rightarrow 1$.
- $i = 1, j = 1$: compare $s[1] = 1$ with $g[1] = 2$. Since $1 < 2$, this cookie is too small. $j \rightarrow 2$.
- $j = 2$ equals the length of s , so the loop ends.

The final answer is $count = 1$. Indeed, with cookies of size $[1, 1]$ we can satisfy only the child with the smallest greed factor 1; the children needing 2 and 3 can never be satisfied by a cookie of size 1.

Now take a second example: $g = [1, 2]$ and $s = [1, 2, 3]$.

- $i = 0, j = 0$: $s[0] = 1 \geq g[0] = 1$. Satisfied. $count = 1, i \rightarrow 1, j \rightarrow 1$.
- $i = 1, j = 1$: $s[1] = 2 \geq g[1] = 2$. Satisfied. $count = 2, i \rightarrow 2, j \rightarrow 2$.
- $i = 2$ equals the length of g , so the loop ends.

The final answer is $count = 2$: every child is satisfied, and the leftover cookie of size 3 is simply not needed.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int findContentChildren(vector<int>& g, vector<int>& s) {
5     sort(g.begin(), g.end());
6     sort(s.begin(), s.end());
7
8     int i = 0, j = 0, count = 0;
9     int n = g.size(), m = s.size();

```

```

10
11     while (i < n && j < m) {
12         if (s[j] >= g[i]) {
13             count++;
14             i++;
15             j++;
16         } else {
17             j++;
18         }
19     }
20     return count;
21 }
22
23 int main() {
24     vector<int> g = {1, 2, 3};
25     vector<int> s = {1, 1};
26
27     cout << "Number of content children: "
28           << findContentChildren(g, s) << endl;
29     return 0;
30 }

```

Complexity Analysis

Time Complexity. Sorting g takes $O(n \log n)$ and sorting s takes $O(m \log m)$. The subsequent two-pointer sweep visits each index of g and s at most once, so it costs $O(n + m)$. The sorting step dominates, giving an overall time complexity of

$$O(n \log n + m \log m).$$

Space Complexity. Sorting can be done in place, and we use only the two pointers i, j and the counter as extra variables, so the auxiliary space complexity is $O(1)$ (ignoring the in-place sort's recursion stack, which is $O(\log n)$ in the worst case for typical `std::sort` implementations).

Key Takeaway

When a problem asks you to *match* two sorted resources against each other to maximize the count of satisfied requirements, sort both sides and sweep with two pointers, always trying to satisfy the smallest unsatisfied requirement with the cheapest resource capable of doing so. This pattern – *sort, then greedily match smallest-feasible-to-smallest-need* – recurs throughout greedy algorithms and is worth recognizing on sight.

1.3 Fractional Knapsack Problem

Problem Statement. We are given n items, where the i -th item has a value $val[i]$ and a weight $wt[i]$, and a knapsack of capacity W . Unlike the classical 0/1 knapsack, here we are allowed to take any *fraction* of an item – if we take a fraction $x \in [0, 1]$ of item i , we gain $x \cdot val[i]$ value while consuming $x \cdot wt[i]$ weight. We must choose fractions for each item, subject to the total weight taken not exceeding W , so as to **maximize the total value** carried out of the knapsack.

Pattern Recognition

The single word “*fraction*” (or “*can be broken into smaller units*”) attached to a knapsack-style capacity problem is the loudest signal in this entire chapter: it rules out dynamic programming immediately. The 0/1 knapsack needs DP precisely because an item is all-or-nothing, which creates genuine overlapping subproblems (“did we take item i or not”). The moment items become divisible, that branching disappears – we can always characterize the best item to put in next using a single number, its *value-per-unit-weight ratio*, and greedily fill the knapsack from the best ratio downward. Any time you see “maximize total value/profit under a single weight/capacity/budget constraint, items may be split,” reach for the ratio-greedy pattern.

Understanding the problem carefully

Define, for each item i , its *rate of return*

$$r_i = \frac{val[i]}{wt[i]}.$$

This number tells us exactly how much value we earn per unit of weight we spend if we take item i . Because we are allowed to take fractions, taking item i is like having access to an unlimited (well, limited by $wt[i]$) supply of “value at rate r_i .” If our only constraint is a total weight budget W , then to maximize value, common sense says: spend the budget first on the item with the best rate of return, then the next best, and so on, until the budget runs out – possibly using only part of the last item we touch.

Greedy Choice

Sort the items in decreasing order of $r_i = val[i]/wt[i]$. Process them in this order; for each item, take as much of it as the remaining capacity allows – the whole item if it fits, otherwise exactly the fraction of it that exactly fills the remaining capacity, after which the knapsack is full and we stop.

Why the greedy choice is correct

Suppose, for contradiction, that some optimal solution does not follow this rule – say it takes a positive amount $a > 0$ of a lower-ratio item i while leaving some amount $b > 0$ of a higher-ratio item i' (with $r_{i'} > r_i$) untaken, even though there was room for it. Then we can perform a small exchange: reduce the amount of item i taken by $\varepsilon = \min(a, b)$ in weight, and increase the amount of item i' taken by the same weight ε . The total weight used is unchanged, so the solution remains feasible. But the value changes by

$$\Delta = \varepsilon \cdot r_{i'} - \varepsilon \cdot r_i = \varepsilon(r_{i'} - r_i) > 0,$$

since $r_{i'} > r_i$. So the exchange strictly increases total value, which contradicts the assumption that the original solution was optimal. Hence no optimal solution can avoid fully preferring higher-ratio items over lower-ratio ones whenever weight can be moved between them – which is exactly the greedy rule. Because fractional amounts are allowed, this exchange can always be performed down to an infinitesimal degree, so the greedy strategy is not just a good heuristic here, it is provably optimal, with no gap to close via DP.

Algorithm

1. Compute the ratio $r_i = val[i]/wt[i]$ for every item.

2. Sort the items in decreasing order of r_i .
3. Initialize $remaining \leftarrow W$ and $totalValue \leftarrow 0$.
4. For each item i in sorted order:
 - (a) If $wt[i] \leq remaining$: take the whole item. $totalValue \leftarrow totalValue + val[i]$; $remaining \leftarrow remaining - wt[i]$.
 - (b) Otherwise: take the fraction $f = remaining/wt[i]$ of this item. $totalValue \leftarrow totalValue + f \cdot val[i]$; set $remaining \leftarrow 0$ and stop – the knapsack is now full.
5. Return $totalValue$.

Algorithm 2 Fractional Knapsack

```

1: procedure FRACTIONALKNAPSACK( $val, wt, W$ )
2:   sort items by  $val[i]/wt[i]$  in decreasing order
3:    $remaining \leftarrow W, totalValue \leftarrow 0$ 
4:   for each item  $i$  in sorted order do
5:     if  $remaining = 0$  then
6:       break
7:     end if
8:     if  $wt[i] \leq remaining$  then
9:        $totalValue \leftarrow totalValue + val[i]$ 
10:       $remaining \leftarrow remaining - wt[i]$ 
11:    else
12:       $f \leftarrow remaining/wt[i]$ 
13:       $totalValue \leftarrow totalValue + f \times val[i]$ 
14:       $remaining \leftarrow 0$ 
15:    end if
16:  end for
17:  return  $totalValue$ 
18: end procedure

```

Dry run

Dry Run. Take three items with (val, wt) pairs $(60, 10)$, $(100, 20)$, $(120, 30)$, and knapsack capacity $W = 50$.

First compute ratios:

$$r_1 = 60/10 = 6.0, \quad r_2 = 100/20 = 5.0, \quad r_3 = 120/30 = 4.0.$$

Sorted in decreasing order of ratio, the items are already in the order 1, 2, 3.

- **Item 1** ($val = 60, wt = 10$): remaining capacity is 50, and $10 \leq 50$, so we take all of it. $totalValue = 60, remaining = 40$.
- **Item 2** ($val = 100, wt = 20$): remaining capacity is 40, and $20 \leq 40$, so we take all of it. $totalValue = 160, remaining = 20$.
- **Item 3** ($val = 120, wt = 30$): remaining capacity is 20, but $30 > 20$, so we cannot take the whole item. We take the fraction $f = 20/30 = 2/3$. This contributes $\frac{2}{3} \times 120 = 80$ value. $totalValue = 160 + 80 = 240, remaining = 0$.

The knapsack is now full, so we stop. The maximum value obtainable is 240.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Item {
5     int value;
6     int weight;
7 };
8
9 double fractionalKnapsack(int W, vector<Item>& items) {
10     // Sort items by value/weight ratio in decreasing order
11     sort(items.begin(), items.end(), [](const Item& a, const Item& b) {
12         double r1 = (double)a.value / (double)a.weight;
13         double r2 = (double)b.value / (double)b.weight;
14         return r1 > r2;
15     });
16
17     int remaining = W;
18     double totalValue = 0.0;
19
20     for (auto& item : items) {
21         if (remaining == 0) break;
22
23         if (item.weight <= remaining) {
24             totalValue += item.value;
25             remaining -= item.weight;
26         } else {
27             double fraction = (double)remaining / (double)item.weight;
28             totalValue += fraction * item.value;
29             remaining = 0;
30         }
31     }
32     return totalValue;
33 }
34
35 int main() {
36     vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};
37     int W = 50;
38
39     cout << "Maximum value: " << fractionalKnapsack(W, items) << endl;
40     return 0;
41 }

```

Complexity Analysis

Time Complexity. Sorting the n items by ratio costs $O(n \log n)$. The subsequent single pass over the sorted items costs $O(n)$. Hence the total time complexity is

$$O(n \log n).$$

Space Complexity. Sorting is performed in place (or with a small $O(n)$ auxiliary array depending on implementation), and we use only a constant number of extra scalar variables, giving auxiliary space complexity $O(1)$ beyond the input storage (or $O(n)$ if we count the input array itself, and $O(\log n)$ for the sort's recursion stack).

Key Takeaway

Divisibility is the tell. The instant a knapsack-style problem allows *fractions* of an item, the all-or-nothing branching that forces dynamic programming in 0/1 knapsack disappears, and a single value-per-weight ratio becomes a perfect, provably optimal greedy ranking. Contrast this sharply with Section 1.2: there, sorting gave us a feasibility-matching greedy; here, sorting by ratio gives us a value-maximizing greedy, but the underlying exchange-argument justification is the same family of reasoning.

1.4 Greedy Algorithm to Find Minimum Number of Coins

Problem Statement. We are given a target amount V and an unlimited supply of coins (and notes) of standard denominations – in the version used on the sheet, the Indian currency denominations

$$\{1, 2, 5, 10, 20, 50, 100, 500, 1000\}.$$

We must choose a multiset of these denominations whose values sum exactly to V , using as **few coins/notes as possible**, and report that minimum count (or the coins themselves).

Pattern Recognition

“Minimum number of coins/notes to make a given amount,” when the denomination set is a small, fixed, real-world currency system, is the textbook greedy “always use the largest denomination that fits” problem. The crucial, easy-to-miss keyword here is which *specific* denomination set is being used. If the problem instead says “*given an arbitrary list of coin denominations*” (as in the famous LeetCode “Coin Change” problem), the greedy approach can silently give a wrong answer, and you must switch to dynamic programming. Recognizing *which* of these two near-identical-looking problems you are facing is itself the key skill being tested here.

Understanding the problem carefully

For a fixed real-world currency system such as the Indian denominations above, there is a special structural property: each denomination is large enough, relative to the ones below it, that repeatedly taking the largest denomination not exceeding the remaining amount never forces us into a worse total count later. Concretely, every denomination in this list is at least double the previous one except for the jumps $1 \rightarrow 2$, $2 \rightarrow 5$, $5 \rightarrow 10$, $10 \rightarrow 20$, $20 \rightarrow 50$ – and a short case check (which we will not belabor here, since it is a property of *this particular* currency system rather than something we derive from first principles) confirms that this set is a so-called *canonical coin system*: the greedy algorithm always produces an optimal (minimum-count) representation for it.

Greedy Choice

Sort the denominations in decreasing order. Repeatedly subtract the largest denomination that does not exceed the remaining amount, counting how many times each denomination is used, until the remaining amount reaches zero.

Why greedy works here, and why it can fail elsewhere

The intuitive argument for why greedy is safe here is this: using one coin of the largest available denomination d removes as much value as possible in a single coin. If using a smaller

denomination instead could ever lead to a strictly smaller total coin count, it would have to be because the smaller coins “line up” more efficiently with the rest of the amount – but because each denomination in this system divides evenly into, or is closely compatible with, the next one up, no such inefficiency can arise. This is *not* a universal truth about coin systems; it is a property that must be verified for the specific set of denominations given.

To see the failure mode, suppose instead the denominations were $\{1, 3, 4\}$ and the target amount were $V = 6$. Greedy would take one coin of 4, leaving 2, then two coins of 1, for a total of three coins: $4 + 1 + 1 = 6$. But the optimal answer uses two coins of 3: $3 + 3 = 6$, a total of only two coins. Here greedy fails because 4 is not a “safe” large denomination relative to 3 – taking it first blocks the more efficient combination. This is exactly why the general “Coin Change: minimum coins” problem (with an arbitrary denomination set) requires dynamic programming, while this specific problem, restricted to real-world currency denominations, is safe for greedy. Always check which version of the problem you are being asked before reaching for greedy.

Algorithm

1. Store the denominations in decreasing order: 1000, 500, 100, 50, 20, 10, 5, 2, 1.
2. Initialize $remaining \leftarrow V$ and an empty result list.
3. For each denomination d in decreasing order:
 - (a) While $d \leq remaining$: append d to the result list, and set $remaining \leftarrow remaining - d$.
4. Return the result list (its length is the minimum coin count).

Algorithm 3 Minimum Number of Coins (Canonical System)

```

1: procedure MINIMUMCOINS( $V$ )
2:    $denoms \leftarrow [1000, 500, 100, 50, 20, 10, 5, 2, 1]$ 
3:    $remaining \leftarrow V$ 
4:    $result \leftarrow$  empty list
5:   for each  $d$  in  $denoms$  do
6:     while  $d \leq remaining$  do
7:       append  $d$  to  $result$ 
8:        $remaining \leftarrow remaining - d$ 
9:     end while
10:  end for
11:  return  $result$ 
12: end procedure

```

Dry run

Dry Run. Take $V = 93$.

- $d = 1000$: $1000 > 93$, skip.
- $d = 500$: $500 > 93$, skip.
- $d = 100$: $100 > 93$, skip.
- $d = 50$: $50 \leq 93$. Take one 50. $remaining = 43$. $50 \leq 43$? No, stop this denomination.
- $d = 20$: $20 \leq 43$. Take one 20. $remaining = 23$. $20 \leq 23$. Take another 20. $remaining = 3$. $20 \leq 3$? No, stop.

- $d = 10$: $10 > 3$, skip.
- $d = 5$: $5 > 3$, skip.
- $d = 2$: $2 \leq 3$. Take one 2. $remaining = 1$. $2 \leq 1$? No, stop.
- $d = 1$: $1 \leq 1$. Take one 1. $remaining = 0$.

The coins used are $\{50, 20, 20, 2, 1\}$, a total of 5 coins, and indeed $50 + 20 + 20 + 2 + 1 = 93$. No combination of these denominations can make 93 with fewer than 5 coins.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> minimumCoins(int V) {
5     vector<int> denoms = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
6     vector<int> result;
7
8     int remaining = V;
9     for (int d : denoms) {
10         while (d <= remaining) {
11             result.push_back(d);
12             remaining -= d;
13         }
14     }
15     return result;
16 }
17
18 int main() {
19     int V = 93;
20     vector<int> coins = minimumCoins(V);
21
22     cout << "Number of coins used: " << coins.size() << endl;
23     cout << "Coins: ";
24     for (int c : coins) cout << c << " ";
25     cout << endl;
26     return 0;
27 }

```

Complexity Analysis

Time Complexity. There are only 9 fixed denominations, so the outer loop is $O(1)$ in the number of denominations. The inner **while** loop, across all denominations, executes at most once per coin actually placed into the result, and the number of coins used is $O(\log V)$ in the worst case for this denomination system (since denominations grow roughly geometrically). So the overall time complexity is

$$O(\log V),$$

effectively a constant for realistic input sizes. **Space Complexity.** We store the result list of coins used, which has length $O(\log V)$, so the space complexity is $O(\log V)$ for the output (or $O(1)$ auxiliary space if we only need the *count* rather than the list of coins itself).

Key Takeaway

Greedy coin selection is only correct for *canonical* denomination systems – ones where the structure of the denominations guarantees that taking the largest coin first never blocks an optimal solution. Always distinguish this fixed-currency version of the problem from the general “minimum coins from an arbitrary denomination list” problem, which has no such guarantee and requires dynamic programming (typically an unbounded knapsack-style DP over amounts $0, \dots, V$).

1.5 Lemonade Change

Problem Statement. Customers stand in a queue to buy lemonade, and each glass costs exactly \$5. We are given an array *bills*, where *bills*[*i*] is the bill the *i*-th customer pays with – always a \$5, \$10, or \$20 note. We start with no change at all. For every customer, **immediately** after they pay, we must hand back the correct change using bills we are currently holding (we cannot use a bill the same customer just gave us as their own change). We must process customers strictly in the given order and decide whether it is possible to give correct change to *every* customer; return **true** if so, and **false** the moment it becomes impossible.

Pattern Recognition

Two phrases give this one away. First, “*customers in a queue, processed one at a time, in order*” tells us this is an *online simulation* – we are never allowed to look ahead or rearrange the input, so any algorithm that tries to plan globally (sort, search over future choices) is solving the wrong problem. Second, “*immediately give change*” combined with only three possible bill values means that at any moment the entire relevant state of the world is just two small counters: how many \$5 notes and how many \$10 notes we are holding (\$20 notes are never given as change, so we never need to count them). Whenever a problem reduces to “maintain a couple of counters and react instantly to each new input,” it is a simulation problem with, at most, one local greedy decision per step – not a search problem.

Understanding the problem carefully

Since lemonade costs \$5, the change owed depends only on what the customer pays with:

- paying with \$5 $\Rightarrow$ no change owed;
- paying with \$10 $\Rightarrow$ owe \$5;
- paying with \$20 $\Rightarrow$ owe \$15.

A \$5 note can only ever be used to pay back \$5 of change, never \$15 worth by itself, so it has exactly one job: covering a \$10 customer, or participating in the \$15 change for a \$20 customer. A \$10 note is even more restricted: it is completely useless for a \$10 customer (we cannot give a customer back their own denomination as part of correct change composition here – effectively, the only practical use of a \$10 note as change is as part of the \$15 owed to a \$20 customer, paired with one \$5 note). This asymmetry is the heart of the problem: **\$5 notes are strictly more flexible than \$10 notes**, because a \$5 note can help pay back a \$10 customer *or* a \$20 customer, while a \$10 note can only ever help pay back a \$20 customer.

The only real decision point

Customers paying with \$5 require no decision – we simply keep the note. Customers paying with \$10 also require no decision – if we have a \$5 note we must use it (it is the only way to make \$5 change), and if we do not, failure is immediate and unavoidable, since no other combination of notes we might be holding sums to exactly \$5 with the unit notes available.

The only point where two different valid strategies exist is a customer paying with \$20, who is owed \$15. There are exactly two ways to assemble \$15 from the notes we hold:

$$\$10 + \$5 \quad \text{or} \quad \$5 + \$5 + \$5.$$

Greedy Choice

At a \$20 customer, if we hold at least one \$10 note and at least one \$5 note, prefer to give **\$10 + \$5** over **\$5 + \$5 + \$5**. Only fall back to spending three \$5 notes if we do not have a spare \$10 note available.

Why this greedy choice is safe

Both options return exactly \$15, so the customer is equally satisfied either way; the difference is entirely in what we are left holding afterwards. Giving \$10+\$5 costs us one \$5 note. Giving \$5+\$5+\$5 costs us three \$5 notes. Since \$5 notes are the more broadly useful resource – the *only* resource capable of covering a future \$10 customer, and a necessary ingredient (alongside a \$10 note) for covering a future \$20 customer – depleting three of them at once when we could have depleted only one is never better for the future, and is sometimes strictly worse: it might leave us without enough \$5 notes to cover a \$10 customer who arrives later. Formally, suppose an alternative strategy chooses \$5+\$5+\$5 for some \$20 customer while a \$10 note sits unused in hand. We can always replace that choice with \$10+\$5 instead, ending with two extra \$5 notes in hand compared to the alternative strategy at every subsequent point in the queue, and strictly more \$5 notes in hand can never cause a future failure that would not also have occurred with fewer – so the greedy choice dominates. \$10 notes, by contrast, have no other use, so spending one whenever possible is pure upside.

Algorithm

1. Initialize *five* $\leftarrow$ 0, *ten* $\leftarrow$ 0.
2. For each bill *b* in *bills*, in order:
 - (a) If *b* = 5: *five* $\leftarrow$ *five* + 1.
 - (b) If *b* = 10: if *five* = 0, return **false**; otherwise *five* $\leftarrow$ *five* - 1 and *ten* $\leftarrow$ *ten* + 1.
 - (c) If *b* = 20:
 - if *ten* $\geq$ 1 and *five* $\geq$ 1: subtract one from each;
 - else if *five* $\geq$ 3: subtract three from *five*;
 - else: return **false**.
3. If every customer was processed without returning **false**, return **true**.

Algorithm 4 Lemonade Change

```

1: procedure LEMONADECHANGE(bills)
2:   five  $\leftarrow$  0, ten  $\leftarrow$  0
3:   for each b in bills do
4:     if b = 5 then
5:       five  $\leftarrow$  five + 1
6:     else if b = 10 then
7:       if five = 0 then return false
8:       end if
9:       five  $\leftarrow$  five - 1
10:      ten  $\leftarrow$  ten + 1
11:     else  $\triangleright b = 20$ 
12:       if ten  $\geq$  1 and five  $\geq$  1 then
13:         ten  $\leftarrow$  ten - 1
14:         five  $\leftarrow$  five - 1
15:       else if five  $\geq$  3 then
16:         five  $\leftarrow$  five - 3
17:       else
18:         return false
19:       end if
20:     end if
21:   end for
22:   return true
23: end procedure

```

Dry run

Dry Run. Take *bills* = [5, 5, 5, 10, 20].

Start with *five* = 0, *ten* = 0.

- Bill 5: *five* = 1.
- Bill 5: *five* = 2.
- Bill 5: *five* = 3.
- Bill 10: *five* > 0, so give one \$5. *five* = 2, *ten* = 1.
- Bill 20: *ten* $\geq$ 1 and *five* $\geq$ 1, so give \$10+\$5. *five* = 1, *ten* = 0.

All five customers are served, so the answer is **true**.

Now take *bills* = [5, 5, 10, 10, 20].

- Bill 5: *five* = 1.
- Bill 5: *five* = 2.
- Bill 10: give one \$5. *five* = 1, *ten* = 1.
- Bill 10: give one \$5. *five* = 0, *ten* = 2.
- Bill 20: need *ten* $\geq$ 1 and *five* $\geq$ 1, but *five* = 0. Fall back to *five* $\geq$ 3, but again *five* = 0. Neither option works, so return **false**.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool lemonadeChange(vector<int>& bills) {
5     int five = 0, ten = 0;
6
7     for (int bill : bills) {
8         if (bill == 5) {
9             five++;
10        } else if (bill == 10) {
11            if (five == 0) return false;
12            five--;
13            ten++;
14        } else { // bill == 20
15            if (ten >= 1 && five >= 1) {
16                ten--;
17                five--;
18            } else if (five >= 3) {
19                five -= 3;
20            } else {
21                return false;
22            }
23        }
24    }
25    return true;
26 }
27
28 int main() {
29     vector<int> bills = {5, 5, 5, 10, 20};
30     cout << (lemonadeChange(bills) ? "true" : "false") << endl;
31     return 0;
32 }

```

Complexity Analysis

Time Complexity. Each of the n customers is processed exactly once, with only constant work (a couple of comparisons and increments) per customer, giving

$$O(n).$$

Space Complexity. We track only the two counters *five* and *ten*, so the space complexity is $O(1)$.

Key Takeaway

When several local choices return the same immediate result (here, both change-giving options return exactly \$15), break the tie by asking which choice preserves the resource that is more broadly useful for the future. Here, \$5 notes serve two future needs while \$10 notes serve only one, so spending a \$10 note whenever possible is always at least as good, and sometimes strictly necessary, for surviving the rest of the queue.

1.6 Valid Parenthesis String

Problem Statement. We are given a string s consisting only of the characters $'('$, $)'$, and $'*'$. The character $'*'$ is a wildcard: each occurrence of it may be treated, independently, as either $'('$, $)'$, or the empty string. We must decide whether there is *some* way to resolve every $'*'$ so that the resulting string is a valid parenthesis sequence (every prefix has at least as many $'('$ as $)'$, and the total counts of $'('$ and $)'$ are equal).

Pattern Recognition

The phrase “each wildcard may independently be one of several things; does *some* assignment make the whole thing valid” looks, at first glance, like it demands trying all 3^k assignments of the k wildcards – an exponential brute force. But the underlying validity condition for parentheses is itself expressible as a simple running balance (“open count never goes negative, and ends at zero”), and whenever a validity condition can be tracked by a *running numeric range* rather than an exact value, that is a strong signal that we can collapse all exponentially many choices into tracking just the minimum and maximum possible value of that running quantity at each step – a classic “track an interval of reachable states” greedy, no recursion or backtracking required.

Understanding the problem carefully

For a string with only $'('$ and $)'$, validity is checked by maintaining a single counter, the *open balance*: add 1 for $'('$, subtract 1 for $)'$, and the string is valid exactly when the balance never goes negative and ends at exactly 0.

With $'*'$ in the mix, the open balance after processing a prefix is no longer a single number – it is a *range* of possible numbers, depending on how each $'*'$ so far was resolved. Let lo be the smallest possible balance consistent with some resolution of the wildcards seen so far, and hi be the largest possible balance. We update this range character by character:

- On $'('$: both the smallest and largest possible balance increase by 1, so $lo += 1$, $hi += 1$.
- On $)'$: both decrease by 1, so $lo -= 1$, $hi -= 1$.
- On $'*'$: in the most pessimistic resolution (treat it as $)'$, the balance decreases by 1, so $lo -= 1$. In the most optimistic resolution (treat it as $'('$), the balance increases by 1, so $hi += 1$. (Treating it as empty would leave the balance unchanged, which always lies between these two extremes, so it never needs separate tracking.)

Greedy Choice

At every position, maintain only the *interval* $[lo, hi]$ of balances that are achievable by *some* resolution of the wildcards seen so far – not every individual achievable value, just the two extremes. Whenever hi drops below 0, no resolution can recover (the balance has gone irrecoverably negative even in the best case), so we can answer **false** immediately. Whenever lo drops below 0, clamp it to 0, because a balance can never validly be negative, so any resolution that would have produced a negative running balance is simply discarded from consideration – it is never the right choice for an eventual valid string, so we do not need to track it.

Why tracking only the interval is enough

It might seem like we are throwing away information by tracking only $[lo, hi]$ rather than the full set of achievable balances. But notice that the set of achievable balances after processing

a prefix is always a contiguous range of integers with the *same parity step of 1* apart (because every character changes the balance by exactly -1 , 0 , or $+1$, and a `'*'` contributes all three options, which fills in every integer between the all-`' ) '` extreme and the all-`' ( '` extreme). So the reachable set truly is an interval, and tracking its endpoints loses nothing. At the very end, the string can be made valid if and only if 0 lies in the final interval – but since we always clamp lo to be at least 0 throughout, checking that the final $lo = 0$ is equivalent to checking that 0 is reachable, because if 0 were reachable, clamping would have kept lo pinned at (or below, then clamped to) 0 rather than drifting strictly positive.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow 0$.
2. For each character c in s :
 - (a) If $c = '('$: $lo \leftarrow lo + 1$, $hi \leftarrow hi + 1$.
 - (b) If $c = ')'$: $lo \leftarrow lo - 1$, $hi \leftarrow hi - 1$.
 - (c) If $c = '*'$: $lo \leftarrow lo - 1$, $hi \leftarrow hi + 1$.
 - (d) If $hi < 0$: return **false** immediately.
 - (e) If $lo < 0$: clamp $lo \leftarrow 0$.
3. After processing the whole string, return ($lo = 0$).

Algorithm 5 Valid Parenthesis String

```

1: procedure CHECKVALIDSTRING( $s$ )
2:    $lo \leftarrow 0$ ,  $hi \leftarrow 0$ 
3:   for each character  $c$  in  $s$  do
4:     if  $c = '('$  then
5:        $lo \leftarrow lo + 1$ ,  $hi \leftarrow hi + 1$ 
6:     else if  $c = ')'$  then
7:        $lo \leftarrow lo - 1$ ,  $hi \leftarrow hi - 1$ 
8:     else  $\triangleright c = '*'$ 
9:        $lo \leftarrow lo - 1$ ,  $hi \leftarrow hi + 1$ 
10:    end if
11:    if  $hi < 0$  then
12:      return false
13:    end if
14:    if  $lo < 0$  then
15:       $lo \leftarrow 0$ 
16:    end if
17:  end for
18:  return ( $lo = 0$ )
19: end procedure

```

Dry run

Dry Run. Take $s = "(*)"$.
Start with $lo = 0$, $hi = 0$.

- $c = '('$: $lo = 1$, $hi = 1$.

- $c = '*'$: $lo = 0$, $hi = 2$.
- $c = ')'$: $lo = -1 \rightarrow$ clamp to 0, $hi = 1$.
- $c = ')'$: $lo = -1 \rightarrow$ clamp to 0, $hi = 0$.

At no point did hi go negative, and the final $lo = 0$, so the answer is **true**. We can confirm this directly: resolving the $'*'$ as $'('$ turns s into $"(())"$, whose balances are 1, 2, 1, 0 – never negative, ending at 0 – so this is indeed a valid resolution, and the algorithm's **true** verdict is justified.

Now take $s = "(*)"$, which appends one extra unmatched $'('$ to the previous string.

- As above, after $"(*)"$: $lo = 0$, $hi = 0$.
- $c = '('$: $lo = 1$, $hi = 1$.

Final $lo = 1 \neq 0$, so the answer is **false** – correctly, since the trailing unmatched $'('$ can never be closed by anything after it.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool checkValidString(string s) {
5     int lo = 0, hi = 0;
6
7     for (char c : s) {
8         if (c == '(') {
9             lo++;
10            hi++;
11        } else if (c == ')') {
12            lo--;
13            hi--;
14        } else { // c == '*'
15            lo--;
16            hi++;
17        }
18
19        if (hi < 0) return false;
20        if (lo < 0) lo = 0;
21    }
22    return lo == 0;
23 }
24
25 int main() {
26     string s = "(*)";
27     cout << (checkValidString(s) ? "true" : "false") << endl;
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. We scan the string once, doing constant work per character, giving

$$O(n).$$

Space Complexity. We maintain only the two integers lo and hi , so the space complexity

is $O(1)$.

Key Takeaway

Whenever a validity check can be expressed as a single running numeric quantity that must stay within bounds, and the input introduces wildcards with several possible numeric effects, try tracking the *interval* of reachable values for that quantity instead of branching over every combination of wildcard resolutions. This turns an exponential “try every assignment” search into a single linear greedy sweep.

1.7 N Meetings in One Room

Problem Statement. We are given n meetings, where the i -th meeting has a start time $start[i]$ and a finish time $end[i]$. Only one meeting room is available, so a meeting occupies the room for its entire duration $[start[i], end[i]]$, and the room becomes free only strictly after $end[i]$ – so two meetings can both be held in the room only if one of them starts strictly after the other has ended. We must select the **maximum number of non-overlapping meetings** that can be held in this single room.

Pattern Recognition

“One resource (a room), many requests each with a start and an end, maximize the number of non-conflicting requests you can serve” is the exact signature of the classical *Activity Selection Problem*, the flagship example of interval scheduling. The keyword combination to listen for is “maximum number of non-overlapping intervals” together with a *single shared resource*. The unintuitive but crucial insight that makes this greedy rather than a search problem is that you should sort by **finish time**, not by start time and not by duration – a fact that is worth memorizing because it differs from a tempting but incorrect first instinct.

Understanding the problem carefully

It is tempting to think that scheduling the *shortest* meetings first, or the *earliest-starting* meetings first, would leave the most room for others. Both of these intuitions are wrong, and small examples expose the flaw immediately. Consider two meetings, $A = [1, 100]$ and $B = [2, 3]$. Sorting by start time would consider A first (it starts earliest); taking A blocks B entirely, leaving only 1 meeting scheduled, whereas taking B instead would have left room afterward for further meetings starting after time 3. Sorting by duration would favor B here by luck, but that is not a reliable rule either – a short meeting that starts very early and a slightly longer meeting that finishes only one unit later can still flip which choice is better, depending on what comes after.

What actually matters is not how long a meeting takes, nor when it starts, but *when it frees up the room for everyone else*. A meeting that finishes early frees the room early, leaving the largest possible remaining time window for future meetings – regardless of how late it started or how long it lasted.

Greedy Choice

Sort all meetings by finish time, in increasing order. Always select the next meeting (in this sorted order) whose start time is strictly after the finish time of the most recently selected meeting.

Why sorting by finish time is correct

This is the standard exchange-argument proof used for activity selection. Let m_1 be the meeting with the earliest finish time among all n meetings. We claim there is always an optimal solution that includes m_1 . Suppose some optimal solution O does not include m_1 , but includes some other meeting m' as its first (earliest-finishing) chosen meeting instead. Since m_1 has the globally earliest finish time of *all* meetings, in particular $\text{end}[m_1] \leq \text{end}[m']$. Replacing m' with m_1 inside O cannot create any new conflict with the meetings that come after m' in O , because m_1 frees the room *no later* than m' did. So this swap produces another solution that is still feasible and has exactly the same number of meetings as O – meaning some optimal solution does include m_1 . Once m_1 is fixed as selected, the remaining problem is exactly the same problem, restricted to meetings that start after $\text{end}[m_1]$, so the same argument applies recursively (or, in iterative form, repeatedly) to the next-earliest-finishing remaining meeting, and so on. This is what justifies scanning the finish-time-sorted list exactly once, greedily picking whichever next meeting is compatible with the last one picked.

Algorithm

1. Sort all n meetings by $\text{end}[i]$ in increasing order.
2. Initialize $\text{count} \leftarrow 1$ and $\text{lastEnd} \leftarrow \text{end}$ of the first meeting in sorted order (the very first meeting is always selected, since nothing has been scheduled yet to conflict with it).
3. For each remaining meeting i in sorted order, starting from the second one:
 - (a) If $\text{start}[i] > \text{lastEnd}$: select this meeting. $\text{count} \leftarrow \text{count} + 1$; $\text{lastEnd} \leftarrow \text{end}[i]$.
 - (b) Otherwise: skip this meeting (it conflicts with the last selected meeting).
4. Return count .

Algorithm 6 Maximum Meetings in One Room

```

1: procedure MAXMEETINGS( $\text{start}, \text{end}$ )
2:   sort meetings by  $\text{end}[i]$  in increasing order
3:    $\text{count} \leftarrow 1$ 
4:    $\text{lastEnd} \leftarrow \text{end}[0]$  ▷ always take the first meeting
5:   for  $i \leftarrow 1$  to  $n - 1$  do
6:     if  $\text{start}[i] > \text{lastEnd}$  then
7:        $\text{count} \leftarrow \text{count} + 1$ 
8:        $\text{lastEnd} \leftarrow \text{end}[i]$ 
9:     end if
10:  end for
11:  return  $\text{count}$ 
12: end procedure

```

Dry run

Dry Run. Take meetings with $(\text{start}, \text{end})$ pairs: $(1, 2), (3, 4), (0, 6), (5, 7), (8, 9), (5, 9)$.
 Sorting by finish time gives the order:

$(1, 2), (3, 4), (0, 6), (5, 7), (8, 9), (5, 9)$.

- Select $(1, 2)$ unconditionally. $\text{count} = 1$, $\text{lastEnd} = 2$.
- $(3, 4)$: $\text{start} = 3 > \text{lastEnd} = 2$. Select. $\text{count} = 2$, $\text{lastEnd} = 4$.

- (0,6): $start = 0 \not> lastEnd = 4$. Skip.
- (5,7): $start = 5 > lastEnd = 4$. Select. $count = 3$, $lastEnd = 7$.
- (8,9): $start = 8 > lastEnd = 7$. Select. $count = 4$, $lastEnd = 9$.
- (5,9): $start = 5 \not> lastEnd = 9$. Skip.

The maximum number of meetings that can be held is $\boxed{4}$, namely (1, 2), (3, 4), (5, 7), (8, 9).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int maxMeetings(vector<int>& start, vector<int>& end) {
5     int n = start.size();
6     vector<pair<int,int>> meetings(n);
7     for (int i = 0; i < n; i++) {
8         meetings[i] = {end[i], start[i]};
9     }
10
11     sort(meetings.begin(), meetings.end()); // sorts by end time first
12
13     int count = 1;
14     int lastEnd = meetings[0].first;
15
16     for (int i = 1; i < n; i++) {
17         int s = meetings[i].second;
18         int e = meetings[i].first;
19         if (s > lastEnd) {
20             count++;
21             lastEnd = e;
22         }
23     }
24     return count;
25 }
26
27 int main() {
28     vector<int> start = {1, 3, 0, 5, 8, 5};
29     vector<int> end = {2, 4, 6, 7, 9, 9};
30
31     cout << "Maximum meetings: " << maxMeetings(start, end) << endl;
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. Sorting the n meetings by finish time costs $O(n \log n)$. The subsequent single linear pass costs $O(n)$. So the overall time complexity is

$$O(n \log n).$$

Space Complexity. We use an auxiliary array of n pairs to sort by finish time, giving $O(n)$ space (this can be reduced to $O(1)$ auxiliary space if the input is given as an array of pairs already, sorted in place).

Key Takeaway

For interval-scheduling problems with a single shared resource, always sort by **finish time**, never by start time or duration. The meeting that frees the resource soonest is always at least as good a first choice as any other meeting, by a direct exchange argument – and this single fact is the entire content of the activity selection problem, a pattern that reappears throughout the rest of this chapter under different names.

1.8 Jump Game

Problem Statement. We are given an array *nums* of length *n*, where *nums*[*i*] is the maximum number of steps we are allowed to jump forward from index *i* (we may jump anywhere from 1 up to *nums*[*i*] steps, or choose to jump fewer). Starting at index 0, we must decide whether it is possible to reach index *n* – 1 **at all**, following any sequence of valid jumps. Return **true** or **false**.

Pattern Recognition

“Can you reach the last index, given a maximum jump length at each position” is the defining example of a *reachability-horizon* greedy. The keyword to notice is that we only need a yes/no feasibility answer, not the actual sequence of jumps or the minimum number of jumps – that distinction matters, because it means we never need to track *how* we got somewhere, only the single number *how far forward can we possibly see from everything visited so far*. Whenever a problem asks “can position *X* be reached” under a rule where each position extends your reach by some amount, track a running *farthest reachable index* and sweep once, left to right.

Understanding the problem carefully

Define *farthest* as the largest index reachable using any combination of jumps from indices we have already confirmed are reachable, considering only indices up to the one we are currently examining. As we scan left to right through index $i = 0, 1, \dots, n - 1$:

- If $i \leq \textit{farthest}$, then index *i* itself is reachable (it lies within the horizon established by earlier indices), so we are allowed to jump from it: *farthest* can potentially be extended to $i + \textit{nums}[i]$.
- If $i > \textit{farthest}$, then index *i* is *not* reachable by anything before it, and since we are scanning strictly left to right, nothing later can make index *i* reachable either – the whole array up to this point has failed to bridge the gap, so the answer is immediately **false**.

Greedy Choice

Maintain a single running value *farthest*, the best (largest) index reachable using everything examined so far. At each index *i*, first check whether *i* itself is still within *farthest* (if not, fail immediately); otherwise, update $\textit{farthest} \leftarrow \max(\textit{farthest}, i + \textit{nums}[i])$. If at any point $\textit{farthest} \geq n - 1$, the last index is reachable and we may stop early and answer **true**.

Why tracking only the farthest reach is correct

This works because reachability is monotonic in a useful sense: if index $j \leq \textit{farthest}$ is reachable, every index from 0 up to *farthest* is also reachable, because we can always choose to jump *short of*

our maximum allowance at any earlier reachable index to land exactly where we want (jumping any number of steps from 1 up to the maximum is allowed, so every intermediate landing spot is achievable, not just the farthest one). This means we never need to track the *set* of reachable indices, nor which specific path of jumps was used – only the single supremum of that set, *farthest*, fully describes everything we need to know about reachability at each step. There is no benefit to "saving" a long jump for later or being clever about which specific jump length to use, because every index up to the maximum reach is already reachable regardless of which path got us there. This collapses what could look like an exponential search over jump sequences into one linear pass that only ever needs to remember one number.

Algorithm

1. Initialize $farthest \leftarrow 0$.
2. For $i \leftarrow 0$ to $n - 1$:
 - (a) If $i > farthest$: return **false** (index i , and therefore the rest of the array, is unreachable).
 - (b) $farthest \leftarrow \max(farthest, i + nums[i])$.
 - (c) If $farthest \geq n - 1$: return **true** early.
3. If the loop completes without returning **false**, return **true**.

Algorithm 7 Jump Game (Reachability)

```

1: procedure CANJUMP( $nums$ )
2:    $farthest \leftarrow 0$ 
3:    $n \leftarrow \text{length}(nums)$ 
4:   for  $i \leftarrow 0$  to  $n - 1$  do
5:     if  $i > farthest$  then
6:       return false
7:     end if
8:      $farthest \leftarrow \max(farthest, i + nums[i])$ 
9:     if  $farthest \geq n - 1$  then
10:      return true
11:    end if
12:  end for
13:  return true
14: end procedure

```

Dry run

Dry Run. Take $nums = [2, 3, 1, 1, 4]$ ($n = 5$, last index = 4).

- $i = 0$: $0 \leq farthest = 0$. Update $farthest = \max(0, 0 + 2) = 2$.
- $i = 1$: $1 \leq 2$. Update $farthest = \max(2, 1 + 3) = 4$. Since $4 \geq n - 1 = 4$, return **true** immediately.

The answer is **true**: from index 0 we can jump 2 steps to index 2... but more directly, from index 1 (reachable, since $1 \leq 2$) a jump of 3 steps reaches index 4, the last index.

Now take $nums = [3, 2, 1, 0, 4]$ ($n = 5$, last index = 4).

- $i = 0$: $0 \leq 0$. $farthest = \max(0, 0 + 3) = 3$.

- $i = 1$: $1 \leq 3$. $farthest = \max(3, 1 + 2) = 3$.
- $i = 2$: $2 \leq 3$. $farthest = \max(3, 2 + 1) = 3$.
- $i = 3$: $3 \leq 3$. $farthest = \max(3, 3 + 0) = 3$.
- $i = 4$: $4 > farthest = 3$. Return **false**.

The answer is **false**: index 3 has a jump length of 0, and everything before it can reach at most index 3, so index 4 – and the useful 4 sitting right there at index 4 – can never be reached.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool canJump(vector<int>& nums) {
5     int n = nums.size();
6     int farthest = 0;
7
8     for (int i = 0; i < n; i++) {
9         if (i > farthest) return false;
10        farthest = max(farthest, i + nums[i]);
11        if (farthest >= n - 1) return true;
12    }
13    return true;
14 }
15
16 int main() {
17     vector<int> nums = {2, 3, 1, 1, 4};
18     cout << (canJump(nums) ? "true" : "false") << endl;
19     return 0;
20 }

```

Complexity Analysis

Time Complexity. We scan the array once, doing constant work per index, so the time complexity is

$$O(n).$$

Space Complexity. We maintain only the single variable *farthest*, so the space complexity is $O(1)$.

Key Takeaway

When a problem only asks *whether* some destination is reachable under a rule that extends your reach from any currently-reachable position, you rarely need to enumerate paths. Track the single furthest horizon reachable so far and sweep once – this is strictly more efficient than recursion or BFS/DP over individual positions, and it generalizes directly to the minimum-jumps variant in the next section.

1.9 Jump Game II

Problem Statement. We are given an array *nums* of length n , where $nums[i]$ is the maximum number of steps we may jump forward from index i . It is guaranteed that index $n - 1$ is reachable

from index 0. Starting at index 0, we must find the **minimum number of jumps** required to reach index $n - 1$.

Pattern Recognition

This is the natural sequel to Section 1.8: the keywords change from “*is it possible*” to “*minimum number of jumps*,” which upgrades the problem from a feasibility check to an optimization. Whenever a reachability-horizon greedy problem is upgraded to ask for the *minimum number of steps/jumps/levels* to get somewhere, that is a strong signal to think in terms of *BFS levels* – here, each “level” is exactly one jump, and we want to know how many levels it takes for the frontier of reachable positions to engulf the last index. We will see that this BFS-by-levels idea can be implemented without an actual queue, using two horizon variables instead.

Understanding the problem carefully

Think of the jumps as expanding in waves, exactly like breadth-first search on a graph where node i has an edge to every node in $[i+1, i + \text{nums}[i]]$. After 0 jumps, we have only reached index 0. After 1 jump, we have reached every index reachable directly from index 0 – a contiguous range $[1, \text{nums}[0]]$. After 2 jumps, we have reached every index reachable from *any* index in that first range, and so on. The number of jumps needed to first include index $n - 1$ in the reachable range is exactly the answer we want.

The key realization is that we do not need an actual BFS queue holding every individual index. Instead, at each “wave” (each fixed number of jumps used so far), we only need to know the boundary of the current range, and the farthest index reachable by spending one more jump from anywhere inside the current range.

Greedy Choice

Maintain two horizons: *currentEnd*, the farthest index reachable using the jumps counted so far, and *farthest*, the farthest index reachable using *one more* jump from anywhere within the current range. Scan left to right; for every index i within the current range, update $\text{farthest} \leftarrow \max(\text{farthest}, i + \text{nums}[i])$. The instant we finish processing the current range (when i reaches *currentEnd*), we have exhausted everything reachable with the current jump count, so we commit to spending one more jump: increment the jump counter and set $\text{currentEnd} \leftarrow \text{farthest}$, beginning the next wave.

Why this greedy expansion is correct

This is exactly BFS, level by level, except that instead of storing every node at the current level in a queue, we only ever need the single number *farthest* – the best possible reach from *any* node at the current level – because, just as in Jump Game I, every index between the previous horizon and the new farthest reach is automatically included in this level’s frontier the moment any node within the previous range can reach it. We never need to know *which* specific index within the current range produced the best *farthest* value, only the value itself, since the next wave starts from the entire new range regardless of which earlier index unlocked it. This is what allows an $O(n)$ single pass to replace what would otherwise look like an explicit multi-source BFS.

Algorithm

1. Initialize $\text{jumps} \leftarrow 0$, $\text{currentEnd} \leftarrow 0$, $\text{farthest} \leftarrow 0$.
2. For $i \leftarrow 0$ to $n - 2$ (we never need to jump *from* the last index):

- (a) $farthest \leftarrow \max(farthest, i + nums[i])$.
- (b) If $i = currentEnd$: this wave is exhausted, so spend one more jump. $jumps \leftarrow jumps + 1$;
 $currentEnd \leftarrow farthest$.

3. Return $jumps$.

Algorithm 8 Jump Game II (Minimum Jumps)

```

1: procedure MINJUMPS( $nums$ )
2:    $n \leftarrow \text{length}(nums)$ 
3:    $jumps \leftarrow 0$ ,  $currentEnd \leftarrow 0$ ,  $farthest \leftarrow 0$ 
4:   for  $i \leftarrow 0$  to  $n - 2$  do
5:      $farthest \leftarrow \max(farthest, i + nums[i])$ 
6:     if  $i = currentEnd$  then
7:        $jumps \leftarrow jumps + 1$ 
8:        $currentEnd \leftarrow farthest$ 
9:     end if
10:  end for
11:  return  $jumps$ 
12: end procedure

```

Dry run

Dry Run. Take $nums = [2, 3, 1, 1, 4]$ ($n = 5$, indices 0..4).

Start with $jumps = 0$, $currentEnd = 0$, $farthest = 0$. We loop i from 0 to 3 (i.e. $n - 2 = 3$).

- $i = 0$: $farthest = \max(0, 0 + 2) = 2$. Since $i = 0 = currentEnd$, spend a jump: $jumps = 1$, $currentEnd = 2$.
- $i = 1$: $farthest = \max(2, 1 + 3) = 4$. Since $i = 1 \neq currentEnd = 2$, no new jump yet.
- $i = 2$: $farthest = \max(4, 2 + 1) = 4$. Since $i = 2 = currentEnd$, spend a jump: $jumps = 2$, $currentEnd = 4$.
- $i = 3$: $farthest = \max(4, 3 + 1) = 4$. Since $i = 3 \neq currentEnd = 4$, no new jump.

The loop ends (we stop before $i = 4$, the last index). The answer is $jumps = 2$: jump from index 0 to index 1 (using $nums[0] = 2$ to reach as far as index 1, the start of the best next wave), then jump from index 1 directly to index 4 using $nums[1] = 3$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int minJumps(vector<int>& nums) {
5     int n = nums.size();
6     int jumps = 0, currentEnd = 0, farthest = 0;
7
8     for (int i = 0; i < n - 1; i++) {
9         farthest = max(farthest, i + nums[i]);
10        if (i == currentEnd) {
11            jumps++;
12            currentEnd = farthest;

```

```

13     }
14 }
15 return jumps;
16 }
17
18 int main() {
19     vector<int> nums = {2, 3, 1, 1, 4};
20     cout << "Minimum jumps: " << minJumps(nums) << endl;
21     return 0;
22 }

```

Complexity Analysis

Time Complexity. The single loop visits each of the first $n - 1$ indices exactly once, doing constant work per index, giving

$$O(n).$$

Space Complexity. We maintain only three scalar variables (*jumps*, *currentEnd*, *farthest*), so the space complexity is $O(1)$.

Key Takeaway

A feasibility-horizon greedy (Section 1.8) upgrades cleanly into a minimum-steps greedy by reframing it as implicit, level-by-level BFS: track the boundary of the current wave (*currentEnd*) and the best reach achievable from anywhere inside that wave (*farthest*); each time the scan catches up to the current boundary, a new wave – one more jump – begins. This pattern of “BFS without an explicit queue, using two horizon pointers” generalizes to any problem phrased as “minimum number of steps to extend reach to a target.”

1.10 Minimum Number of Platforms Required for a Railway

Problem Statement. We are given the arrival times $arr[0..n - 1]$ and departure times $dep[0..n - 1]$ of n trains scheduled at a single railway station, where train i occupies a platform from $arr[i]$ until $dep[i]$ (inclusive; if one train departs at the exact same time another arrives, they are considered to overlap and require separate platforms). We must find the **minimum number of platforms** the station needs so that no train is ever forced to wait for a free platform.

Pattern Recognition

“Minimum number of resources (platforms/rooms/servers) needed so that no request ever waits, given many overlapping time intervals” is the classic *maximum overlap / interval point sweep* pattern – it is closely related to, but distinct from, the activity-selection pattern in Section 1.7. There, we had *one* room and asked how many of the requests we could serve; here, every request *must* be served, and we ask how many rooms that requires. The keyword shift from “maximum number you can fit in one resource” to “minimum resources needed to fit everyone” is the signal to switch from finish-time-only greedy scheduling to a *two-pointer sweep over both arrivals and departures*.

Understanding the problem carefully

At any instant in time, the number of platforms required is exactly the number of trains whose interval $[arr[i], dep[i]]$ currently contains that instant – i.e., the number of trains *simultaneously*

present at the station. The answer to the whole problem is simply the **maximum**, over all instants in time, of this simultaneous count. So the task reduces to efficiently finding the peak number of overlapping intervals.

A direct way to track this is to imagine sweeping a clock forward through time. Every arrival event increases the number of trains currently at the station by one; every departure event decreases it by one. If we process all arrival and departure events in time order, the running count after each event tells us exactly how many platforms are occupied at that moment, and the maximum value this running count ever reaches is our answer.

Greedy Choice

Sort the arrival times and the departure times *independently* (we do not need to keep them paired to the same train – only the multiset of arrival instants and the multiset of departure instants matter for counting overlaps). Use two pointers, one walking through sorted arrivals and one through sorted departures. At each step, compare the next unprocessed arrival to the next unprocessed departure: if the arrival happens at or before that departure, a new train has shown up while the previous one has not yet left, so increment the platform count needed and advance the arrival pointer; otherwise, a train has left before this conflict arises, so decrement the platform count and advance the departure pointer. Track the maximum platform count seen at any point.

Why sorting arrivals and departures independently is valid

It may seem surprising that we are allowed to sort *arr* and *dep* separately, losing the information of which arrival belongs to which train. But the quantity we are computing – the number of simultaneously occupied platforms at any instant – depends only on *how many* arrival events have occurred and *how many* departure events have occurred up to that instant, not on which specific train each event belongs to. The running difference (arrivals processed so far minus departures processed so far) is precisely the number of trains currently at the station, regardless of how we have paired up arrival times with departure times internally. This is what justifies treating *arr* and *dep* as two independent sorted streams of timestamps and sweeping through them with two pointers, taking the running difference's maximum value.

Algorithm

1. Sort *arr* in increasing order; sort *dep* in increasing order.
2. Initialize $i \leftarrow 0$, $j \leftarrow 0$, $platforms \leftarrow 0$, $maxPlatforms \leftarrow 0$.
3. While $i < n$ and $j < n$:
 - (a) If $arr[i] \leq dep[j]$: a new train has arrived while the earliest still-pending train has not yet left. $platforms \leftarrow platforms + 1$; $maxPlatforms \leftarrow \max(maxPlatforms, platforms)$; $i \leftarrow i + 1$.
 - (b) Otherwise: a train has departed. $platforms \leftarrow platforms - 1$; $j \leftarrow j + 1$.
4. Return $maxPlatforms$.

Algorithm 9 Minimum Platforms

```

1: procedure MINPLATFORMS(arr, dep)
2:   sort arr in increasing order
3:   sort dep in increasing order
4:   i  $\leftarrow$  0, j  $\leftarrow$  0, platforms  $\leftarrow$  0, maxPlatforms  $\leftarrow$  0
5:   while i < n and j < n do
6:     if arr[i]  $\leq$  dep[j] then
7:       platforms  $\leftarrow$  platforms + 1
8:       maxPlatforms  $\leftarrow$  max(maxPlatforms, platforms)
9:       i  $\leftarrow$  i + 1
10:    else
11:      platforms  $\leftarrow$  platforms - 1
12:      j  $\leftarrow$  j + 1
13:    end if
14:  end while
15:  return maxPlatforms
16: end procedure

```

Dry run

Dry Run. Take *arr* = [900, 940, 950, 1100, 1500, 1800] and *dep* = [910, 1200, 1120, 1130, 1900, 2000].

Sorted: *arr* = [900, 940, 950, 1100, 1500, 1800] (already sorted), *dep* = [910, 1120, 1130, 1200, 1900, 2000] (sorted).

- *arr*[0] = 900 $\leq$ *dep*[0] = 910. *platforms* = 1, *maxPlatforms* = 1, *i* = 1.
- *arr*[1] = 940 $\leq$ *dep*[0] = 910? No (940 > 910). *platforms* = 0, *j* = 1.
- *arr*[1] = 940 $\leq$ *dep*[1] = 1120. *platforms* = 1, *maxPlatforms* = 1, *i* = 2.
- *arr*[2] = 950 $\leq$ *dep*[1] = 1120. *platforms* = 2, *maxPlatforms* = 2, *i* = 3.
- *arr*[3] = 1100 $\leq$ *dep*[1] = 1120. *platforms* = 3, *maxPlatforms* = 3, *i* = 4.
- *arr*[4] = 1500 $\leq$ *dep*[1] = 1120? No. *platforms* = 2, *j* = 2.
- *arr*[4] = 1500 $\leq$ *dep*[2] = 1130? No. *platforms* = 1, *j* = 3.
- *arr*[4] = 1500 $\leq$ *dep*[3] = 1200? No. *platforms* = 0, *j* = 4.
- *arr*[4] = 1500 $\leq$ *dep*[4] = 1900. *platforms* = 1, *maxPlatforms* stays 3, *i* = 5.
- *arr*[5] = 1800 $\leq$ *dep*[4] = 1900. *platforms* = 2, *maxPlatforms* stays 3, *i* = 6.
- *i* = *n* = 6, loop ends.

The minimum number of platforms required is 3, reached around time 1100–1120, when three trains are simultaneously at the station.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int minPlatforms(vector<int>& arr, vector<int>& dep) {

```

```

5   int n = arr.size();
6   sort(arr.begin(), arr.end());
7   sort(dep.begin(), dep.end());
8
9   int i = 0, j = 0;
10  int platforms = 0, maxPlatforms = 0;
11
12  while (i < n && j < n) {
13      if (arr[i] <= dep[j]) {
14          platforms++;
15          maxPlatforms = max(maxPlatforms, platforms);
16          i++;
17      } else {
18          platforms--;
19          j++;
20      }
21  }
22  return maxPlatforms;
23 }
24
25 int main() {
26     vector<int> arr = {900, 940, 950, 1100, 1500, 1800};
27     vector<int> dep = {910, 1200, 1120, 1130, 1900, 2000};
28
29     cout << "Minimum platforms required: "
30          << minPlatforms(arr, dep) << endl;
31     return 0;
32 }

```

Complexity Analysis

Time Complexity. Sorting both arrays costs $O(n \log n)$. The two-pointer sweep afterward visits each arrival and each departure exactly once, costing $O(n)$. The overall time complexity is

$$O(n \log n).$$

Space Complexity. Sorting can be performed in place (or with $O(n)$ auxiliary space depending on the sorting routine), and the sweep itself uses only a handful of scalar variables, so the auxiliary space complexity is $O(1)$ beyond the input arrays.

Key Takeaway

When a problem asks for the minimum number of identical resources needed so that *every* request can be served without waiting, think in terms of the maximum simultaneous overlap of intervals, computed by sweeping sorted arrival and departure timestamps with a running counter. This is a different flavor of greedy from activity selection: there, sorting by finish time let us discard conflicting requests one at a time; here, sorting both endpoints lets us measure the peak load directly, with nothing discarded.

1.11 Job Sequencing Problem

Problem Statement. We are given n jobs, where the i -th job has a deadline $deadline[i]$ and a profit $profit[i]$. Every job takes exactly one unit of time to complete, and only one job can be processed at any given unit of time (time slots are $1, 2, 3, \dots$). A job earns its profit only if it is completed at or before its deadline. We must choose which jobs to schedule, and at which

time slots, so as to **maximize total profit**; we are also asked to report how many jobs end up being scheduled.

Pattern Recognition

“Each job has a deadline and a profit; schedule jobs (each taking one unit of time) to maximize total profit” is the signature of the *Job Sequencing Problem*. The two keywords to notice together are “**profit, maximize**” (suggesting we should prioritize high-value jobs first) and “**deadline**” (suggesting that, once we have decided *which* jobs to take, we still need a rule for placing them into time slots without violating their deadlines). This combination – rank by value, then place greedily as late as possible – is a different flavor of greedy from interval scheduling, and is often paired conceptually with the Disjoint Set Union (Union-Find) data structure for an even faster slot-finding step.

Understanding the problem carefully

Two separate questions are tangled together here: *which* jobs should we choose to do, and *when* should we do each chosen job. Profit is earned per job, independent of which legal slot it occupies, so among several candidate sets of jobs we might choose, more total profit is always better, with no benefit to a particular schedule beyond it being legal at all. This suggests considering jobs in decreasing order of profit, and greedily trying to fit each one in, since a job with higher profit is always worth taking over a lower-profit job if there is room for only one of them.

Once we decide to take a particular job with deadline d , the only real question is *which* slot among $1, 2, \dots, d$ to place it in. Here a second, equally important greedy idea applies: place the job in the **latest** available slot not exceeding its deadline, rather than the earliest. The reasoning is that an early slot is more valuable to keep *open*, because it can still accommodate jobs with very tight (small) deadlines that arrive later in our profit-sorted processing order, whereas a late slot close to d is useless to any job whose deadline is smaller than d anyway.

Greedy Choice

Sort all jobs in decreasing order of profit. For each job, in this order, scan backward from its deadline toward slot 1 and place it in the first free slot found. If no free slot exists at or before its deadline, skip this job permanently – it can never be scheduled later, since slots only get more occupied as we proceed, not less.

Why processing by profit, and placing as late as possible, is correct

Consider why we never want to revisit a profit-order decision. Once a higher-profit job has successfully claimed a slot, no later (lower-profit) job can ever displace it without decreasing total profit, so committing immediately, in profit order, is safe – this is the same “highest value first” exchange argument used throughout greedy scheduling: if some optimal solution skipped a higher-profit job J to make room for a lower-profit job J' that conflicts with it, swapping J in for J' (which is always possible, since J has at least as early a deadline requirement satisfied by any slot J' could use, or an even more flexible range) cannot decrease, and may increase, total profit.

Placing each accepted job as late as possible within its own deadline window is the secondary greedy choice that makes the slot-assignment feasible without any backtracking. By reserving early slots for as long as possible, we never block a future job (processed later, because it has lower profit) that might desperately need one of those early slots due to having a very small deadline – a job with deadline 1 has nowhere else to go, so an early slot occupied unnecessarily

by an earlier, more flexible job would be a wasted opportunity that greedy avoids by always preferring the latest legal slot first.

Algorithm

1. Sort the jobs in decreasing order of profit.
2. Let $maxDeadline$ be the largest deadline among all jobs. Create a boolean array $slot[1..maxDeadline]$, all initially free.
3. Initialize $countJobs \leftarrow 0$ and $totalProfit \leftarrow 0$.
4. For each job, in decreasing profit order, with deadline d :
 - (a) Scan t from $\min(d, maxDeadline)$ down to 1:
 - i. If $slot[t]$ is free: mark it occupied, increment $countJobs$, add this job's profit to $totalProfit$, and stop scanning (break out of this inner loop) – the job has been placed.
 - (b) If no free slot was found in the scan, this job is permanently skipped.
5. Return $countJobs$ and $totalProfit$.

Algorithm 10 Job Sequencing

```

1: procedure JOBSEQUENCING( $jobs$ )
2:   sort  $jobs$  in decreasing order of profit
3:    $maxDeadline \leftarrow \max_i deadline[i]$ 
4:    $slot[1..maxDeadline] \leftarrow$  all free
5:    $countJobs \leftarrow 0$ ,  $totalProfit \leftarrow 0$ 
6:   for each job  $(d, p)$  in sorted order do
7:     for  $t \leftarrow \min(d, maxDeadline)$  down to 1 do
8:       if  $slot[t]$  is free then
9:         mark  $slot[t]$  occupied
10:         $countJobs \leftarrow countJobs + 1$ 
11:         $totalProfit \leftarrow totalProfit + p$ 
12:        break
13:      end if
14:    end for
15:  end for
16:  return ( $countJobs, totalProfit$ )
17: end procedure

```

Dry run

Dry Run. Take jobs with (deadline, profit) pairs: (4, 20), (1, 10), (1, 40), (1, 30).

Sorted by decreasing profit: (1, 40), (1, 30), (4, 20), (1, 10).

Slots 1, 2, 3, 4 all start free.

- Job (1, 40): scan from $t = 1$ down to 1. Slot 1 is free; place it there. $countJobs = 1$, $totalProfit = 40$. Slots: [1=occupied, 2, 3, 4=free].
- Job (1, 30): scan from $t = 1$ down to 1. Slot 1 is occupied. No free slot found within its deadline. Skip this job.

- Job (4,20): scan from $t = 4$ down to 1. Slot 4 is free; place it there. $countJobs = 2$, $totalProfit = 60$. Slots: [1=occ, 2, 3=free, 4=occ].
- Job (1,10): scan from $t = 1$ down to 1. Slot 1 is occupied. Skip.

The final result is $countJobs = 2$ jobs scheduled, with $totalProfit = 60$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Job {
5     int id;
6     int deadline;
7     int profit;
8 };
9
10 pair<int,int> jobSequencing(vector<Job>& jobs) {
11     sort(jobs.begin(), jobs.end(), [](const Job& a, const Job& b) {
12         return a.profit > b.profit;
13     });
14
15     int maxDeadline = 0;
16     for (auto& job : jobs) {
17         maxDeadline = max(maxDeadline, job.deadline);
18     }
19
20     vector<bool> slot(maxDeadline + 1, false); // 1-indexed
21     int countJobs = 0, totalProfit = 0;
22
23     for (auto& job : jobs) {
24         for (int t = min(job.deadline, maxDeadline); t >= 1; t--) {
25             if (!slot[t]) {
26                 slot[t] = true;
27                 countJobs++;
28                 totalProfit += job.profit;
29                 break;
30             }
31         }
32     }
33     return {countJobs, totalProfit};
34 }
35
36 int main() {
37     vector<Job> jobs = {
38         {1, 4, 20},
39         {2, 1, 10},
40         {3, 1, 40},
41         {4, 1, 30}
42     };
43
44     auto result = jobSequencing(jobs);
45     cout << "Jobs done: " << result.first << endl;
46     cout << "Total profit: " << result.second << endl;
47     return 0;
48 }

```

Complexity Analysis

Time Complexity. Sorting the n jobs by profit costs $O(n \log n)$. For each job, scanning backward through slots costs up to $O(\text{maxDeadline})$ in the worst case, giving a total of $O(n \cdot \text{maxDeadline})$ for the slot-placement phase. Hence the overall time complexity is

$$O(n \log n + n \cdot \text{maxDeadline}).$$

This can be improved to $O(n \log n + n \log n) = O(n \log n)$ overall by using a Disjoint Set Union structure (covered in Chapter 6) to jump directly to the latest free slot at or before a given deadline, rather than scanning slot by slot. **Space Complexity.** We use a boolean array of size $O(\text{maxDeadline})$ to track slot occupancy, so the space complexity is $O(\text{maxDeadline})$, in addition to $O(n)$ for sorting the jobs.

Key Takeaway

When a problem combines “*maximize total value*” with “*each item also has a deadline/capacity constraint on where it can be placed*,” a two-layer greedy often applies: sort by value to decide *which* items to take, and within that order, place each accepted item as late (or otherwise as flexibly) as possible so as not to block items that have less placement freedom and are still waiting to be considered.

1.12 Candy

Problem Statement. There are n children standing in a line, and the i -th child has a rating $\text{ratings}[i]$. We must give each child at least one candy, subject to the rule that any child with a strictly higher rating than an **immediate neighbor** (the child directly to the left or directly to the right) must receive strictly more candies than that neighbor. We must find the **minimum total number of candies** required to satisfy all of these constraints.

Pattern Recognition

“*Each element must be strictly greater than its immediate neighbor whenever its value is strictly greater*” – a constraint defined purely in terms of comparisons with the elements directly to the left and right – is the signature of a *two-pass directional greedy*. Whenever a problem states a rule of the form “compare element i only to $i - 1$ and $i + 1$, and the rule is not symmetric (higher rating needs strictly more, but the rule says nothing directly about how much more when ratings are equal or lower),” the standard technique is to satisfy the left-neighbor rule and the right-neighbor rule in two completely separate linear passes, then combine the two results – rather than trying to track both directions simultaneously in a single pass, which is awkward because a child’s required candy count can be driven up by either side independently.

Understanding the problem carefully

The rule is purely *local*: child i needs more candies than child $i - 1$ if and only if $\text{ratings}[i] > \text{ratings}[i - 1]$, and child i needs more candies than child $i + 1$ if and only if $\text{ratings}[i] > \text{ratings}[i + 1]$. These are two independent constraints, one looking left and one looking right, and a child’s final candy count must simultaneously satisfy both. This naturally suggests solving the left-looking constraint and the right-looking constraint separately, each with its own simple left-to-right or right-to-left greedy sweep, and then merging the two answers by taking, for each child, whichever requirement is stricter (i.e. larger).

The left-to-right pass

Greedy Choice – Left Pass

Scan left to right. Give every child at least 1 candy initially. If $\text{ratings}[i] > \text{ratings}[i - 1]$, this child must have strictly more candy than the child to its left, so set $\text{left}[i] \leftarrow \text{left}[i - 1] + 1$. Otherwise, the left-neighbor constraint places no extra demand on child i beyond the baseline of 1 candy, so $\text{left}[i] \leftarrow 1$.

This single pass correctly satisfies every left-looking constraint, because each child's requirement here only ever depends on the child immediately before it, which has already been finalized by the time we reach the current child – there is no need to look further back, since any chain of strictly increasing ratings to the left has already been correctly propagated forward by the recursive nature of the update rule.

The right-to-left pass

Greedy Choice – Right Pass

Symmetrically, scan right to left. Give every child at least 1 candy initially. If $\text{ratings}[i] > \text{ratings}[i + 1]$, set $\text{right}[i] \leftarrow \text{right}[i + 1] + 1$. Otherwise, $\text{right}[i] \leftarrow 1$.

Why taking the maximum of both passes is correct

Once both arrays are computed, child i 's final candy count must be at least $\text{left}[i]$ (to satisfy the left-neighbor rule) and at least $\text{right}[i]$ (to satisfy the right-neighbor rule). Since these are two independent lower bounds on the same quantity (the number of candies given to child i), and we want the *minimum* total, the smallest value that satisfies both lower bounds simultaneously is exactly their maximum:

$$\text{candies}[i] = \max(\text{left}[i], \text{right}[i]).$$

No smaller value could satisfy both constraints, and this value, taken for every child simultaneously, does not interfere with any other child's constraint, because each constraint here only relates two physically adjacent children, and the relative order between $\text{left}[i]$ and $\text{left}[i \pm 1]$ (or $\text{right}[i]$ and $\text{right}[i \pm 1]$) already correctly encodes the required strict inequality wherever the rating comparison demands it. Taking componentwise maxima of two arrays that each individually satisfy their own one-sided constraint is guaranteed to satisfy both constraints together, without ever needing to re-verify the combined array.

Algorithm

1. Initialize arrays $\text{left}[0..n - 1]$ and $\text{right}[0..n - 1]$, each filled with 1.
2. Left pass: for $i \leftarrow 1$ to $n - 1$, if $\text{ratings}[i] > \text{ratings}[i - 1]$, set $\text{left}[i] \leftarrow \text{left}[i - 1] + 1$.
3. Right pass: for $i \leftarrow n - 2$ down to 0, if $\text{ratings}[i] > \text{ratings}[i + 1]$, set $\text{right}[i] \leftarrow \text{right}[i + 1] + 1$.
4. Compute $\text{total} \leftarrow \sum_{i=0}^{n-1} \max(\text{left}[i], \text{right}[i])$.
5. Return total .

Algorithm 11 Candy Distribution

```

1: procedure CANDY(ratings)
2:    $n \leftarrow \text{length}(\text{ratings})$ 
3:    $\text{left}[0..n-1] \leftarrow 1$ ,  $\text{right}[0..n-1] \leftarrow 1$ 
4:   for  $i \leftarrow 1$  to  $n-1$  do
5:     if  $\text{ratings}[i] > \text{ratings}[i-1]$  then
6:        $\text{left}[i] \leftarrow \text{left}[i-1] + 1$ 
7:     end if
8:   end for
9:   for  $i \leftarrow n-2$  down to  $0$  do
10:    if  $\text{ratings}[i] > \text{ratings}[i+1]$  then
11:       $\text{right}[i] \leftarrow \text{right}[i+1] + 1$ 
12:    end if
13:  end for
14:   $\text{total} \leftarrow 0$ 
15:  for  $i \leftarrow 0$  to  $n-1$  do
16:     $\text{total} \leftarrow \text{total} + \max(\text{left}[i], \text{right}[i])$ 
17:  end for
18:  return  $\text{total}$ 
19: end procedure

```

Dry run

Dry Run. Take $\text{ratings} = [1, 0, 2]$.

Left pass. $\text{left} = [1, 1, 1]$ initially.

- $i = 1$: $\text{ratings}[1] = 0 > \text{ratings}[0] = 1$? No. Stays $\text{left}[1] = 1$.
- $i = 2$: $\text{ratings}[2] = 2 > \text{ratings}[1] = 0$? Yes. $\text{left}[2] = \text{left}[1] + 1 = 2$.

Result: $\text{left} = [1, 1, 2]$.

Right pass. $\text{right} = [1, 1, 1]$ initially.

- $i = 1$: $\text{ratings}[1] = 0 > \text{ratings}[2] = 2$? No. Stays $\text{right}[1] = 1$.
- $i = 0$: $\text{ratings}[0] = 1 > \text{ratings}[1] = 0$? Yes. $\text{right}[0] = \text{right}[1] + 1 = 2$.

Result: $\text{right} = [2, 1, 1]$.

Combine. $\text{candies} = [\max(1, 2), \max(1, 1), \max(2, 1)] = [2, 1, 2]$.

The total is $2 + 1 + 2 = 5$. This is correct: child 0 (rating 1) needs more than child 1 (rating 0), so child 0 gets 2 and child 1 gets the baseline 1; child 2 (rating 2) needs more than child 1, so child 2 gets 2. No constraint links child 0 and child 2 directly (they are not neighbors), so this is fully consistent and minimal.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int candy(vector<int>& ratings) {
5     int n = ratings.size();
6     vector<int> left(n, 1), right(n, 1);
7
8     for (int i = 1; i < n; i++) {

```

```

9      if (ratings[i] > ratings[i - 1]) {
10          left[i] = left[i - 1] + 1;
11      }
12  }
13
14  for (int i = n - 2; i >= 0; i--) {
15      if (ratings[i] > ratings[i + 1]) {
16          right[i] = right[i + 1] + 1;
17      }
18  }
19
20  int total = 0;
21  for (int i = 0; i < n; i++) {
22      total += max(left[i], right[i]);
23  }
24  return total;
25 }
26
27 int main() {
28     vector<int> ratings = {1, 0, 2};
29     cout << "Minimum candies: " << candy(ratings) << endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. We perform two linear passes (left-to-right and right-to-left) plus one linear combination pass, each $O(n)$, giving an overall time complexity of

$$O(n).$$

Space Complexity. We use two auxiliary arrays, *left* and *right*, each of size n , giving a space complexity of $O(n)$. This can be reduced to $O(1)$ extra space with a more careful single-pass peak-detection technique, though the two-array version is the clearest to derive and verify correct.

Key Takeaway

When a constraint compares each element only to its immediate left and right neighbors, and the comparison is directionally asymmetric (a strict inequality matters only in one direction at a time), decompose the problem into two independent one-directional sweeps – one enforcing the left-looking rule, one enforcing the right-looking rule – and combine the two results with a pointwise maximum (for a “need at least this many” constraint) or minimum (for a “cannot exceed this many” constraint).

1.13 Shortest Job First (SJF) CPU Scheduling

Problem Statement. We are given the burst times (the amount of CPU time each process needs) of n processes, all of which arrive and are ready to run at time 0. Only one process can run on the CPU at a time, and once we commit to running a process we run it to completion (non-preemptive scheduling). We must decide the order in which to run the processes so as to **minimize the average waiting time**, where the waiting time of a process is the amount of time it sits ready in the queue before it actually starts executing.

Pattern Recognition

“Minimize average waiting time, single CPU, run-to-completion (non-preemptive), all jobs ready at the same time” is the textbook *Shortest Job First* scheduling rule. The keyword combination to watch for is “**minimize the average (or total) waiting/completion time**” together with “**one task at a time, no preemption.**” Whenever every job is available from the very start, the order of execution is the only choice that matters, and there is a clean, provable rule for the best order: process the **shortest job first**. This is conceptually close to a sorting greedy, but the insight worth absorbing is *why* ordering by burst time, rather than any other attribute, minimizes the *average* of a quantity that depends on cumulative sums.

Understanding the problem carefully

If we decide to run the processes in some order $p_1, p_2, \dots, p_n$ with burst times $b_1, b_2, \dots, b_n$ (relabelled according to this order), then process p_1 waits 0 time units, process p_2 waits b_1 time units (it has to wait for p_1 to finish), process p_3 waits $b_1 + b_2$, and in general process p_k waits

$$W_k = b_1 + b_2 + \dots + b_{k-1}.$$

The total waiting time is

$$\sum_{k=1}^n W_k = \sum_{k=1}^n \sum_{j=1}^{k-1} b_j = \sum_{j=1}^n (n-j) b_j,$$

since burst time b_j is counted once for every process that comes after it in the order – exactly $n-j$ of them. This formula reveals the key structural fact: **burst times that appear earlier in the order are multiplied by a larger coefficient** ($n-1$ for the very first job, down to 0 for the last job). To minimize this weighted sum, we should pair the *largest* coefficients with the *smallest* burst times – which means running the shortest jobs first.

Greedy Choice

Sort the processes in increasing order of burst time, and run them strictly in that order.

Why shortest-job-first is optimal

This is a direct instance of the *rearrangement inequality*: given two sequences of numbers, the sum of pairwise products is minimized when one sequence is sorted increasing and the other is sorted decreasing (so that large values are paired with small values), and maximized when both are sorted in the same direction. Here, the coefficients $n-1, n-2, \dots, 0$ are fixed and strictly decreasing by position; to minimize $\sum_j (n-j)b_j$, the burst times b_j must be arranged in strictly *increasing* order alongside these strictly decreasing coefficients – that is, the smallest burst time should receive the largest coefficient (run first, waited-for by the most subsequent jobs), and the largest burst time should receive the smallest coefficient (run last, waited-for by nobody).

An equivalent and perhaps more intuitive exchange argument: suppose two adjacent jobs in some schedule are out of order, i.e. a longer job runs immediately before a shorter job. Swapping them leaves the waiting time of every other job completely unchanged (the total time consumed by this pair together is the same regardless of their internal order, so everything after them starts at the same time either way), but it strictly decreases the combined waiting time of *this pair*, because the shorter job, now running first, makes the longer job wait less than the longer job, when running first, made the shorter job wait. Repeatedly fixing such adjacent inversions

– exactly what sorting does – can only decrease total waiting time, so the fully sorted (shortest first) order is optimal.

Algorithm

1. Sort the burst times in increasing order.
2. Initialize $waitingTime \leftarrow 0$ and $totalWait \leftarrow 0$.
3. For $i \leftarrow 0$ to $n - 1$ (processing jobs in sorted order):
 - (a) Add $waitingTime$ to $totalWait$ (this is the waiting time of the current job).
 - (b) $waitingTime \leftarrow waitingTime + burst[i]$ (the next job must wait for this one to finish too).
4. Return $totalWait/n$ as the average waiting time.

Algorithm 12 Shortest Job First Scheduling

```

1: procedure SJF(burst)
2:   sort burst in increasing order
3:   waitingTime  $\leftarrow 0$ , totalWait  $\leftarrow 0$ 
4:   for  $i \leftarrow 0$  to  $n - 1$  do
5:     totalWait  $\leftarrow totalWait + waitingTime$ 
6:     waitingTime  $\leftarrow waitingTime + burst[i]$ 
7:   end for
8:   return totalWait/ $n$ 
9: end procedure

```

Dry run

Dry Run. Take burst times [6, 1, 8, 9, 3].

Sorted increasing: [1, 3, 6, 8, 9].

- $i = 0$ (burst 1): job waits $waitingTime = 0$. $totalWait = 0$. Update $waitingTime = 0 + 1 = 1$.
- $i = 1$ (burst 3): job waits 1. $totalWait = 1$. Update $waitingTime = 1 + 3 = 4$.
- $i = 2$ (burst 6): job waits 4. $totalWait = 5$. Update $waitingTime = 4 + 6 = 10$.
- $i = 3$ (burst 8): job waits 10. $totalWait = 15$. Update $waitingTime = 10 + 8 = 18$.
- $i = 4$ (burst 9): job waits 18. $totalWait = 33$. Update $waitingTime = 18 + 9 = 27$.

Total waiting time is 33, and the average waiting time is $33/5 = 6.6$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 double shortestJobFirst(vector<int>& burst) {
5     sort(burst.begin(), burst.end());
6
7     int n = burst.size();
8     long long waitingTime = 0, totalWait = 0;
9

```

```

10     for (int i = 0; i < n; i++) {
11         totalWait += waitingTime;
12         waitingTime += burst[i];
13     }
14
15     return (double)totalWait / n;
16 }
17
18 int main() {
19     vector<int> burst = {6, 1, 8, 9, 3};
20     cout << "Average waiting time: "
21          << shortestJobFirst(burst) << endl;
22     return 0;
23 }

```

Complexity Analysis

Time Complexity. Sorting the n burst times costs $O(n \log n)$. The subsequent single pass to accumulate waiting times costs $O(n)$. The overall time complexity is

$$O(n \log n).$$

Space Complexity. Sorting is performed in place, and we use only a constant number of extra scalar variables, giving auxiliary space complexity $O(1)$ (ignoring sort's recursion stack).

Key Takeaway

Whenever the objective is a sum of the form $\sum (\text{fixed position-dependent coefficient}) \times (\text{value you get to order})$, and the coefficients are monotonic in position, the rearrangement inequality tells you exactly how to order the values for the minimum (or maximum) possible sum: pair the largest coefficients with the smallest values (or vice versa for maximization). Recognizing this pattern – “minimize a sum of weighted, orderable quantities” – immediately tells you to sort, without needing to reconstruct the exchange argument from scratch every time.

1.14 Insert Interval

Problem Statement. We are given a list of intervals *intervals* that is already sorted by start time and contains no overlaps (it represents, say, a calendar of existing non-conflicting events). We are also given one new interval *newInterval*. We must insert *newInterval* into the list, merging it with any existing intervals it overlaps, so that the result is still a list of intervals sorted by start time with no overlaps.

Pattern Recognition

“The existing intervals are already sorted and non-overlapping; insert one new interval and merge where necessary” is a strong hint that we do not need to re-sort or re-scan everything from scratch – the existing order is a resource we should exploit with a single linear pass, split into three natural phases (intervals entirely before the new one, intervals overlapping the new one, intervals entirely after). Whenever a problem promises that the input is already sorted, a greedy single pass that respects that order is almost always available, replacing what would otherwise look like a sort-and-merge operation.

Understanding the problem carefully

Because the existing intervals are sorted by start time and mutually non-overlapping, as we scan through them from left to right, each interval falls into exactly one of three categories relative to $newInterval = [ns, ne]$:

1. **Entirely before:** the interval's end is strictly less than ns . It cannot overlap $newInterval$ at all, and because the list is sorted, every interval we have not yet visited could still potentially overlap, but every interval already confirmed to end before ns can be output immediately, unchanged.
2. **Overlapping:** the interval's start is at most ne and its end is at least ns . Any such interval must be merged into a single growing interval together with $newInterval$ and every other overlapping interval.
3. **Entirely after:** the interval's start is strictly greater than ne . Once we reach such an interval, we know $newInterval$ (possibly already merged with some earlier intervals) is finished growing, because the remaining intervals, being sorted, can only start even later.

Greedy Choice

Scan the existing sorted intervals once, left to right. First, copy every interval that ends strictly before $newInterval$ starts directly into the answer, untouched. Next, while the current interval overlaps the (possibly already-growing) new interval, absorb it by extending the new interval's start to the minimum of the two starts and its end to the maximum of the two ends. Once no more intervals overlap, insert this final merged interval into the answer. Finally, copy every remaining interval directly into the answer, untouched.

Why a single pass suffices

This works cleanly because of two facts guaranteed by the problem: the existing intervals are sorted by start time, and they do not overlap each other. Sortedness means that once we pass an interval ending before ns , no later interval can also need to be classified as “entirely before” incorrectly – the boundary between “before” and “possibly overlapping” is crossed exactly once as we scan forward. Non-overlap among the existing intervals means that, when merging the overlapping group, we never need to worry about two existing intervals needing to be merged with each other independently of $newInterval$ – they only ever come into contact with each other *through* their shared overlap with the growing merged interval. This is what allows the entire operation to be a single $O(n)$ pass with no need to sort or to revisit any interval more than once.

Algorithm

Write $[s_i, e_i]$ for $intervals[i]$, and $[ns, ne]$ for the (possibly growing) new interval.

1. Initialize an empty result list and an index $i \leftarrow 0$.
2. **Phase 1.** While $i < n$ and $e_i < ns$: append $[s_i, e_i]$ to the result; $i \leftarrow i + 1$.
3. **Phase 2.** While $i < n$ and $s_i \leq ne$: merge by setting $ns \leftarrow \min(ns, s_i)$ and $ne \leftarrow \max(ne, e_i)$; $i \leftarrow i + 1$. Once the loop ends, append the final merged $[ns, ne]$ to the result.
4. **Phase 3.** While $i < n$: append $[s_i, e_i]$ to the result; $i \leftarrow i + 1$.
5. Return the result.

Algorithm 13 Insert Interval

```

1: procedure INSERTINTERVAL(intervals, newInterval)
2:   result  $\leftarrow$  empty list, i  $\leftarrow$  0, n  $\leftarrow$  length(intervals)
3:   while i < n and intervals[i].end < newInterval.start do
4:     append intervals[i] to result; i  $\leftarrow$  i + 1
5:   end while
6:   while i < n and intervals[i].start  $\leq$  newInterval.end do
7:     newInterval.start  $\leftarrow$  min(newInterval.start, intervals[i].start)
8:     newInterval.end  $\leftarrow$  max(newInterval.end, intervals[i].end)
9:     i  $\leftarrow$  i + 1
10:  end while
11:  append newInterval to result
12:  while i < n do
13:    append intervals[i] to result; i  $\leftarrow$  i + 1
14:  end while
15:  return result
16: end procedure

```

Dry run

Dry Run. Take *intervals* = [[1, 3], [6, 9]] and *newInterval* = [2, 5].

- **Phase 1:** check [1, 3]. Is $3 < 2$? No. Phase 1 ends with nothing copied, *i* = 0.
- **Phase 2:** check [1, 3]. Is $1 \leq 5$? Yes, overlaps. Merge: *newInterval* = [min(2, 1), max(5, 3)] = [1, 5]. *i* = 1. Check [6, 9]. Is $6 \leq 5$? No. Phase 2 ends. Append [1, 5] to result.
- **Phase 3:** append [6, 9] unchanged.

The result is [[1, 5], [6, 9]].

Now take *intervals* = [[1, 2], [3, 5], [6, 7], [8, 10], [12, 16]] and *newInterval* = [4, 8].

- **Phase 1:** [1, 2] has end $2 < 4$, copy it. [3, 5] has end $5 \not< 4$, stop. Result so far: [[1, 2]], *i* points at [3, 5].
- **Phase 2:** [3, 5]: start $3 \leq 8$, overlaps. Merge: *newInterval* = [min(4, 3), max(8, 5)] = [3, 8]. Next, [6, 7]: start $6 \leq 8$, overlaps. Merge: [min(3, 6), max(8, 7)] = [3, 8] (unchanged). Next, [8, 10]: start $8 \leq 8$, overlaps. Merge: [min(3, 8), max(8, 10)] = [3, 10]. Next, [12, 16]: start $12 \leq 10$? No, stop. Append [3, 10] to result.
- **Phase 3:** append [12, 16] unchanged.

The result is [[1, 2], [3, 10], [12, 16]].

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<vector<int>> insertInterval(vector<vector<int>>& intervals,
5                                vector<int>& newInterval) {
6     vector<vector<int>> result;
7     int n = intervals.size();
8     int i = 0;

```

```

9
10 // Phase 1: intervals ending before newInterval starts
11 while (i < n && intervals[i][1] < newInterval[0]) {
12     result.push_back(intervals[i]);
13     i++;
14 }
15
16 // Phase 2: merge all overlapping intervals
17 while (i < n && intervals[i][0] <= newInterval[1]) {
18     newInterval[0] = min(newInterval[0], intervals[i][0]);
19     newInterval[1] = max(newInterval[1], intervals[i][1]);
20     i++;
21 }
22 result.push_back(newInterval);
23
24 // Phase 3: remaining intervals
25 while (i < n) {
26     result.push_back(intervals[i]);
27     i++;
28 }
29
30 return result;
31 }
32
33 int main() {
34     vector<vector<int>> intervals = {{1,2},{3,5},{6,7},{8,10},{12,16}};
35     vector<int> newInterval = {4, 8};
36
37     auto result = insertInterval(intervals, newInterval);
38     for (auto& iv : result) {
39         cout << "[" << iv[0] << "," << iv[1] << "]" ";
40     }
41     cout << endl;
42     return 0;
43 }

```

Complexity Analysis

Time Complexity. Each of the n existing intervals is visited exactly once across the three phases combined (no interval is revisited), and each visit does constant work, giving

$$O(n).$$

Space Complexity. The result list holds at most $n + 1$ intervals, so the space complexity is $O(n)$ for the output (no other significant auxiliary storage is used).

Key Takeaway

When the input is explicitly guaranteed to already be sorted, look for a single linear pass with a small, fixed number of phases (here: before-overlap-after) instead of reapplying a general sort-and-merge algorithm. Exploiting a sortedness guarantee given for free in the problem statement is itself a greedy-style optimization: it converts an $O(n \log n)$ general merge into an $O(n)$ specialized one.

1.15 Merge Intervals

Problem Statement. We are given an array of intervals $intervals[i] = [start_i, end_i]$, in no particular order, and possibly overlapping with one another. We must merge all overlapping intervals and return an array of the resulting non-overlapping intervals that together cover exactly the same set of points as the original collection.

Pattern Recognition

“Merge all overlapping intervals” with **no** guarantee that the input is already sorted is the unsorted counterpart of Section 1.14. The keyword shift – the absence of a sortedness guarantee – is the signal that we must add a sorting step ourselves before the same kind of single linear merge pass can apply. Whenever you see “merge/combine all overlapping intervals” with arbitrary input order, the standard recipe is always: *sort by start time, then sweep once, extending or closing off a current “running” merged interval.*

Understanding the problem carefully

Once the intervals are sorted by start time, two consecutive intervals (in this sorted order) either overlap or they do not. Crucially, after sorting by start time, if interval j does not overlap the currently-growing merged interval, no interval appearing even later (with an even larger start time) can overlap it either, *unless* merging interval j itself first extended the running interval far enough to reach that later interval – but if j did not overlap and was therefore not merged, the running interval’s end did not change, so this concern does not arise. This guarantees that a single forward sweep, maintaining only the one currently-open merged interval, is sufficient; we never need to revisit an interval once we have moved past it.

Greedy Choice

Sort all intervals by start time. Initialize the answer with the first interval as the current “running” interval. For each subsequent interval, in sorted order, check whether its start is at most the end of the running interval. If so, it overlaps: extend the running interval’s end to the maximum of the two ends. If not, the running interval is finished growing: push it into the answer, and start a new running interval from the current one.

Why sorting first makes the greedy sweep correct

This is, in essence, the merging phase of Section 1.14 applied repeatedly, once for every interval, rather than just once for a single new interval. The correctness argument is the same: because we process intervals in increasing order of start time, the moment an interval fails to overlap the current running interval, every future interval also has a start time at least as large, hence also cannot reach backward to overlap the running interval – so the running interval can be safely finalized and emitted. This sequential, never-look-back property is exactly what makes a single $O(n)$ sweep (after the $O(n \log n)$ sort) sufficient, with no need for any kind of nested loop checking all pairs of intervals for overlap.

Algorithm

1. Sort $intervals$ by start time, increasing.
2. Initialize the result list with the first interval as the current running interval.
3. For each subsequent interval $[s, e]$ in sorted order:

- (a) If $s \leq \text{lastEnd}$ (the end of the last interval placed in the result): extend that last interval's end to $\max(\text{lastEnd}, e)$.
- (b) Otherwise: append $[s, e]$ as a new entry in the result.

4. Return the result.

Algorithm 14 Merge Intervals

```

1: procedure MERGEINTERVALS(intervals)
2:   sort intervals by start time, increasing
3:   result  $\leftarrow$  [intervals[0]]
4:   for  $i \leftarrow 1$  to  $n - 1$  do
5:     last  $\leftarrow$  result[length(result) - 1]
6:     if intervals[i].start  $\leq$  last.end then
7:       last.end  $\leftarrow$  max(last.end, intervals[i].end)
8:     else
9:       append intervals[i] to result
10:    end if
11:  end for
12:  return result
13: end procedure

```

Dry run

Dry Run. Take $\text{intervals} = [[1, 3], [2, 6], [8, 10], [15, 18]]$.

Sorted by start time, the order is already $[1, 3], [2, 6], [8, 10], [15, 18]$.

- Initialize $\text{result} = [[1, 3]]$.
- Process $[2, 6]$: last in result is $[1, 3]$, and $2 \leq 3$, so overlap. Extend: $[1, \max(3, 6)] = [1, 6]$. Result = $[[1, 6]]$.
- Process $[8, 10]$: last in result is $[1, 6]$, and $8 \leq 6$? No. Append. Result = $[[1, 6], [8, 10]]$.
- Process $[15, 18]$: last in result is $[8, 10]$, and $15 \leq 10$? No. Append. Result = $[[1, 6], [8, 10], [15, 18]]$.

The final merged set is $[[1, 6], [8, 10], [15, 18]]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<vector<int>> mergeIntervals(vector<vector<int>>& intervals) {
5     if (intervals.empty()) return {};
6
7     sort(intervals.begin(), intervals.end());
8
9     vector<vector<int>> result;
10    result.push_back(intervals[0]);
11
12    for (int i = 1; i < (int)intervals.size(); i++) {
13        if (intervals[i][0] <= result.back()[1]) {
14            result.back()[1] = max(result.back()[1], intervals[i][1]);
15        } else {

```

```

16         result.push_back(intervals[i]);
17     }
18 }
19 return result;
20 }
21
22 int main() {
23     vector<vector<int>> intervals = {{1,3},{2,6},{8,10},{15,18}};
24     auto merged = mergeIntervals(intervals);
25
26     for (auto& iv : merged) {
27         cout << "[" << iv[0] << "," << iv[1] << " ] ";
28     }
29     cout << endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. Sorting the n intervals by start time costs $O(n \log n)$. The subsequent single sweep costs $O(n)$. The overall time complexity is

$$O(n \log n).$$

Space Complexity. The result list holds at most n intervals, so the space complexity is $O(n)$. If in-place sorting is used and we are allowed to modify the input, no further auxiliary space beyond the output is required.

Key Takeaway

“Merge overlapping intervals” and “insert one interval into an already-sorted, non-overlapping set” (Section 1.14) are really the same merging logic at two different scales. Once you notice that a single new interval inserted into a sorted list is structurally identical to a sorted list being built up one interval at a time, both problems collapse into the same primitive: maintain one “currently open” interval and only close it off when the next candidate’s start exceeds its end.

1.16 Non-overlapping Intervals

Problem Statement. We are given an array of intervals $intervals[i] = [start_i, end_i]$. We must find the **minimum number of intervals to remove** so that the remaining intervals are pairwise non-overlapping (two intervals that merely touch at an endpoint, i.e. one ends exactly where another begins, are *not* considered overlapping).

Pattern Recognition

“Minimum number of intervals to remove so that the rest are non-overlapping” is logically the complement of “maximum number of non-overlapping intervals you can keep” – and the latter is exactly the activity selection problem from Section 1.7. Whenever a problem asks for the **minimum number to delete/remove** to enforce some property, check whether the property is equivalent to a maximization you already know how to solve; if so, the answer is simply $n -$ (the maximum achievable count). Recognizing this equivalence saves you from re-deriving a new greedy strategy from scratch.

Understanding the problem carefully

Removing the fewest intervals to make the rest non-overlapping is the same goal, viewed backward, as keeping the most intervals while staying non-overlapping: every interval not removed must survive, and the survivors must be mutually non-overlapping. So if we can compute

$$\text{maxKept} = \text{largest number of pairwise non-overlapping intervals selectable,}$$

then the answer to this problem is immediately

$$n - \text{maxKept}.$$

And computing *maxKept* is precisely the activity selection problem solved in Section 1.7: sort by finish time, and greedily keep an interval whenever it does not conflict with the most recently kept one.

Greedy Choice

Sort intervals by end time, increasing. Walk through them in this order, maintaining the end time of the last interval kept. Keep the current interval (it does not need to be removed) if its start is at least the end of the last kept interval (recall that touching endpoints are allowed, so the comparison is $\geq$, not strictly $>$, unlike the room-booking version in Section 1.7 where touching endpoints *do* conflict because a physical room cannot instantly reset). Otherwise, this interval must be removed, since it overlaps the last interval we chose to keep.

Why this reduction and greedy choice are correct

The correctness of greedily building *maxKept* by always preferring the interval that finishes earliest among the remaining candidates is exactly the exchange argument given in Section 1.7: the interval with the globally earliest finish time can always be included in some optimal non-overlapping selection, because including it can never block more future options than including any later-finishing alternative would. Once that first choice is fixed, the same argument applies recursively to the remaining intervals that start after it ends. The only change from Section 1.7 is the boundary condition at matching endpoints – this problem explicitly states that touching is not overlapping, so the compatibility check uses $\text{start}_i \geq \text{lastEnd}$ rather than a strict inequality.

Algorithm

1. Sort *intervals* by end time, increasing.
2. Initialize $\text{lastEnd} \leftarrow -\infty$ (or the end of the first interval after selecting it) and $\text{kept} \leftarrow 0$.
3. For each interval $[s, e]$ in sorted order:
 - (a) If $s \geq \text{lastEnd}$: keep this interval. $\text{kept} \leftarrow \text{kept} + 1$; $\text{lastEnd} \leftarrow e$.
 - (b) Otherwise: this interval must be removed (it overlaps the last kept interval); do nothing further.
4. Return $n - \text{kept}$ as the minimum number of removals.

Algorithm 15 Minimum Removals for Non-overlapping Intervals

```

1: procedure ERASEOVERLAPINTERVALS(intervals)
2:   sort intervals by end time, increasing
3:   lastEnd  $\leftarrow -\infty$ , kept  $\leftarrow 0$ 
4:   for each  $[s, e]$  in sorted order do
5:     if  $s \geq \textit{lastEnd}$  then
6:       kept  $\leftarrow \textit{kept} + 1$ 
7:       lastEnd  $\leftarrow e$ 
8:     end if
9:   end for
10:  return  $n - \textit{kept}$ 
11: end procedure

```

Dry run

Dry Run. Take $\textit{intervals} = [[1, 2], [2, 3], [3, 4], [1, 3]]$.

Sorted by end time: $[1, 2], [2, 3], [1, 3], [3, 4]$ (ties between intervals ending at the same time can be broken arbitrarily; here $[2, 3]$ and $[1, 3]$ both end at 3).

- $\textit{lastEnd} = -\infty$. Interval $[1, 2]$: $1 \geq -\infty$. Keep. $\textit{kept} = 1$, $\textit{lastEnd} = 2$.
- Interval $[2, 3]$: $2 \geq 2$. Keep (touching endpoints are allowed). $\textit{kept} = 2$, $\textit{lastEnd} = 3$.
- Interval $[1, 3]$: $1 \geq 3$? No. Remove (do not update $\textit{lastEnd}$ or $\textit{kept}$).
- Interval $[3, 4]$: $3 \geq 3$. Keep. $\textit{kept} = 3$, $\textit{lastEnd} = 4$.

We kept 3 out of 4 intervals, so the minimum number of removals is $4 - 3 = 1$ – specifically, removing $[1, 3]$ leaves $[1, 2], [2, 3], [3, 4]$, which are pairwise non-overlapping (touching only at shared endpoints).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int eraseOverlapIntervals(vector<vector<int>>& intervals) {
5      if (intervals.empty()) return 0;
6
7      sort(intervals.begin(), intervals.end(),
8           [](const vector<int>& a, const vector<int>& b) {
9               return a[1] < b[1];
10            });
11
12     int n = intervals.size();
13     int kept = 1;
14     int lastEnd = intervals[0][1];
15
16     for (int i = 1; i < n; i++) {
17         if (intervals[i][0] >= lastEnd) {
18             kept++;
19             lastEnd = intervals[i][1];
20         }
21     }
22     return n - kept;

```

```
23 }
24
25 int main() {
26     vector<vector<int>> intervals = {{1,2},{2,3},{3,4},{1,3}};
27     cout << "Minimum removals: "
28           << eraseOverlapIntervals(intervals) << endl;
29     return 0;
30 }
```

Complexity Analysis

Time Complexity. Sorting the n intervals by end time costs $O(n \log n)$. The subsequent single pass costs $O(n)$. The overall time complexity is

$$O(n \log n).$$

Space Complexity. Beyond the input array (sorted in place or with $O(n)$ auxiliary space, depending on the sort implementation), we use only a constant number of extra scalar variables, giving auxiliary space complexity $O(1)$.

Key Takeaway

Many “minimum removals/deletions to enforce property P ” problems are secretly “maximum subset satisfying property P ” problems wearing a disguise: the answer is n minus the size of the largest valid subset. Whenever you see a removal-minimization phrasing, check whether flipping it to a keep-maximization phrasing turns it into a pattern you already recognize – here, it turns directly into the activity selection problem from Section 1.7.

Chapter 2

Binary Search

Binary search is not a trick for sorted arrays. It is a trick for any space of candidate answers in which feasibility is monotonic – once you see that monotonicity, the array itself becomes optional.

2.1 What Is Binary Search, Really?

Most students first meet binary search as a way to find a target value in a sorted array: compare the middle element to the target, and discard half the array based on the result. That description is correct, but it is also dangerously narrow, because it hides the actual reason binary search works behind the specific setting of “a sorted array.” The real requirement is much more general, and recognizing it is what lets you apply binary search to problems that do not look anything like array search at first glance.

The One Property Binary Search Actually Needs

Binary search applies whenever you are searching over a totally ordered space of candidates (indices, integers, real numbers, even configurations) on which some predicate P is **monotonic** – that is, P is **false** for a prefix of the space and **true** for the remaining suffix (or vice versa), with a single crossover point. If such a predicate exists, binary search finds the crossover point in $O(\log(\text{range}))$ evaluations of P , regardless of what P actually computes or how expensive each evaluation is.

In classical array search, the candidate space is the array’s indices $0, 1, \dots, n - 1$, and the predicate is $P(i) = (\text{arr}[i] \geq \text{target})$ – which is monotonic precisely because the array is sorted. But the same machinery works just as well when the candidate space is, say, every integer capacity a ship could have, or every possible eating speed for Koko’s bananas, or every real number that could be a square root – as long as we can answer “does this candidate satisfy the requirement, or not” in a way that flips exactly once as the candidate increases. This chapter is organized around exactly that distinction.

Three Flavors of Binary Search in This Chapter

How to Tell Which Flavor You Are Facing

- **Binary search on a 1D array (Sections 2.2–2.14).** The keywords are usually “*sorted array*,” “*rotated sorted array*,” or “*find an index/value such that...*” The candidate space is literally the array’s indices, and lo, hi start as 0 and $n - 1$.
- **Binary search on the answer (Sections 2.15–2.28).** The keywords are “*minimize the maximum ...*,” “*maximize the minimum ...*,” or “*find the smallest/largest value such that some condition holds.*” Here there may be no array to search at all – the candidate space is a numeric range of possible answers (e.g. every integer from 1 to 10^9), and we binary search directly on that range, using a helper feasibility check (often itself an $O(n)$ scan) as the monotonic predicate.

- **Binary search on a 2D array (Sections 2.29–2.33).** The keywords are “*row-wise and/or column-wise sorted matrix.*” Here the candidate space is two-dimensional, and the technique is usually to either flatten the 2D index space into a 1D one, or to run a 1D binary search independently along each row or column, or to binary search on the answer (as above) using a 2D-aware feasibility check.

The single most important habit to build in this chapter is translating a problem’s wording into “what is the candidate space, and what is the monotonic predicate that flips exactly once across it” *before* writing any code. Once those two things are identified, the `while(lo <= hi)` skeleton is almost always the same.

The Skeleton, and Where Bugs Actually Come From

Nearly every binary search bug comes from getting one of three things wrong: the loop condition (`lo <= hi` vs. `lo < hi`), the update after a comparison (`mid - 1 / mid + 1` vs. `mid` itself), or an integer-overflow in the midpoint computation. We will use the convention

$$mid = lo + (hi - lo) / 2$$

throughout this chapter, rather than $(lo + hi) / 2$, because it avoids overflow when lo and hi are both close to the maximum representable integer – a habit worth having permanently, even in problems where the bound is small enough that it would not matter today.

Each section in this chapter follows the same structure used in the Greedy chapter: a precise problem statement and the keywords that identify it; identification of the candidate space and the monotonic predicate; the algorithm in words and in formal pseudocode; a hand-traced dry run; complete compilable C++ code; and a careful time/space complexity analysis.

2.2 Binary Search to Find X in a Sorted Array

Problem Statement. We are given an array arr of n elements, sorted in increasing order, and a target value X . We must determine the index of X in the array if it is present, and return -1 if it is not. We must do this faster than the obvious $O(n)$ linear scan.

Pattern Recognition

“Sorted array” appearing anywhere in a problem statement, combined with a request to *find/locate* a specific value or the boundary of some property, is the single most reliable trigger for binary search in this entire chapter. The reason a sorted array enables binary search is precisely the monotonicity property from Section 2.1: define $P(i) = (arr[i] \geq X)$; because the array is sorted, P is **false** for every index before X ’s position and **true** from X ’s position onward (or, if X is absent, from the position where elements first exceed X onward) – a single crossover, which is exactly what binary search is built to find.

Understanding the problem carefully

If the array were unsorted, knowing that $arr[mid] \neq X$ would tell us nothing about where X might be – it could be anywhere among the remaining elements, so we would have no choice but to check them all. Sortedness changes this completely: if $arr[mid] < X$, then because every element to the left of mid is even smaller (the array is sorted in increasing order), *none* of them can equal X either, so the entire left half can be discarded in one step. Symmetrically, if $arr[mid] > X$, the entire right half can be discarded. Only when $arr[mid] = X$ do we stop, having found the answer.

Candidate Space and Predicate

The candidate space is the set of indices $\{0, 1, \dots, n - 1\}$. We maintain a search window $[lo, hi]$, initially the whole array, and repeatedly look at the midpoint, using the comparison with X to discard exactly the half of the window that provably cannot contain X .

Why halving the search space each time is correct

Each comparison at mid produces one of exactly three outcomes, and in every outcome we either find the answer immediately or can prove that an entire half of the remaining window is irrelevant. Since no element is ever incorrectly discarded (the sortedness guarantee makes every discard provably safe) and the window strictly shrinks by roughly half on every iteration, the process must terminate, either by finding X or by the window becoming empty ($lo > hi$), at which point we can safely conclude X is not present.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo) / 2$.
 - (b) If $arr[mid] = X$: return mid .
 - (c) If $arr[mid] < X$: $lo \leftarrow mid + 1$.
 - (d) Otherwise: $hi \leftarrow mid - 1$.
3. If the loop ends without returning, return -1 .

Algorithm 16 Binary Search

```

1: procedure BINARYSEARCH( $arr, X$ )
2:    $lo \leftarrow 0, hi \leftarrow n - 1$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo) / 2$ 
5:     if  $arr[mid] = X$  then
6:       return  $mid$ 
7:     else if  $arr[mid] < X$  then
8:        $lo \leftarrow mid + 1$ 
9:     else
10:       $hi \leftarrow mid - 1$ 
11:    end if
12:  end while
13:  return  $-1$ 
14: end procedure

```

Dry run

Dry Run. Take $arr = [-1, 0, 3, 5, 9, 12]$, $X = 9$.

- $lo = 0, hi = 5$: $mid = 2$, $arr[2] = 3 < 9$. $lo = 3$.
- $lo = 3, hi = 5$: $mid = 4$, $arr[4] = 9 = 9$. Return 4.

The answer is index 4.

Now take $X = 2$ on the same array.

- $lo = 0, hi = 5$: $mid = 2$, $arr[2] = 3 > 2$. $hi = 1$.
- $lo = 0, hi = 1$: $mid = 0$, $arr[0] = -1 < 2$. $lo = 1$.
- $lo = 1, hi = 1$: $mid = 1$, $arr[1] = 0 < 2$. $lo = 2$.
- $lo = 2 > hi = 1$: loop ends. Return -1 .

The answer is -1 : 2 is not present in the array.

C++ implementation (iterative and recursive)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Iterative version
5 int binarySearch(vector<int>& arr, int X) {
6   int lo = 0, hi = arr.size() - 1;
7
8   while (lo <= hi) {
9     int mid = lo + (hi - lo) / 2;
10    if (arr[mid] == X) return mid;
11    else if (arr[mid] < X) lo = mid + 1;
12    else hi = mid - 1;
13  }
14  return -1;
15 }
16

```

```

17 // Recursive version
18 int binarySearchRecursive(vector<int>& arr, int lo, int hi, int X) {
19     if (lo > hi) return -1;
20
21     int mid = lo + (hi - lo) / 2;
22     if (arr[mid] == X) return mid;
23     else if (arr[mid] < X) return binarySearchRecursive(arr, mid + 1,
24                                                         hi, X);
25     else return binarySearchRecursive(arr, lo, mid - 1, X);
26 }
27
28 int main() {
29     vector<int> arr = {-1, 0, 3, 5, 9, 12};
30     cout << "Iterative: " << binarySearch(arr, 9) << endl;
31     cout << "Recursive: "
32          << binarySearchRecursive(arr, 0, arr.size() - 1, 9) << endl;
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. Each iteration (or recursive call) discards at least half of the remaining search window, so the window shrinks from size n to size 0 in at most $\lceil \log_2 n \rceil + 1$ steps, giving

$$O(\log n).$$

Space Complexity. The iterative version uses only the three scalar variables lo , hi , mid , giving $O(1)$ space. The recursive version uses $O(\log n)$ space for the call stack, since the recursion depth equals the number of halvings performed.

Key Takeaway

Binary search's correctness rests entirely on being able to *prove*, at each step, that an entire half of the remaining search space cannot contain the answer. In plain array search this proof comes directly from sortedness; every other binary search problem in this chapter is really asking you to find or construct an equivalent monotonicity proof in a different setting.

2.3 Implement Lower Bound

Problem Statement. We are given a sorted array arr of n elements and a value X . We must find the **lower bound** of X : the index of the first element in arr that is **greater than or equal to** X . If no such element exists (every element is smaller than X), the lower bound is n (one past the end of the array).

Pattern Recognition

“Find the first/smallest index such that some condition holds” – here, the condition is $arr[i] \geq X$ – is the **find-the-boundary** variant of binary search, distinct from the exact-match search of Section 2.2. The keyword is not “find X ” but “**find the first position where a property becomes true.**” This phrasing recurs constantly throughout this chapter (first/last occurrence, search insert position, floor/ceil, and many “binary search on the answer” problems), so internalizing the boundary-search template here pays off many times over.

Understanding the problem carefully

Define the predicate $P(i) = (arr[i] \geq X)$. Because the array is sorted in increasing order, P is **false** for some prefix of indices and **true** for the remaining suffix, with exactly one crossover point – the lower bound is, by definition, that crossover index. Unlike Section 2.2, we are not looking for a single exact match; we are looking for the *boundary* between “too small” and “big enough,” which exists regardless of whether X itself is actually present in the array.

Candidate Space and Predicate

The candidate space is indices $0, \dots, n$ (note: n itself is a valid answer, representing “no element is large enough”). The predicate is $P(i) = (arr[i] \geq X)$ for $i < n$, and we want the smallest index where P holds.

Why we keep a separate “best answer so far” variable

In exact-match search, the loop stops the instant we find a match. Here, we cannot stop early, because finding *one* index where $arr[i] \geq X$ does not tell us it is the *first* such index – there might be an even smaller valid index hiding in the left half. So instead of returning immediately, whenever $arr[mid] \geq X$ we record mid as a candidate answer and *keep searching to the left*, hoping to find an even smaller valid index; whenever $arr[mid] < X$, we know the entire left portion up to and including mid is too small, so we discard it and search to the right. This is the general template for every “find the first/last index satisfying a monotonic property” problem: maintain a running best answer, and keep shrinking the window based on the predicate, rather than stopping on the first observed success.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$, $ans \leftarrow n$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) If $arr[mid] \geq X$: record $ans \leftarrow mid$, and search left: $hi \leftarrow mid - 1$.
 - (c) Otherwise: search right: $lo \leftarrow mid + 1$.
3. Return ans .

Algorithm 17 Lower Bound

```

1: procedure LOWERBOUND( $arr, X$ )
2:    $lo \leftarrow 0, hi \leftarrow n - 1, ans \leftarrow n$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:     if  $arr[mid] \geq X$  then
6:        $ans \leftarrow mid$ 
7:        $hi \leftarrow mid - 1$ 
8:     else
9:        $lo \leftarrow mid + 1$ 
10:    end if
11:  end while
12:  return  $ans$ 
13: end procedure

```

Dry run

Dry Run. Take $arr = [1, 2, 2, 3]$, $X = 2$.

- $lo = 0, hi = 3, ans = 4$: $mid = 1, arr[1] = 2 \geq 2$. $ans = 1, hi = 0$.
- $lo = 0, hi = 0$: $mid = 0, arr[0] = 1 \geq 2$? No. $lo = 1$.
- $lo = 1 > hi = 0$: loop ends.

The lower bound is index 1 – the first occurrence of 2.

Now take $X = 5$ on the same array (larger than every element).

- $lo = 0, hi = 3, ans = 4$: $mid = 1, arr[1] = 2 \geq 5$? No. $lo = 2$.
- $lo = 2, hi = 3$: $mid = 2, arr[2] = 2 \geq 5$? No. $lo = 3$.
- $lo = 3, hi = 3$: $mid = 3, arr[3] = 3 \geq 5$? No. $lo = 4$.
- $lo = 4 > hi = 3$: loop ends. ans remains 4 (its initial value, never overwritten).

The lower bound is $4 = n$, correctly signaling that no element is large enough.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lowerBound(vector<int>& arr, int X) {
5     int n = arr.size();
6     int lo = 0, hi = n - 1, ans = n;
7
8     while (lo <= hi) {
9         int mid = lo + (hi - lo) / 2;
10        if (arr[mid] >= X) {
11            ans = mid;
12            hi = mid - 1;
13        } else {
14            lo = mid + 1;
15        }
16    }
17    return ans;
18 }
19
20 int main() {
21     vector<int> arr = {1, 2, 2, 3};
22     cout << "Lower bound of 2: " << lowerBound(arr, 2) << endl;
23     cout << "Lower bound of 5: " << lowerBound(arr, 5) << endl;
24     return 0;
25 }
```

Complexity Analysis

Time Complexity. The search window halves each iteration, giving

$$O(\log n).$$

Space Complexity. Only a constant number of scalar variables are used, giving $O(1)$ space.

Key Takeaway

Whenever a problem asks for the *first* (or *last*) index satisfying a monotonic condition, do not stop the binary search the moment you observe the condition holding – instead, record it as a tentative answer and keep narrowing the search toward the boundary, in the direction that could produce an even better (earlier, or later) answer. This “record and keep narrowing” template is reused, almost verbatim, in upper bound, floor/ceil, and first/last occurrence.

2.4 Implement Upper Bound

Problem Statement. We are given a sorted array *arr* of *n* elements and a value *X*. We must find the **upper bound** of *X*: the index of the first element in *arr* that is **strictly greater than** *X*. If no such element exists, the upper bound is *n*.

Pattern Recognition

Upper bound is lower bound’s twin, with one word changed: “greater than or equal to” becomes “strictly greater than.” Whenever you see “*first index strictly greater than X*” as opposed to “*first index at least X*,” you are being asked for an upper bound rather than a lower bound. The two are easy to confuse under time pressure, so it is worth fixing the distinction precisely: lower bound finds where *X could first appear*; upper bound finds where the elements equal to *X end*.

Understanding the problem carefully

The predicate here is $P(i) = (arr[i] > X)$, which, exactly as in lower bound, is **false** for a prefix and **true** for a suffix of the sorted array, with a single crossover. The only change from Section 2.3 is the strictness of the comparison: an element exactly equal to *X* satisfies the lower-bound predicate ($\geq X$) but *not* the upper-bound predicate ($> X$). Consequently, if *X* appears multiple times in the array, the lower bound points to the first occurrence of *X*, while the upper bound points to the index *immediately after* the last occurrence of *X* – a fact we will use directly in Section 2.7 and Section 2.8.

Candidate Space and Predicate

The candidate space is indices $0, \dots, n$. The predicate is $P(i) = (arr[i] > X)$ for $i < n$, and we want the smallest index where *P* holds.

Why the same template applies, with one flipped comparison

The reasoning is identical to lower bound: whenever $arr[mid] > X$, this index is a valid candidate for the upper bound, but there might be an even earlier index that is also strictly greater than *X*, so we record it and search left. Whenever $arr[mid] \leq X$ (note: this now includes equality, unlike lower bound), this index and everything before it is disqualified, so we search right. The only code change relative to lower bound is replacing the comparison `arr[mid] >= X` with `arr[mid] > X`.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$, $ans \leftarrow n$.
2. While $lo \leq hi$:

- (a) $mid \leftarrow lo + (hi - lo)/2$.
- (b) If $arr[mid] > X$: record $ans \leftarrow mid$; $hi \leftarrow mid - 1$.
- (c) Otherwise: $lo \leftarrow mid + 1$.

3. Return ans .

Algorithm 18 Upper Bound

```

1: procedure UPPERBOUND( $arr, X$ )
2:    $lo \leftarrow 0, hi \leftarrow n - 1, ans \leftarrow n$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:     if  $arr[mid] > X$  then
6:        $ans \leftarrow mid$ 
7:        $hi \leftarrow mid - 1$ 
8:     else
9:        $lo \leftarrow mid + 1$ 
10:    end if
11:  end while
12:  return  $ans$ 
13: end procedure

```

Dry run

Dry Run. Take $arr = [1, 2, 2, 3]$, $X = 2$.

- $lo = 0, hi = 3, ans = 4$: $mid = 1$, $arr[1] = 2 > 2$? No. $lo = 2$.
- $lo = 2, hi = 3$: $mid = 2$, $arr[2] = 2 > 2$? No. $lo = 3$.
- $lo = 3, hi = 3$: $mid = 3$, $arr[3] = 3 > 2$? Yes. $ans = 3$, $hi = 2$.
- $lo = 3 > hi = 2$: loop ends.

The upper bound is index 3 – one past the last occurrence of 2 (which sits at index 2), consistent with the lower bound of 2 found in Section 2.3 being index 1: the occurrences of 2 span indices $[1, 2]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int upperBound(vector<int>& arr, int X) {
5     int n = arr.size();
6     int lo = 0, hi = n - 1, ans = n;
7
8     while (lo <= hi) {
9         int mid = lo + (hi - lo) / 2;
10        if (arr[mid] > X) {
11            ans = mid;
12            hi = mid - 1;
13        } else {
14            lo = mid + 1;

```

```

15     }
16 }
17 return ans;
18 }
19
20 int main() {
21     vector<int> arr = {1, 2, 2, 3};
22     cout << "Upper bound of 2: " << upperBound(arr, 2) << endl;
23     return 0;
24 }

```

Complexity Analysis

Time Complexity. $O(\log n)$, identical to lower bound, since the algorithm structure is unchanged. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

Lower bound and upper bound differ by exactly one comparison operator ($\geq$ versus $>$), but that single difference is what makes the pair together so useful: the number of occurrences of X in a sorted array is always $upperBound(X) - lowerBound(X)$, and the first and last occurrence indices of X are $lowerBound(X)$ and $upperBound(X) - 1$ respectively (when X is present at all).

2.5 Search Insert Position

Problem Statement. We are given a sorted array arr of n distinct integers and a target value X . If X is found in the array, return its index. If not, return the index at which X **would be inserted** to keep the array sorted.

Pattern Recognition

This problem is worth pausing on specifically *because* it looks new but is not: “the index where X would be inserted to keep the array sorted” is, word for word, a description of the **lower bound** of X from Section 2.3. The position just before the first element $\geq X$ is exactly where X slots in. Recognizing when a new-sounding problem statement is a known pattern in disguise is as important a skill in this chapter as deriving a pattern from scratch.

Understanding the problem carefully

If X is present in the array, the lower bound of X is precisely the index of X (since the array has distinct elements, the first index with $arr[i] \geq X$ is the unique index where $arr[i] = X$). If X is absent, the lower bound is the first index whose value exceeds X – which is exactly the index where X should be inserted, since everything before that index is smaller than X and everything from that index onward is larger.

Candidate Space and Predicate

Identical to lower bound: candidate space is indices $0, \dots, n$, predicate $P(i) = (arr[i] \geq X)$, answer is the smallest index where P holds.

Why no new derivation is needed

There is nothing new to prove here – this section exists to train the habit of *reducing* an unfamiliar-sounding problem to a pattern you already have a proven, working template for, rather than re-deriving binary search bounds from scratch every time a problem uses different words for the same underlying question.

Algorithm

Identical to Algorithm 2.3’s lower-bound procedure, called directly on X .

Dry run

Dry Run. Take $arr = [1, 3, 5, 6]$, $X = 5$.

This is present at index 2; the lower bound of 5 is index 2 (verify: $arr[2] = 5 \geq 5$, and $arr[1] = 3 < 5$), so the answer is 2.

Now take $X = 2$ on the same array. 2 is absent. The lower bound of 2 is index 1 (the first element ≥ 2 is $arr[1] = 3$), so 2 would be inserted at index 1, giving $[1, 2, 3, 5, 6]$ – correct.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int searchInsert(vector<int>& arr, int X) {
5      int n = arr.size();
6      int lo = 0, hi = n - 1, ans = n;
7
8      while (lo <= hi) {
9          int mid = lo + (hi - lo) / 2;
10         if (arr[mid] >= X) {
11             ans = mid;
12             hi = mid - 1;
13         } else {
14             lo = mid + 1;
15         }
16     }
17     return ans;
18 }
19
20 int main() {
21     vector<int> arr = {1, 3, 5, 6};
22     cout << "Insert position for 5: " << searchInsert(arr, 5) << endl;
23     cout << "Insert position for 2: " << searchInsert(arr, 2) << endl;
24     return 0;
25 }
```

Complexity Analysis

Time Complexity. $O(\log n)$. **Space Complexity.** $O(1)$.

Key Takeaway

Before deriving a new binary search predicate from scratch, check whether the problem is just lower bound, upper bound, or a simple combination of the two, restated in different words. Many “new” binary search problems on interview sheets are exactly this.

2.6 Floor and Ceil in a Sorted Array

Problem Statement. We are given a sorted array arr of n elements and a value X . We must find the **floor** of X – the largest element in arr that is $\leq X$ – and the **ceil** of X – the smallest element in arr that is $\geq X$. If no floor (resp. ceil) exists, report -1 .

Pattern Recognition

“*Floor*” and “*ceil*” are simply “largest $\leq X$ ” and “smallest $\geq X$ ” – and “smallest $\geq X$ ” is, value-for-value, the *value* sitting at the lower-bound index from Section 2.3. “Largest $\leq X$ ” is its mirror image, and is found with the symmetric boundary-search template, narrowing toward the right instead of the left. Whenever a problem uses the words “*floor*” or “*ceil*” (or, equivalently, “*closest element not exceeding*” / “*closest element not less than*”), translate it immediately into a boundary search.

Understanding the problem carefully

Ceil of X : define $P(i) = (arr[i] \geq X)$. As in lower bound, find the smallest index where P holds; the ceil is the value at that index (or -1 if no such index exists, i.e. X exceeds every element).

Floor of X : define $Q(i) = (arr[i] \leq X)$. This predicate is **true** for a *prefix* of the array and **false** for the remaining suffix – the mirror image of P . We want the *largest* index where Q holds, so instead of narrowing left after a success (as in lower/upper bound), we narrow *right* after a success, since we are now searching for the rightmost true position rather than the leftmost one.

Candidate Space and Predicate

For ceil: candidate space is indices $0, \dots, n-1$ (or “not found”), predicate $P(i) = (arr[i] \geq X)$, want smallest true index. For floor: same candidate space, predicate $Q(i) = (arr[i] \leq X)$, want *largest* true index.

Why floor needs the mirrored template

In every boundary search so far (lower bound, upper bound), we wanted the *leftmost* index satisfying a predicate that becomes true only after some crossover point, so a successful check let us narrow the window to the *left*, hunting for an even earlier success. Floor flips this: the predicate $arr[i] \leq X$ is true *before* the crossover and false after, so we want the *rightmost* successful index, meaning a successful check should narrow the window to the *right*, hunting for an even later success that is still within the true region. This mirrored narrowing direction is the only structural change; everything else about the binary search skeleton is identical.

Algorithm

Ceil:

1. $lo \leftarrow 0, hi \leftarrow n-1, ceil \leftarrow -1$.
2. While $lo \leq hi$: $mid \leftarrow lo + (hi - lo)/2$. If $arr[mid] \geq X$: $ceil \leftarrow arr[mid], hi \leftarrow mid - 1$. Else: $lo \leftarrow mid + 1$.
3. Return $ceil$.

Floor:

1. $lo \leftarrow 0, hi \leftarrow n - 1, floor \leftarrow -1$.
2. While $lo \leq hi$: $mid \leftarrow lo + (hi - lo)/2$. If $arr[mid] \leq X$: $floor \leftarrow arr[mid], lo \leftarrow mid + 1$.
Else: $hi \leftarrow mid - 1$.
3. Return $floor$.

Algorithm 19 Floor and Ceil

```

1: procedure FLOORCEIL( $arr, X$ )
2:    $lo \leftarrow 0, hi \leftarrow n - 1$ 
3:    $floor \leftarrow -1, ceil \leftarrow -1$ 
4:   while  $lo \leq hi$  do
5:      $mid \leftarrow lo + (hi - lo)/2$ 
6:     if  $arr[mid] \leq X$  then
7:        $floor \leftarrow arr[mid], lo \leftarrow mid + 1$ 
8:     else
9:        $hi \leftarrow mid - 1$ 
10:    end if
11:  end while
12:  (reset  $lo, hi$  and repeat symmetrically for ceil)
13:  return ( $floor, ceil$ )
14: end procedure

```

Dry run

Dry Run. Take $arr = [3, 4, 4, 7, 8, 10]$, $X = 5$.

Ceil (smallest ≥ 5):

- $lo = 0, hi = 5$: $mid = 2, arr[2] = 4 \geq 5$? No. $lo = 3$.
- $lo = 3, hi = 5$: $mid = 4, arr[4] = 8 \geq 5$? Yes. $ceil = 8, hi = 3$.
- $lo = 3, hi = 3$: $mid = 3, arr[3] = 7 \geq 5$? Yes. $ceil = 7, hi = 2$.
- $lo = 3 > hi = 2$: loop ends. $ceil = 7$.

Floor (largest ≤ 5):

- $lo = 0, hi = 5$: $mid = 2, arr[2] = 4 \leq 5$? Yes. $floor = 4, lo = 3$.
- $lo = 3, hi = 5$: $mid = 4, arr[4] = 8 \leq 5$? No. $hi = 3$.
- $lo = 3, hi = 3$: $mid = 3, arr[3] = 7 \leq 5$? No. $hi = 2$.
- $lo = 3 > hi = 2$: loop ends. $floor = 4$.

So $floor(5) = 4$ and $ceil(5) = 7$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 pair<int, int> floorCeil(vector<int>& arr, int X) {
5     int n = arr.size();
6     int floorVal = -1, ceilVal = -1;

```

```

7
8 // Floor: largest element <= X
9 int lo = 0, hi = n - 1;
10 while (lo <= hi) {
11     int mid = lo + (hi - lo) / 2;
12     if (arr[mid] <= X) {
13         floorVal = arr[mid];
14         lo = mid + 1;
15     } else {
16         hi = mid - 1;
17     }
18 }
19
20 // Ceil: smallest element >= X
21 lo = 0; hi = n - 1;
22 while (lo <= hi) {
23     int mid = lo + (hi - lo) / 2;
24     if (arr[mid] >= X) {
25         ceilVal = arr[mid];
26         hi = mid - 1;
27     } else {
28         lo = mid + 1;
29     }
30 }
31
32 return {floorVal, ceilVal};
33 }
34
35 int main() {
36     vector<int> arr = {3, 4, 4, 7, 8, 10};
37     auto [floorVal, ceilVal] = floorCeil(arr, 5);
38     cout << "Floor: " << floorVal << ", Ceil: " << ceilVal << endl;
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. Two independent binary searches, each $O(\log n)$, giving an overall time complexity of

$$O(\log n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Floor and ceil are the value-returning siblings of lower and upper bound. Whichever direction the predicate's "true" region lies in (a prefix or a suffix of the sorted array) tells you which direction to narrow the search window after a successful check: narrow toward the *far* end of the true region if you want its near boundary, or keep the running answer updated while narrowing toward the *near* end if you want the boundary closest to where you started.

2.7 Find the First and Last Occurrence of an Element

Problem Statement. We are given a sorted array *arr* of *n* elements, possibly with duplicates, and a target *X*. We must find the index of the **first** occurrence of *X* and the index of the **last**

occurrence of X . If X is not present, both should be -1 .

Pattern Recognition

“First occurrence” and “last occurrence” of a value in a sorted array are, respectively, exactly the lower bound and (upper bound -1) of that value, provided the value is actually present. This section is mostly about correctly handling the “what if it is not present at all” case, since lower bound and upper bound alone do not distinguish between “ X is present, starting here” and “ X is absent, and this is just where it would be inserted.”

Understanding the problem carefully

Recall from Section 2.4 that, when X is present,

$$\text{firstOccurrence} = \text{lowerBound}(X), \quad \text{lastOccurrence} = \text{upperBound}(X) - 1.$$

The subtlety is verifying presence. The lower bound of X gives the first index whose value is $\geq X$; this is the same as the first occurrence of X *only if* the value actually stored at that index equals X (and the index is within bounds). If $\text{lowerBound}(X) = n$, or if $\text{arr}[\text{lowerBound}(X)] \neq X$, then X is simply not in the array at all.

Approach

Compute $lb \leftarrow \text{lowerBound}(X)$. If $lb = n$ or $\text{arr}[lb] \neq X$, return $(-1, -1)$. Otherwise, the first occurrence is lb , and the last occurrence is $\text{upperBound}(X) - 1$.

Why checking presence only once is sufficient

Because the array is sorted, every occurrence of X forms a single contiguous block. If the very first candidate position (the lower bound) does not actually hold X , no later position can hold X either – the lower bound is, by construction, the leftmost position any element $\geq X$ could occupy, so if even that position fails to be exactly X , X does not appear anywhere in the array. This means a single equality check after computing the lower bound is enough to settle presence, with no need for an additional independent search.

Algorithm

1. $lb \leftarrow \text{lowerBound}(\text{arr}, X)$ (using Algorithm 2.3).
2. If $lb = n$ or $\text{arr}[lb] \neq X$: return $(-1, -1)$.
3. Otherwise: $ub \leftarrow \text{upperBound}(\text{arr}, X)$ (using Algorithm 2.4). Return $(lb, ub - 1)$.

Algorithm 20 First and Last Occurrence

```

1: procedure FIRSTANDLAST( $\text{arr}, X$ )
2:    $lb \leftarrow \text{LOWERBOUND}(\text{arr}, X)$ 
3:   if  $lb = n$  or  $\text{arr}[lb] \neq X$  then
4:     return  $(-1, -1)$ 
5:   end if
6:    $ub \leftarrow \text{UPPERBOUND}(\text{arr}, X)$ 
7:   return  $(lb, ub - 1)$ 
8: end procedure

```

Dry run

Dry Run. Take $arr = [5, 7, 7, 8, 8, 10]$, $X = 8$.

$lowerBound(8)$: scanning, the first index with value ≥ 8 is index 3 ($arr[3] = 8$). Since $arr[3] = 8 = X$, X is present. $upperBound(8)$: the first index with value > 8 is index 5 ($arr[5] = 10$). So $lastOccurrence = 5 - 1 = 4$.

The answer is (first = 3, last = 4).

Now take $X = 6$ on the same array. $lowerBound(6) = 1$ ($arr[1] = 7 \geq 6$ is the first such index). But $arr[1] = 7 \neq 6$, so 6 is not present, and the answer is $(-1, -1)$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int lowerBound(vector<int>& arr, int X) {
5      int n = arr.size();
6      int lo = 0, hi = n - 1, ans = n;
7      while (lo <= hi) {
8          int mid = lo + (hi - lo) / 2;
9          if (arr[mid] >= X) { ans = mid; hi = mid - 1; }
10         else lo = mid + 1;
11     }
12     return ans;
13 }
14
15 int upperBound(vector<int>& arr, int X) {
16     int n = arr.size();
17     int lo = 0, hi = n - 1, ans = n;
18     while (lo <= hi) {
19         int mid = lo + (hi - lo) / 2;
20         if (arr[mid] > X) { ans = mid; hi = mid - 1; }
21         else lo = mid + 1;
22     }
23     return ans;
24 }
25
26 pair<int,int> firstAndLast(vector<int>& arr, int X) {
27     int n = arr.size();
28     int lb = lowerBound(arr, X);
29     if (lb == n || arr[lb] != X) return {-1, -1};
30
31     int ub = upperBound(arr, X);
32     return {lb, ub - 1};
33 }
34
35 int main() {
36     vector<int> arr = {5, 7, 7, 8, 8, 10};
37     auto [first, last] = firstAndLast(arr, 8);
38     cout << "First: " << first << ", Last: " << last << endl;
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. Two binary searches (lower bound and, if needed, upper bound), each $O(\log n)$, giving an overall time complexity of

$$O(\log n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Lower bound and upper bound, used together with a single presence check, solve any “range of occurrences” question about a sorted array. There is rarely a need to write a bespoke binary search for “first occurrence” or “last occurrence” from scratch – compose them from the lower/upper bound primitives instead.

2.8 Count Occurrences of an Element in a Sorted Array

Problem Statement. We are given a sorted array *arr* of *n* elements, possibly with duplicates, and a target *X*. We must count how many times *X* occurs in *arr*.

Pattern Recognition

“Count occurrences of *X* in a sorted array” is immediately solvable once you recall that all occurrences of *X* occupy a single contiguous block (Section 2.7), bounded by the lower bound and upper bound of *X*. The keyword to internalize: any time a sorted-array problem asks “how many,” check first whether the answer is simply the *width* of a contiguous range you can find with two boundary searches, rather than something requiring an actual scan.

Understanding the problem carefully

The lower bound of *X* is the first index where *X* could occur; the upper bound of *X* is the first index strictly past where *X* can occur. Every index in between holds a value equal to *X* (since the array is sorted and both boundaries are tight), and no index outside that range does. So the count of occurrences is simply the number of indices in $[lowerBound(X), upperBound(X))$, i.e.

$$count = upperBound(X) - lowerBound(X).$$

This formula is correct even when *X* is entirely absent: in that case $lowerBound(X) = upperBound(X)$ (both point to the same insertion position), giving a count of 0 automatically, with no special case needed.

Why No Presence Check Is Needed Here

Unlike Section 2.7, where we needed an explicit presence check before reporting indices (an absent element has no valid index to report), here the quantity we want is a *count*, and the subtraction $upperBound(X) - lowerBound(X)$ naturally evaluates to 0 when *X* is absent, without requiring a separate branch.

Algorithm

1. $lb \leftarrow lowerBound(arr, X)$.
2. $ub \leftarrow upperBound(arr, X)$.

3. Return $ub - lb$.

Algorithm 21 Count Occurrences

```

1: procedure COUNTOccurrences(arr, X)
2:   lb  $\leftarrow$  LOWERBOUND(arr, X)
3:   ub  $\leftarrow$  UPPERBOUND(arr, X)
4:   return ub - lb
5: end procedure

```

Dry run

Dry Run. Take $arr = [2, 4, 4, 4, 4, 8, 10]$, $X = 4$.

$lowerBound(4)$: first index with value ≥ 4 is index 1. $upperBound(4)$: first index with value > 4 is index 5 ($arr[5] = 8$). So $count = 5 - 1 = 4$, correctly matching the four occurrences of 4 at indices 1, 2, 3, 4.

Now take $X = 6$ on the same array (absent). Both $lowerBound(6)$ and $upperBound(6)$ equal 5 (the first index with value ≥ 6 , and also the first index with value > 6 , is index 5, where $arr[5] = 8$). So $count = 5 - 5 = 0$, correctly reporting absence.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int lowerBound(vector<int>& arr, int X) {
5      int n = arr.size();
6      int lo = 0, hi = n - 1, ans = n;
7      while (lo <= hi) {
8          int mid = lo + (hi - lo) / 2;
9          if (arr[mid] >= X) { ans = mid; hi = mid - 1; }
10         else lo = mid + 1;
11     }
12     return ans;
13 }
14
15 int upperBound(vector<int>& arr, int X) {
16     int n = arr.size();
17     int lo = 0, hi = n - 1, ans = n;
18     while (lo <= hi) {
19         int mid = lo + (hi - lo) / 2;
20         if (arr[mid] > X) { ans = mid; hi = mid - 1; }
21         else lo = mid + 1;
22     }
23     return ans;
24 }
25
26 int countOccurrences(vector<int>& arr, int X) {
27     return upperBound(arr, X) - lowerBound(arr, X);
28 }
29
30 int main() {
31     vector<int> arr = {2, 4, 4, 4, 4, 8, 10};
32     cout << "Occurrences of 4: " << countOccurrences(arr, 4) << endl;
33     cout << "Occurrences of 6: " << countOccurrences(arr, 6) << endl;

```

```

34     return 0;
35 }

```

Complexity Analysis

Time Complexity. Two binary searches, each $O(\log n)$, giving

$$O(\log n),$$

a dramatic improvement over the $O(n)$ linear scan that counting occurrences would otherwise require. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

“How many elements satisfy property P ” on a sorted array, where P itself is monotonic (true on a contiguous range), almost always reduces to “find the two boundaries of the true range and subtract.” This is a strictly more general restatement of the lower/upper-bound subtraction trick, worth recognizing beyond just counting exact value matches.

2.9 Search in a Rotated Sorted Array I

Problem Statement. An array that was originally sorted in increasing order has been **rotated** at some unknown pivot – for example, $[0, 1, 2, 4, 5, 6, 7]$ rotated at index 3 becomes $[4, 5, 6, 7, 0, 1, 2]$. All elements are distinct. Given such a rotated array arr and a target X , find the index of X , or return -1 if it is not present. We must still achieve $O(\log n)$ time.

Pattern Recognition

“*Rotated sorted array*” is a clear signal that plain binary search (Section 2.2) cannot be applied directly, because the array as a whole is no longer globally sorted – comparing $arr[mid]$ to X no longer tells us which half to discard. But the keyword “*sorted, then rotated*” carries a crucial piece of remaining structure: although the *whole* array is not sorted, **at least one of the two halves split at any midpoint always is**. Whenever a problem mentions rotation, immediately look for this “one half is always sorted” structure, since it is what restores binary search’s monotonic-discard guarantee.

Understanding the problem carefully

Pick any midpoint mid in a rotated sorted array. The two halves $[lo, mid]$ and $[mid, hi]$ cannot both contain the rotation pivot (the single place where a larger element is immediately followed by a smaller one) – the pivot lies in at most one of the two halves, so the other half is guaranteed to be a genuinely sorted, unrotated run of increasing values. We can detect which half is sorted with one comparison: if $arr[lo] \leq arr[mid]$, the left half is sorted; otherwise, the right half must be sorted.

Greedy Decision at Each Step

At each step, first determine which half is sorted using $arr[lo] \leq arr[mid]$. Then, check whether X could possibly lie within that sorted half’s value range (using ordinary comparisons, since that half really is sorted). If X ’s value falls within the sorted half’s range, discard the other half and recurse into the sorted half; otherwise, X (if present at all) must be in the *other* half, so discard the sorted half and recurse into the unsorted-looking half instead – which, on the next iteration, will itself be split into one sorted and one possibly-unsorted

piece, maintaining the invariant.

Why this discard rule is correct

The key invariant maintained throughout is that the current search window $[lo, hi]$ is itself a rotated sorted array (a sub-array of the original, which inherits the “rotated sorted” structure since removing elements from one end of a rotated sorted run leaves a smaller rotated sorted run). At every step we identify a genuinely sorted half by comparing its endpoints directly – a valid comparison because that half truly has no rotation inside it. If X lies within that sorted half’s numeric range (i.e. between its smallest and largest values), it can only be found there, since the other half’s values lie entirely outside this range (otherwise the sortedness of the identified half, combined with the single-rotation-point structure, would be violated). So discarding the half that provably cannot contain X is always safe, exactly mirroring the discard argument from ordinary binary search, just with one extra case-split to first identify which half is safe to reason about directly.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) If $arr[mid] = X$: return mid .
 - (c) If $arr[lo] \leq arr[mid]$ (left half is sorted):
 - If $arr[lo] \leq X < arr[mid]$: search left, $hi \leftarrow mid - 1$.
 - Else: search right, $lo \leftarrow mid + 1$.
 - (d) Otherwise (right half is sorted):
 - If $arr[mid] < X \leq arr[hi]$: search right, $lo \leftarrow mid + 1$.
 - Else: search left, $hi \leftarrow mid - 1$.
3. If the loop ends without returning, return -1 .

Algorithm 22 Search in Rotated Sorted Array (Distinct Elements)

```

1: procedure SEARCHROTATED( $arr, X$ )
2:    $lo \leftarrow 0, hi \leftarrow n - 1$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo) / 2$ 
5:     if  $arr[mid] = X$  then return  $mid$ 
6:     end if
7:     if  $arr[lo] \leq arr[mid]$  then
8:       if  $arr[lo] \leq X$  and  $X < arr[mid]$  then
9:          $hi \leftarrow mid - 1$ 
10:      else
11:         $lo \leftarrow mid + 1$ 
12:      end if
13:    else
14:      if  $arr[mid] < X$  and  $X \leq arr[hi]$  then
15:         $lo \leftarrow mid + 1$ 
16:      else
17:         $hi \leftarrow mid - 1$ 
18:      end if
19:    end if
20:  end while
21:  return  $-1$ 
22: end procedure

```

Dry run

Dry Run. Take $arr = [4, 5, 6, 7, 0, 1, 2]$, $X = 0$.

- $lo = 0, hi = 6$: $mid = 3$, $arr[3] = 7 \neq 0$. Is $arr[0] = 4 \leq arr[3] = 7$? Yes, left half sorted. Is $4 \leq 0 < 7$? No ($0 < 4$). So search right: $lo = 4$.
- $lo = 4, hi = 6$: $mid = 5$, $arr[5] = 1 \neq 0$. Is $arr[4] = 0 \leq arr[5] = 1$? Yes, left half sorted. Is $0 \leq 0 < 1$? Yes. Search left: $hi = 4$.
- $lo = 4, hi = 4$: $mid = 4$, $arr[4] = 0 = 0$. Return 4.

The answer is index 4.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int searchRotated(vector<int>& arr, int X) {
5     int n = arr.size();
6     int lo = 0, hi = n - 1;
7
8     while (lo <= hi) {
9         int mid = lo + (hi - lo) / 2;
10        if (arr[mid] == X) return mid;
11
12        if (arr[lo] <= arr[mid]) { // left half sorted
13            if (arr[lo] <= X && X < arr[mid]) {
14                hi = mid - 1;

```

```

15         } else {
16             lo = mid + 1;
17         }
18     } else { // right half sorted
19         if (arr[mid] < X && X <= arr[hi]) {
20             lo = mid + 1;
21         } else {
22             hi = mid - 1;
23         }
24     }
25 }
26 return -1;
27 }
28
29 int main() {
30     vector<int> arr = {4, 5, 6, 7, 0, 1, 2};
31     cout << "Index of 0: " << searchRotated(arr, 0) << endl;
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. Each iteration discards at least one full half of the current window, exactly as in ordinary binary search, giving

$$O(\log n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

When an array's global sortedness is broken in one specific, structured way (here, a single rotation point), look for a local property that still holds at every step of binary search – here, “at least one of the two halves around any midpoint is genuinely sorted.” Identifying and exploiting that local invariant is what lets binary search survive structural violations of global sortedness.

2.10 Search in a Rotated Sorted Array II

Problem Statement. Same setting as Section 2.9 – a sorted array rotated at an unknown pivot – except now the array **may contain duplicate values**. Given the rotated array *arr* and a target *X*, determine whether *X* is present (**true/false**).

Pattern Recognition

The keyword change from Section 2.9 is just one word: “*may contain duplicates*.” This single word breaks the comparison $arr[lo] \leq arr[mid]$ that we relied on to decide which half is sorted – with duplicates, this comparison can be **true** even when *lo*, *mid*, and the rotation pivot are tangled together in a way that makes *neither* half safely identifiable as sorted. Whenever a rotated-array problem mentions duplicates, expect to need an escape hatch for this ambiguous case, and expect the worst-case time complexity to degrade as a result.

Understanding the problem carefully

Consider $arr = [3, 3, 3, 1, 2, 3]$ with $lo = 0$ pointing to value 3, and suppose mid also happens to land on a 3 (say $mid = 2$, $arr[2] = 3$). Then $arr[lo] = arr[mid] = 3$, so the inequality $arr[lo] \leq arr[mid]$ holds, which the Section 2.9 logic would interpret as “the left half is sorted.” But the rotation pivot (where 1 follows 3) actually sits *inside* this very range, so the left half is not genuinely a clean sorted run with respect to deciding where X could be – the equal values at the boundary hide the pivot’s location. With distinct elements this ambiguity cannot arise (any two unequal boundary values immediately tell us which side the pivot is on), but duplicates can make $arr[lo] = arr[mid]$ true regardless of where the pivot actually is.

Greedy Decision, with an Escape Hatch

Use the exact same logic as Section 2.9, *except*: whenever $arr[lo] = arr[mid]$ and $arr[mid] = arr[hi]$ (or, more simply, whenever the usual “which half is sorted” check is ambiguous because of equal boundary values), we cannot safely discard either half based on comparisons alone. In that situation, shrink the window by one element from each side ($lo \leftarrow lo + 1$, $hi \leftarrow hi - 1$) and try again – this never discards a position that could uniquely hold X in a way that loses correctness, but it does cost us the usual halving speed in the worst case.

Why shrinking by one is the right (and only) safe fallback

When $arr[lo] = arr[mid]$, we genuinely cannot tell, from this comparison alone, which half contains the rotation pivot – both possibilities are consistent with the observed equality. Since we cannot identify a provably safe half to discard, we are not entitled to discard a full half at all without risking skipping over X . The one adjustment we *can* make safely is to shrink the boundary by a single element: if $arr[lo]$ duplicates the value at mid , removing just lo from consideration cannot lose X , because if $X = arr[lo]$, the duplicate value persists at mid and remains discoverable; if $X \neq arr[lo]$, removing lo loses nothing relevant. This is why the worst case degrades to $O(n)$: an adversarial array of nearly all-equal elements (e.g. $[2, 2, 2, 2, 2, 2, 1, 2]$) can force this one-at-a-time shrinking on almost every step.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo) / 2$.
 - (b) If $arr[mid] = X$: return **true**.
 - (c) If $arr[lo] = arr[mid]$ and $arr[mid] = arr[hi]$: $lo \leftarrow lo + 1$, $hi \leftarrow hi - 1$ (ambiguous; shrink by one from each side).
 - (d) Else if $arr[lo] \leq arr[mid]$ (left half safely sorted): apply the same range check as Section 2.9.
 - (e) Else (right half safely sorted): apply the same range check as Section 2.9.
3. If the loop ends without returning, return **false**.

Algorithm 23 Search in Rotated Sorted Array (With Duplicates)

```

1: procedure SEARCHROTATEDDUP(arr, X)
2:   lo  $\leftarrow$  0, hi  $\leftarrow$  n - 1
3:   while lo  $\leq$  hi do
4:     mid  $\leftarrow$  lo + (hi - lo) / 2
5:     if arr[mid] = X then return true
6:     end if
7:     if arr[lo] = arr[mid] and arr[mid] = arr[hi] then
8:       lo  $\leftarrow$  lo + 1, hi  $\leftarrow$  hi - 1
9:     else if arr[lo]  $\leq$  arr[mid] then
10:      if arr[lo]  $\leq$  X and X < arr[mid] then
11:        hi  $\leftarrow$  mid - 1
12:      else
13:        lo  $\leftarrow$  mid + 1
14:      end if
15:    else
16:      if arr[mid] < X and X  $\leq$  arr[hi] then
17:        lo  $\leftarrow$  mid + 1
18:      else
19:        hi  $\leftarrow$  mid - 1
20:      end if
21:    end if
22:  end while
23:  return false
24: end procedure

```

Dry run

Dry Run. Take *arr* = [3, 1, 2, 3, 3, 3, 3], *X* = 1.

- *lo* = 0, *hi* = 6: *mid* = 3, *arr*[3] = 3 $\neq$ 1. Is *arr*[0] = 3 = *arr*[3] = 3 and *arr*[3] = 3 = *arr*[6] = 3? Yes – ambiguous. Shrink: *lo* = 1, *hi* = 5.
- *lo* = 1, *hi* = 5: *mid* = 3, *arr*[3] = 3 $\neq$ 1. Is *arr*[1] = 1 = *arr*[3] = 3? No. Is *arr*[1] = 1 $\leq$ *arr*[3] = 3? Yes, left half sorted. Is $1 \leq 1 < 3$? Yes. Search left: *hi* = 2.
- *lo* = 1, *hi* = 2: *mid* = 1, *arr*[1] = 1 = 1. Return **true**.

The answer is **true**.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool searchRotatedDup(vector<int>& arr, int X) {
5     int n = arr.size();
6     int lo = 0, hi = n - 1;
7
8     while (lo <= hi) {
9         int mid = lo + (hi - lo) / 2;
10        if (arr[mid] == X) return true;
11

```

```

12     if (arr[lo] == arr[mid] && arr[mid] == arr[hi]) {
13         lo++;
14         hi--;
15     } else if (arr[lo] <= arr[mid]) {
16         if (arr[lo] <= X && X < arr[mid]) {
17             hi = mid - 1;
18         } else {
19             lo = mid + 1;
20         }
21     } else {
22         if (arr[mid] < X && X <= arr[hi]) {
23             lo = mid + 1;
24         } else {
25             hi = mid - 1;
26         }
27     }
28 }
29 return false;
30 }
31
32 int main() {
33     vector<int> arr = {3, 1, 2, 3, 3, 3, 3};
34     cout << (searchRotatedDup(arr, 1) ? "true" : "false") << endl;
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. In the best/average case, the same halving as Section 2.9 applies, giving $O(\log n)$. But in the worst case – an array of almost entirely duplicate values, e.g. $[2, 2, 2, 2, 2, 2, 1, 2]$ – the ambiguous case forces shrinking by one element at a time, giving a worst-case time complexity of

$$O(n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Duplicates are binary search’s natural enemy: they tend to destroy the clean monotonicity or structural invariants that comparisons rely on, forcing a fallback to linear-time shrinking in the worst case. Whenever a sheet problem adds “with duplicates” to a previously clean binary search problem, expect to need an explicit ambiguous-case branch, and expect to state a degraded worst-case complexity honestly rather than claiming $O(\log n)$ that the duplicates actually prevent.

2.11 Find Minimum in a Rotated Sorted Array

Problem Statement. We are given a sorted array (with all distinct elements) that has been rotated at some unknown pivot. We must find the **minimum element** of the array in $O(\log n)$ time.

Pattern Recognition

“Minimum element of a rotated sorted array” is really asking us to **find the rotation pivot itself** – the minimum element is exactly the element sitting at the pivot (the one

place where the rotated array's increasing run restarts). Once you see “rotated sorted array” combined with “find the minimum/find the rotation point,” reach for the same “one half is always sorted” invariant used in Sections 2.9–2.10, but this time the goal is to home in on the pivot's location directly, rather than to locate a separately given target value.

Understanding the problem carefully

In a rotated sorted array, exactly one of the following is true at any candidate window $[lo, hi]$: either the window itself is already fully sorted (no rotation point inside it), in which case its minimum is simply $arr[lo]$; or the rotation point lies strictly inside the window, in which case we can use the same left-half/right-half sortedness check from Section 2.9 to decide which half the rotation point falls into, and discard the half that is provably free of it.

Greedy Choice

If $arr[lo] \leq arr[hi]$, the current window is already fully sorted (no rotation inside it), so its minimum is $arr[lo]$ – record it and stop. Otherwise, compare $arr[mid]$ to $arr[lo]$: if $arr[mid] \geq arr[lo]$, the left half $[lo, mid]$ is sorted (so its minimum is $arr[lo]$, not better than what is achievable on the right, since the right half must contain the actual rotation point and hence a smaller value); discard the left half by moving $lo \leftarrow mid + 1$. Otherwise, the rotation point lies in $[lo, mid]$ itself, so we keep mid as a candidate minimum and search left: $hi \leftarrow mid$ (not $mid - 1$, since mid itself might be the true minimum).

Why discarding the sorted half is always correct here

If $arr[mid] \geq arr[lo]$, the segment $[lo, mid]$ has no internal rotation (it is increasing throughout), so its smallest value is $arr[lo]$, which is *not smaller* than the global minimum unless the global minimum happens to equal $arr[lo]$ itself – but that can only happen if the whole array (or at least everything from lo onward that matters) is sorted with no rotation remaining, a case that is only reachable by continuing to shrink toward the actual pivot, which must then lie at or to the right of mid . So it is always safe to move lo past this sorted segment; the true minimum, if not already found, must lie strictly within $(mid, hi]$. Conversely, if $arr[mid] < arr[lo]$, the array has “wrapped around” somewhere in $[lo, mid]$, so the rotation pivot – and hence the true minimum – is guaranteed to lie within that left segment, including possibly at mid itself, which is why we narrow to $hi \leftarrow mid$ rather than excluding it.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$.
2. While $lo < hi$:
 - (a) If $arr[lo] \leq arr[hi]$: the window is sorted; break (the minimum is $arr[lo]$).
 - (b) $mid \leftarrow lo + (hi - lo)/2$.
 - (c) If $arr[mid] \geq arr[lo]$: $lo \leftarrow mid + 1$.
 - (d) Else: $hi \leftarrow mid$.
3. Return $arr[lo]$.

Algorithm 24 Find Minimum in Rotated Sorted Array

```

1: procedure FINDMIN(arr)
2:   lo  $\leftarrow$  0, hi  $\leftarrow$  n - 1
3:   while lo < hi do
4:     if arr[lo]  $\leq$  arr[hi] then
5:       break
6:     end if
7:     mid  $\leftarrow$  lo + (hi - lo) / 2
8:     if arr[mid]  $\geq$  arr[lo] then
9:       lo  $\leftarrow$  mid + 1
10:    else
11:      hi  $\leftarrow$  mid
12:    end if
13:  end while
14:  return arr[lo]
15: end procedure

```

Dry run

Dry Run. Take *arr* = [4, 5, 6, 7, 0, 1, 2].

- *lo* = 0, *hi* = 6: *arr*[0] = 4 $\leq$ *arr*[6] = 2? No. *mid* = 3, *arr*[3] = 7 $\geq$ *arr*[0] = 4? Yes. *lo* = 4.
- *lo* = 4, *hi* = 6: *arr*[4] = 0 $\leq$ *arr*[6] = 2? Yes – window is sorted. Break.

The minimum is *arr*[4] = 0.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int findMin(vector<int>& arr) {
5      int n = arr.size();
6      int lo = 0, hi = n - 1;
7
8      while (lo < hi) {
9          if (arr[lo] <= arr[hi]) break; // already sorted
10
11         int mid = lo + (hi - lo) / 2;
12         if (arr[mid] >= arr[lo]) {
13             lo = mid + 1;
14         } else {
15             hi = mid;
16         }
17     }
18     return arr[lo];
19 }
20
21 int main() {
22     vector<int> arr = {4, 5, 6, 7, 0, 1, 2};
23     cout << "Minimum: " << findMin(arr) << endl;
24     return 0;
25 }

```

Complexity Analysis

Time Complexity. Each iteration halves the search window (or terminates early once the window is detected to be sorted), giving

$$O(\log n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Finding the rotation pivot and searching for a target in a rotated array (Section 2.9) rely on the exact same underlying invariant – one half around any midpoint is always genuinely sorted. The pivot itself is simply the boundary that this repeated halving converges to when there is no separate target value to compare against, only the structure of the rotation itself.

2.12 Find How Many Times an Array Has Been Rotated

Problem Statement. Given a sorted array that has been rotated some number of times (all elements distinct), determine **how many times** it has been rotated – equivalently, find the number of left rotations applied to the original sorted array.

Pattern Recognition

This problem is solved by noticing that it is asking for almost the same quantity as Section 2.11, phrased differently: the number of left rotations applied to a sorted array equals exactly the **index** of the minimum element. If the original sorted array is rotated left by k positions, the element that used to be at index 0 (the global minimum) ends up sitting at index k . Whenever a problem about a rotated array asks for “how many rotations,” translate it immediately into “find the index of the minimum.”

Understanding the problem carefully

Picture the original sorted array $[v_0 < v_1 < \dots < v_{n-1}]$. Rotating it left by k positions produces $[v_k, v_{k+1}, \dots, v_{n-1}, v_0, v_1, \dots, v_{k-1}]$. The smallest value v_0 now sits at position k in the rotated array. So finding the number of rotations is exactly finding the position of the minimum element – the same quantity computed in Section 2.11, except we now want the *index* rather than the *value*.

Reuse, Don't Re-derive

Apply the exact same binary search as Section 2.11 to locate the minimum, but track and return lo (the index) instead of $arr[lo]$ (the value).

Algorithm

Identical to Algorithm 2.11's FINDMIN procedure, returning lo instead of $arr[lo]$.

Dry run

Dry Run. Take $arr = [4, 5, 6, 7, 0, 1, 2]$ (same array as Section 2.11).

The binary search trace is identical to before, terminating with $lo = 4$. So the array has been rotated 4 times – indeed, the original sorted array would have been $[0, 1, 2, 4, 5, 6, 7]$, and rotating it left by 4 positions gives exactly $[4, 5, 6, 7, 0, 1, 2]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int countRotations(vector<int>& arr) {
5     int n = arr.size();
6     int lo = 0, hi = n - 1;
7
8     while (lo < hi) {
9         if (arr[lo] <= arr[hi]) break; // already sorted
10
11         int mid = lo + (hi - lo) / 2;
12         if (arr[mid] >= arr[lo]) {
13             lo = mid + 1;
14         } else {
15             hi = mid;
16         }
17     }
18     return lo; // index of the minimum = number of rotations
19 }
20
21 int main() {
22     vector<int> arr = {4, 5, 6, 7, 0, 1, 2};
23     cout << "Number of rotations: " << countRotations(arr) << endl;
24     return 0;
25 }

```

Complexity Analysis

Time Complexity. $O(\log n)$, identical to Section 2.11. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

When a new problem statement turns out to want the *index* of a quantity you already know how to binary search for (rather than its *value*), there is usually nothing new to derive – just track the index alongside, or instead of, the value throughout the same search.

2.13 Single Element in a Sorted Array

Problem Statement. We are given a sorted array in which **every element appears exactly twice**, except for one element that appears **exactly once**. Find that single element, in $O(\log n)$ time.

Pattern Recognition

At first glance this looks like it has nothing to do with sortedness – “find the element that appears once” sounds like a job for hashing or XOR, both $O(n)$. The keyword that should stop you is “*sorted array*” appearing in the same breath as a uniqueness condition: whenever a problem about counts, pairs, or uniqueness also guarantees sortedness, suspect that sortedness is providing exploitable *positional* structure – here, specifically, that pairs of equal elements occupy predictable (even, odd) index pairs *before* the single element, and that this pairing pattern shifts by one position immediately *after* it.

Understanding the problem carefully

Suppose the single (unpaired) element sits at index s . For every pair of equal elements before s , since each pair occupies two consecutive array slots and there is an even number of elements total before s (every element before s is properly paired), the first element of each such pair lands on an **even** index, and its duplicate lands on the very next (**odd**) index. That is: for $i < s$, pairs look like $(arr[2k], arr[2k + 1])$ with $arr[2k] = arr[2k + 1]$. After s , the single element has shifted the parity of every subsequent pair by one position, so pairs there look like $(arr[2k + 1], arr[2k + 2])$ instead.

Candidate Space and Predicate

The candidate space is indices $0, \dots, n - 1$ (restricted to even indices, by construction). The predicate is $P(\text{even index } i) = (arr[i] = arr[i + 1]) - \text{true}$ for every even index strictly before the single element, and **false** from the single element's (even-aligned) position onward. We binary search for the first even index where P becomes **false** – that index holds the single element.

Why forcing *mid* to be even is necessary and correct

The predicate P above is only meaningful when evaluated at *even* indices, because the pairing pattern is defined relative to even/odd positions, not relative to *mid*'s raw value. So at each step, if the computed midpoint *mid* happens to be odd, we adjust it down by one (to the preceding even index) before testing P . After this adjustment, the same monotonicity argument as ordinary binary search applies directly: P is true for an initial contiguous run of even indices and false afterward, with the crossover landing exactly on the single element's position.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 2$ (we only need to compare each candidate to its right neighbor).
2. While $lo < hi$:
 - (a) $mid \leftarrow lo + (hi - lo) / 2$.
 - (b) If *mid* is odd: $mid \leftarrow mid - 1$ (force to even).
 - (c) If $arr[mid] = arr[mid + 1]$: the single element is to the right; $lo \leftarrow mid + 2$.
 - (d) Else: the single element is at *mid* or to the left; $hi \leftarrow mid$.
3. Return $arr[lo]$.

Algorithm 25 Single Element in a Sorted Array

```

1: procedure SINGLEELEMENT(arr)
2:    $lo \leftarrow 0, hi \leftarrow n - 2$ 
3:   while  $lo < hi$  do
4:      $mid \leftarrow lo + (hi - lo) / 2$ 
5:     if  $mid$  is odd then
6:        $mid \leftarrow mid - 1$ 
7:     end if
8:     if  $arr[mid] = arr[mid + 1]$  then
9:        $lo \leftarrow mid + 2$ 
10:    else
11:       $hi \leftarrow mid$ 
12:    end if
13:  end while
14:  return  $arr[lo]$ 
15: end procedure

```

Dry run

Dry Run. Take $arr = [1, 1, 2, 3, 3, 4, 4, 8, 8]$ ($n = 9$).

- $lo = 0, hi = 7$: $mid = 3$ (odd) $\rightarrow 2$. $arr[2] = 2, arr[3] = 3$. Not equal. $hi = 2$.
- $lo = 0, hi = 2$: $mid = 1$ (odd) $\rightarrow 0$. $arr[0] = 1, arr[1] = 1$. Equal. $lo = 2$.
- $lo = 2, hi = 2$: loop ends ($lo = hi$).

The single element is $arr[2] = 2$, which is correct.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int singleElement(vector<int>& arr) {
5      int n = arr.size();
6      int lo = 0, hi = n - 2;
7
8      while (lo < hi) {
9          int mid = lo + (hi - lo) / 2;
10         if (mid % 2 == 1) mid--;
11
12         if (arr[mid] == arr[mid + 1]) {
13             lo = mid + 2;
14         } else {
15             hi = mid;
16         }
17     }
18     return arr[lo];
19 }
20
21 int main() {
22     vector<int> arr = {1, 1, 2, 3, 3, 4, 4, 8, 8};
23     cout << "Single element: " << singleElement(arr) << endl;
24     return 0;
25 }

```

Complexity Analysis

Time Complexity. The search window halves each iteration (the parity adjustment only ever shifts mid by one, not affecting the overall halving rate), giving

$$O(\log n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Sortedness can encode positional structure beyond simple ordering – here, sortedness plus a pairing guarantee creates an exploitable *parity* pattern. Whenever a sorted-array problem also involves counts, pairs, or duplicates, look for whether index parity (even/odd position) is doing some of the work that the array’s values alone do in simpler binary search problems.

2.14 Find Peak Element

Problem Statement. We are given an array arr of n distinct integers (no two adjacent elements are equal), where the array is conceptually bordered by $-\infty$ on both ends (i.e. $arr[-1] = arr[n] = -\infty$). An index i is a **peak** if $arr[i]$ is strictly greater than both of its neighbors. We must find the index of **any one** peak element, in $O(\log n)$ time.

Pattern Recognition

This problem has **no sortedness at all** – the array can be completely jumbled. This is the most important lesson in this section: the keyword to look for is not “sorted,” but the guarantee of a **boundary condition combined with a local comparison rule** that forces a peak to exist and forces a specific monotonic structure on the direction of “ascent.” Whenever you see “array bordered by $-\infty$ ” or an equivalent guarantee (e.g. “the array is strictly increasing then strictly decreasing”), suspect a peak-finding binary search even though no sortedness is mentioned.

Understanding the problem carefully

Consider the midpoint mid and compare $arr[mid]$ to its right neighbor $arr[mid+1]$ (treating an out-of-range neighbor as $-\infty$ per the problem’s border convention).

- If $arr[mid] < arr[mid+1]$: the sequence is “climbing” at mid . Because the array is bordered by $-\infty$ on the right end too, *somewhere* to the right of mid the climbing must eventually stop and turn into a peak (it cannot climb forever, since it must come back down to $-\infty$ at the far border) – so a peak is guaranteed to exist in $(mid, hi]$.
- If $arr[mid] > arr[mid+1]$: the sequence is “descending” at mid , meaning mid is locally higher than its right neighbor. By a symmetric argument applied leftward (the array is bordered by $-\infty$ on the left too), a peak is guaranteed to exist somewhere in $[lo, mid]$, *including possibly mid itself*.

Greedy Choice

If $arr[mid] < arr[mid+1]$, discard the left half including mid , since a peak is guaranteed further right: $lo \leftarrow mid+1$. Otherwise, discard the right half excluding mid , since a peak is guaranteed at mid or further left: $hi \leftarrow mid$.

Why a peak is always guaranteed to exist in the kept half

This is an existence argument, not a uniqueness or value-comparison argument, which makes it a structurally different kind of binary search correctness proof from the rest of this chapter. The guarantee rests entirely on the $-\infty$ borders: if we always move *toward* increasing values, we are guaranteed to eventually hit a local maximum before falling off either border, because the function value cannot increase forever in a finite array, and any point where increase changes to decrease (or where increase meets the $-\infty$ border with the array ascending all the way to its last real element) is, by definition, a peak. This is the same idea used to prove that a continuous function climbing from $-\infty$ must have a maximum – a discrete, array-based analogue.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$.
2. While $lo < hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) If $arr[mid] < arr[mid + 1]$: $lo \leftarrow mid + 1$.
 - (c) Else: $hi \leftarrow mid$.
3. Return lo (equivalently hi ; they have converged).

Algorithm 26 Find Peak Element

```

1: procedure FINDPEAK( $arr$ )
2:    $lo \leftarrow 0$ ,  $hi \leftarrow n - 1$ 
3:   while  $lo < hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:     if  $arr[mid] < arr[mid + 1]$  then
6:        $lo \leftarrow mid + 1$ 
7:     else
8:        $hi \leftarrow mid$ 
9:     end if
10:  end while
11:  return  $lo$ 
12: end procedure

```

Dry run

Dry Run. Take $arr = [1, 2, 1, 3, 5, 6, 4]$.

- $lo = 0, hi = 6$: $mid = 3$, $arr[3] = 3 < arr[4] = 5$. $lo = 4$.
- $lo = 4, hi = 6$: $mid = 5$, $arr[5] = 6 < arr[6] = 4$? No. $hi = 5$.
- $lo = 4, hi = 5$: $mid = 4$, $arr[4] = 5 < arr[5] = 6$. $lo = 5$.
- $lo = 5 = hi$: loop ends.

The peak index is 5, where $arr[5] = 6$, which is indeed greater than both neighbors $arr[4] = 5$ and $arr[6] = 4$. (Index 1, where $arr[1] = 2$, is also a valid peak; the problem only requires *any* one peak, and this algorithm happens to converge to the one on the side it explored.)

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int findPeak(vector<int>& arr) {
5     int n = arr.size();
6     int lo = 0, hi = n - 1;
7
8     while (lo < hi) {
9         int mid = lo + (hi - lo) / 2;
10        if (arr[mid] < arr[mid + 1]) {
11            lo = mid + 1;
12        } else {
13            hi = mid;
14        }
15    }
16    return lo;
17 }
18
19 int main() {
20     vector<int> arr = {1, 2, 1, 3, 5, 6, 4};
21     cout << "Peak index: " << findPeak(arr) << endl;
22     return 0;
23 }

```

Complexity Analysis

Time Complexity. The search window halves each iteration, giving

$$O(\log n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Binary search does not require sortedness – it requires a monotonic, provable rule for discarding half the search space. Here, the rule comes from a local ascent/descent comparison plus a boundary guarantee, not from any global ordering of the array’s values. Whenever you see “find a local maximum/minimum,” check whether boundary conditions and a single local comparison can replace the sortedness argument used elsewhere in this chapter.

2.15 Find Square Root of a Number using Binary Search

We now begin the second major group of problems in this chapter: **binary search on the answer**. In every problem so far, the candidate space was a set of array indices. From here on, there is often no array at all – the candidate space is a range of numbers that could be the *answer itself*, and we binary search directly over that range, checking feasibility of each candidate with a helper function.

Problem Statement. Given a non-negative integer n , find $\lfloor \sqrt{n} \rfloor$, the floor of its square root, without using a built-in square root function – and ideally without testing every integer from 1 to n one at a time.

Pattern Recognition

“Find the floor of the square root” doesn’t mention an array at all, which is exactly the point: the phrase to watch for is “find the largest/smallest value such that some computable condition holds.” Here, the condition is $candidate^2 \leq n$. This condition is monotonic in *candidate* – true for small candidates, false for large ones, with exactly one crossover – which is all binary search ever needed in the first place. The array, in earlier sections, was just one particular way of presenting a monotonic candidate space; an integer range works just as well.

Understanding the problem carefully

Define $P(candidate) = (candidate^2 \leq n)$ for candidates ranging over $0, 1, 2, \dots, n$. As *candidate* increases, $candidate^2$ increases too (squaring is monotonic for non-negative numbers), so P is **true** for every candidate up to $\lfloor \sqrt{n} \rfloor$ and **false** for every candidate beyond it. The floor of the square root is exactly the *largest* candidate for which P holds.

Candidate Space and Predicate

The candidate space is the integer range $[0, n]$ (or, more tightly, $[1, n]$ for $n \geq 1$). The predicate is $P(candidate) = (candidate^2 \leq n)$, and we want the largest candidate where P holds – the same “find the rightmost true position” template used for floor in Section 2.6, just applied to a numeric range instead of array values.

Why binary searching the answer directly is valid

There is nothing conceptually new here beyond recognizing that the candidate space need not be array indices. The monotonicity of P is guaranteed by basic algebra ($x \leq y \Rightarrow x^2 \leq y^2$ for non-negative x, y), which plays exactly the role that array sortedness played in earlier sections: it guarantees a single, well-defined crossover point that binary search can locate in logarithmically many evaluations of P , each evaluation costing only $O(1)$ (a single multiplication).

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n$, $ans \leftarrow 0$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) If $mid \times mid \leq n$: record $ans \leftarrow mid$; search right: $lo \leftarrow mid + 1$.
 - (c) Else: search left: $hi \leftarrow mid - 1$.
3. Return ans .

Algorithm 27 Floor of Square Root

```

1: procedure FLOORSQRT( $n$ )
2:    $lo \leftarrow 0, hi \leftarrow n, ans \leftarrow 0$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:     if  $mid \times mid \leq n$  then
6:        $ans \leftarrow mid$ 
7:        $lo \leftarrow mid + 1$ 
8:     else
9:        $hi \leftarrow mid - 1$ 
10:    end if
11:  end while
12:  return  $ans$ 
13: end procedure

```

Dry run

Dry Run. Take $n = 28$.

- $lo = 0, hi = 28$: $mid = 14, 14^2 = 196 \leq 28$? No. $hi = 13$.
- $lo = 0, hi = 13$: $mid = 6, 6^2 = 36 \leq 28$? No. $hi = 5$.
- $lo = 0, hi = 5$: $mid = 2, 2^2 = 4 \leq 28$? Yes. $ans = 2, lo = 3$.
- $lo = 3, hi = 5$: $mid = 4, 4^2 = 16 \leq 28$? Yes. $ans = 4, lo = 5$.
- $lo = 5, hi = 5$: $mid = 5, 5^2 = 25 \leq 28$? Yes. $ans = 5, lo = 6$.
- $lo = 6 > hi = 5$: loop ends.

The answer is 5, and indeed $5^2 = 25 \leq 28 < 36 = 6^2$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int floorSqrt(int n) {
5      int lo = 0, hi = n, ans = 0;
6
7      while (lo <= hi) {
8          long long mid = lo + (hi - lo) / 2;
9          if (mid * mid <= n) {
10             ans = mid;
11             lo = mid + 1;
12         } else {
13             hi = mid - 1;
14         }
15     }
16     return ans;
17 }
18
19 int main() {
20     cout << "Floor sqrt of 28: " << floorSqrt(28) << endl;
21     return 0;
22 }

```

Complexity Analysis

Time Complexity. The candidate range $[0, n]$ halves each iteration, giving

$$O(\log n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Binary search on the answer requires exactly two ingredients: a numeric candidate range, and a feasibility predicate that is monotonic across that range. Once both exist, the same `while(lo <= hi)` skeleton from array-based binary search applies verbatim – only the meaning of *lo*, *hi*, and the predicate check change.

2.16 Find the Nth Root of a Number using Binary Search

Problem Statement. Given two positive integers n and m , find x such that $x^n = m$, where x is a positive integer. If no such integer x exists, report -1 .

Pattern Recognition

This is the direct generalization of Section 2.15 from “square root” to “ n -th root.” The keyword pattern is identical: “*find the value x such that x raised to some power equals (or is bounded by) a target.*” The only change is the predicate’s exponent, from 2 to a general n – everything else about the binary-search-on-the-answer template carries over unchanged.

Understanding the problem carefully

Define $P(\text{candidate}) = (\text{candidate}^n \leq m)$ for candidates in $[1, m]$. Just as squaring is monotonic for non-negative integers, raising to any fixed positive power n is monotonic too: $x \leq y \Rightarrow x^n \leq y^n$. So P is **true** for a prefix of candidates and **false** afterward, with a single crossover – the largest candidate satisfying P is a perfect n -th root only if candidate^n exactly equals m at that point; otherwise, no exact integer n -th root exists.

Candidate Space and Predicate

The candidate space is integers $[1, m]$. The predicate is $P(\text{candidate}) = (\text{candidate}^n \leq m)$. Track the largest candidate for which P holds; at the end, check whether that candidate, raised to the n -th power, exactly equals m .

Why we must verify exactness at the end

Unlike the square-root problem, which explicitly asked for the *floor* of the root (so any final value of *ans* is a valid answer), this problem demands an *exact* integer root, or -1 if none exists. So the binary search itself is unchanged – it always finds the largest candidate whose n -th power does not exceed m – but we add one final check: if that candidate’s n -th power is not exactly m , there is no integer solution, and we must report -1 rather than the nearest approximation.

Algorithm

1. Initialize $lo \leftarrow 1$, $hi \leftarrow m$.
2. While $lo \leq hi$:

- (a) $mid \leftarrow lo + (hi - lo)/2$.
- (b) Compute $p \leftarrow mid^n$ (stop early/clamp if it exceeds m , to avoid overflow for large n).
- (c) If $p = m$: return mid .
- (d) If $p < m$: $lo \leftarrow mid + 1$.
- (e) Else: $hi \leftarrow mid - 1$.

3. If the loop ends without returning, return -1 .

Algorithm 28 Nth Root

```

1: procedure NTHROOT( $n, m$ )
2:    $lo \leftarrow 1, hi \leftarrow m$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:      $p \leftarrow mid^n$  ▷ computed with early-exit on overflow
6:     if  $p = m$  then return  $mid$ 
7:     end if
8:     if  $p < m$  then
9:        $lo \leftarrow mid + 1$ 
10:    else
11:       $hi \leftarrow mid - 1$ 
12:    end if
13:  end while
14:  return  $-1$ 
15: end procedure

```

Dry run

Dry Run. Take $n = 3, m = 27$ (find the cube root).

- $lo = 1, hi = 27$: $mid = 14, 14^3 = 2744 \leq 27$? No, $p > m$. $hi = 13$.
- $lo = 1, hi = 13$: $mid = 7, 7^3 = 343 > 27$. $hi = 6$.
- $lo = 1, hi = 6$: $mid = 3, 3^3 = 27 = 27$. Return 3.

The answer is 3, since $3^3 = 27$.

Now take $n = 4, m = 20$. No integer fourth root of 20 exists ($2^4 = 16, 3^4 = 81$), so the binary search will narrow down without ever finding $p = m$, and the final answer will correctly be -1 .

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Computes base^exp, capped to indicate ">m" without overflow.
5 long long power(long long base, int exp, long long cap) {
6     long long result = 1;
7     for (int i = 0; i < exp; i++) {
8         result *= base;
9         if (result > cap) return cap + 1; // overflow guard
10    }

```

```

11     return result;
12 }
13
14 int nthRoot(int n, int m) {
15     int lo = 1, hi = m;
16
17     while (lo <= hi) {
18         int mid = lo + (hi - lo) / 2;
19         long long p = power(mid, n, m);
20
21         if (p == m) return mid;
22         else if (p < m) lo = mid + 1;
23         else hi = mid - 1;
24     }
25     return -1;
26 }
27
28 int main() {
29     cout << "Cube root of 27: " << nthRoot(3, 27) << endl;
30     cout << "4th root of 20: " << nthRoot(4, 20) << endl;
31     return 0;
32 }

```

Complexity Analysis

Time Complexity. The candidate range $[1, m]$ halves each iteration, and each predicate evaluation costs $O(n)$ to compute the power (or $O(\log n)$ with fast exponentiation), giving an overall time complexity of

$$O(n \log m)$$

(or $O(\log n \log m)$ with fast exponentiation). **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

Generalizing a binary-search-on-the-answer problem from a fixed exponent (square root, exponent 2) to a variable exponent (n -th root) changes nothing about the search itself – only the cost of evaluating the predicate changes, since computing $candidate^n$ is more expensive than $candidate^2$. Always separate “how do I search the candidate space” from “how expensive is one feasibility check,” since they contribute independently to the total complexity.

2.17 Koko Eating Bananas

Problem Statement. There are n piles of bananas, where pile i has $piles[i]$ bananas. Koko can eat at most k bananas from a single pile in one hour (if a pile has fewer than k bananas, she finishes that pile in that hour and cannot start another pile in the same hour). Koko wants to eat all the bananas within h hours. Find the **minimum** integer eating speed k that allows her to finish within h hours.

Pattern Recognition

“Find the minimum speed/rate/capacity such that some task completes within a time/resource limit” is the archetypal *binary search on the answer* problem. The two keywords to look for together are “**minimize the (speed/capacity/rate)**” and a **feasibility constraint** (here, a deadline of h hours) that becomes easier to satisfy as the candidate answer

increases. This combination – minimize x subject to “feasible(x)” becoming **true** once x is large enough – is the single most common shape of binary-search-on-the-answer problems on this sheet, and Koko Eating Bananas is its canonical introduction.

Understanding the problem carefully

For a fixed candidate speed k , the number of hours Koko needs to finish pile i is $\lceil \text{piles}[i]/k \rceil$ (she needs a whole extra hour for any remainder), so the total hours needed across all piles is

$$\text{hoursNeeded}(k) = \sum_{i=1}^n \left\lceil \frac{\text{piles}[i]}{k} \right\rceil.$$

As k increases, each term $\lceil \text{piles}[i]/k \rceil$ can only decrease or stay the same (eating faster never requires more hours for the same pile), so $\text{hoursNeeded}(k)$ is a non-increasing function of k . Define the feasibility predicate $P(k) = (\text{hoursNeeded}(k) \leq h)$. Since hoursNeeded is non-increasing in k , P is **false** for small k and **true** from some crossover point onward – exactly the monotonic structure binary search needs, with the answer being the smallest k where P becomes **true**.

Candidate Space and Predicate

The candidate space is integer eating speeds $k \in [1, \max(\text{piles})]$ (eating faster than the largest pile is never useful, since any pile is finished in exactly one hour once k reaches its size). The predicate is $P(k) = (\text{hoursNeeded}(k) \leq h)$, and we want the smallest k where P holds.

Why this reduces feasibility-checking to a simple scan

The crucial design decision is to separate two concerns: *deciding which k to try next* (binary search’s job) and *checking whether a specific k works* (a simple $O(n)$ linear scan over the piles, summing up $\lceil \text{piles}[i]/k \rceil$ for the fixed candidate k). This separation is the heart of every binary-search-on-the-answer problem: we never need to reason about the feasibility check’s internal monotonicity in i – only about the feasibility *function as a whole*’s monotonicity in the single candidate parameter k , which we established above using a much simpler, purely algebraic argument (more bananas per hour can never increase the hours needed).

Algorithm

1. Initialize $lo \leftarrow 1$, $hi \leftarrow \max(\text{piles})$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) Compute $\text{hoursNeeded}(mid)$ by summing $\lceil \text{piles}[i]/mid \rceil$ over all piles.
 - (c) If $\text{hoursNeeded}(mid) \leq h$: this speed works; record it and try slower: $hi \leftarrow mid - 1$.
 - (d) Else: too slow; speed up: $lo \leftarrow mid + 1$.
3. Return the recorded answer (equivalently, lo once the loop ends).

Algorithm 29 Koko Eating Bananas

```

1: procedure MINEATINGSPEED(piles, h)
2:   lo  $\leftarrow$  1, hi  $\leftarrow$  max(piles)
3:   while lo  $\leq$  hi do
4:     mid  $\leftarrow$  lo + (hi - lo) / 2
5:     hours  $\leftarrow$   $\sum_i \lceil \text{piles}[i] / \text{mid} \rceil$ 
6:     if hours  $\leq$  h then
7:       hi  $\leftarrow$  mid - 1
8:     else
9:       lo  $\leftarrow$  mid + 1
10:    end if
11:  end while
12:  return lo
13: end procedure

```

Dry run

Dry Run. Take *piles* = [3, 6, 7, 11], *h* = 8.
 $\text{max}(\text{piles}) = 11$.

- *lo* = 1, *hi* = 11: *mid* = 6. Hours: $\lceil 3/6 \rceil + \lceil 6/6 \rceil + \lceil 7/6 \rceil + \lceil 11/6 \rceil = 1 + 1 + 2 + 2 = 6 \leq 8$. Feasible. *hi* = 5.
- *lo* = 1, *hi* = 5: *mid* = 3. Hours: $\lceil 3/3 \rceil + \lceil 6/3 \rceil + \lceil 7/3 \rceil + \lceil 11/3 \rceil = 1 + 2 + 3 + 4 = 10 > 8$. Infeasible. *lo* = 4.
- *lo* = 4, *hi* = 5: *mid* = 4. Hours: $\lceil 3/4 \rceil + \lceil 6/4 \rceil + \lceil 7/4 \rceil + \lceil 11/4 \rceil = 1 + 2 + 2 + 3 = 8 \leq 8$. Feasible. *hi* = 3.
- *lo* = 4 > *hi* = 3: loop ends.

The answer is *lo* = 4: the minimum eating speed is 4 bananas per hour.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long hoursNeeded(vector<int>& piles, int k) {
5      long long hours = 0;
6      for (int p : piles) {
7          hours += (p + k - 1) / k; // ceil(p / k)
8      }
9      return hours;
10 }
11
12 int minEatingSpeed(vector<int>& piles, int h) {
13     int lo = 1, hi = *max_element(piles.begin(), piles.end());
14
15     while (lo <= hi) {
16         int mid = lo + (hi - lo) / 2;
17         if (hoursNeeded(piles, mid) <= h) {
18             hi = mid - 1;
19         } else {
20             lo = mid + 1;

```

```

21     }
22 }
23 return lo;
24 }
25
26 int main() {
27     vector<int> piles = {3, 6, 7, 11};
28     cout << "Minimum eating speed: "
29          << minEatingSpeed(piles, 8) << endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. The candidate speed range $[1, \max(\text{piles})]$ halves each iteration ($O(\log(\max(\text{piles})))$ iterations), and each feasibility check costs $O(n)$ to scan all piles, giving an overall time complexity of

$$O(n \log(\max(\text{piles}))).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

The general recipe for binary search on the answer: (1) identify the quantity being minimized or maximized as the candidate; (2) write a feasibility function that, for a fixed candidate, answers a yes/no question with a simple (often linear) computation; (3) prove that feasibility is monotonic in the candidate using simple algebraic reasoning about the feasibility function's components; (4) binary search the candidate range using that feasibility check as the predicate. Every remaining “binary search on the answer” problem in this chapter follows exactly this four-step recipe.

2.18 Minimum Number of Days to Make M Bouquets

Problem Statement. We are given an array *bloomDay* of n flowers, where *bloomDay*[i] is the day on which flower i blooms. We need to make m bouquets, each requiring exactly k **adjacent** (consecutive in the array) flowers that have *already bloomed*. Find the **minimum day** on which it becomes possible to make all m bouquets, or -1 if it is never possible (e.g. if $m \times k > n$).

Pattern Recognition

“Minimum day on which some condition becomes achievable” is binary search on the answer again, with the candidate now being a *day* rather than a speed (Section 2.17) or a numeric root (Sections 2.15–2.16). The keyword combination “minimum day/time such that a task becomes possible” is one of the most common phrasings of this pattern, and the underlying monotonicity is intuitive: waiting longer can only help (more flowers have bloomed), never hurt.

Understanding the problem carefully

For a fixed candidate day d , a flower i is usable if *bloomDay*[i] $\leq d$. Walking through the array in order, every maximal run of consecutive usable flowers of length L can be split into $\lfloor L/k \rfloor$ bouquets (using groups of k adjacent usable flowers within that run; any leftover flowers in the run, fewer than k , cannot form another bouquet). Summing $\lfloor L/k \rfloor$ over every maximal

usable run gives the total number of bouquets makeable by day d . Define $bouquetsPossible(d)$ as this total. As d increases, no flower that was previously usable becomes unusable, and some additional flowers may become usable, so usable runs can only grow or merge – meaning $bouquetsPossible(d)$ is non-decreasing in d .

Candidate Space and Predicate

The candidate space is days in $[\min(bloomDay), \max(bloomDay)]$ (waiting before the earliest bloom or past the latest bloom is never useful). The predicate is $P(d) = (bouquetsPossible(d) \geq m)$, and we want the smallest d where P holds.

Why merging consecutive usable flowers correctly counts bouquets

The requirement that a bouquet's k flowers be *adjacent* in the original array is what makes this feasibility check more than a simple count of usable flowers divided by k : scattered usable flowers separated by an unbloomed flower cannot be combined into one bouquet, even if their total count would otherwise suffice. By scanning left to right and resetting a running streak counter to 0 whenever an unusable flower is encountered (and incrementing it, and adding to the bouquet count whenever the streak count reaches a multiple of k , otherwise just incrementing), we correctly respect the adjacency constraint while still computing $bouquetsPossible(d)$ in a single $O(n)$ pass.

Algorithm

1. If $m \times k > n$: return -1 immediately (not enough flowers could ever exist).
2. Initialize $lo \leftarrow \min(bloomDay)$, $hi \leftarrow \max(bloomDay)$.
3. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) Compute $bouquetsPossible(mid)$ via a single linear scan with a streak counter.
 - (c) If $bouquetsPossible(mid) \geq m$: feasible; try earlier: $hi \leftarrow mid - 1$.
 - (d) Else: infeasible; try later: $lo \leftarrow mid + 1$.
4. Return lo .

Algorithm 30 Minimum Day for M Bouquets

```

1: procedure MINDAYS( $bloomDay, m, k$ )
2:   if  $m \times k > n$  then return  $-1$ 
3:   end if
4:    $lo \leftarrow \min(bloomDay)$ ,  $hi \leftarrow \max(bloomDay)$ 
5:   while  $lo \leq hi$  do
6:      $mid \leftarrow lo + (hi - lo)/2$ 
7:      $bouquets \leftarrow \text{BOUQUETSPossible}(bloomDay, mid, k)$ 
8:     if  $bouquets \geq m$  then
9:        $hi \leftarrow mid - 1$ 
10:    else
11:       $lo \leftarrow mid + 1$ 
12:    end if
13:  end while
14:  return  $lo$ 
15: end procedure

```

Dry run

Dry Run. Take $\text{bloomDay} = [7, 7, 7, 7, 12, 7, 7]$, $m = 2$, $k = 3$.

- $m \times k = 6 \leq n = 7$, so an answer may exist.
- Try $d = 12$ (the maximum bloom day, as an early sanity check): every flower is usable, giving one run of length 7, so $\text{bouquetsPossible}(12) = \lfloor 7/3 \rfloor = 2 \geq 2$. Feasible.
- Try $d = 7$ (binary search would reach this candidate): flowers at indices 0, 1, 2, 3, 5, 6 are usable (bloom day $7 \leq 7$), but index 4 (bloom day 12) is not. This splits the array into two runs: indices [0, 3] (length 4) and [5, 6] (length 2). Bouquets: $\lfloor 4/3 \rfloor + \lfloor 2/3 \rfloor = 1 + 0 = 1 < 2$. Infeasible.

The binary search narrows between 7 and 12; checking the remaining candidates confirms that day 12 is the earliest day at which 2 bouquets become possible (the run of 4 usable flowers before index 4 cannot be split to free up a third group without waiting for the last flower to bloom).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int bouquetsPossible(vector<int>& bloomDay, int day, int k) {
5      int n = bloomDay.size();
6      int streak = 0, bouquets = 0;
7
8      for (int i = 0; i < n; i++) {
9          if (bloomDay[i] <= day) {
10             streak++;
11             if (streak == k) {
12                 bouquets++;
13                 streak = 0;
14             }
15             } else {
16                 streak = 0;
17             }
18         }
19     return bouquets;
20 }
21
22 int minDays(vector<int>& bloomDay, int m, int k) {
23     int n = bloomDay.size();
24     if ((long long)m * k > n) return -1;
25
26     int lo = *min_element(bloomDay.begin(), bloomDay.end());
27     int hi = *max_element(bloomDay.begin(), bloomDay.end());
28
29     while (lo <= hi) {
30         int mid = lo + (hi - lo) / 2;
31         if (bouquetsPossible(bloomDay, mid, k) >= m) {
32             hi = mid - 1;
33         } else {
34             lo = mid + 1;
35         }
36     }

```

```

37     return lo;
38 }
39
40 int main() {
41     vector<int> bloomDay = {7, 7, 7, 7, 12, 7, 7};
42     cout << "Minimum day: " << minDays(bloomDay, 2, 3) << endl;
43     return 0;
44 }

```

Complexity Analysis

Time Complexity. The candidate day range halves each iteration, costing $O(\log(\max(\text{bloomDay}) - \min(\text{bloomDay})))$ iterations, and each feasibility check costs $O(n)$, giving an overall time complexity of

$$O(n \log(\max(\text{bloomDay}))).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

When a feasibility check involves an *adjacency* or *contiguity* requirement (here, k consecutive bloomed flowers), the linear-scan feasibility function typically needs a running streak counter that resets on disqualification, rather than a simple global count. This adjacency-aware counting pattern recurs in Section 2.25 and is worth recognizing as its own small sub-template within the larger binary-search-on-the-answer recipe.

2.19 Find the Smallest Divisor Given a Threshold

Problem Statement. We are given an array *nums* of n positive integers and an integer *threshold*. We must choose a positive integer *divisor*, and the resulting sum $\sum_i \lceil \text{nums}[i] / \text{divisor} \rceil$ must be $\leq \text{threshold}$. Find the **smallest** such divisor.

Pattern Recognition

This is structurally identical to Koko Eating Bananas (Section 2.17): the keywords “*smallest divisor*” and “*sum of ceiling divisions must not exceed a threshold*” map directly onto “*smallest eating speed*” and “*total hours must not exceed the limit*.” Once you notice the structural identity, the entire derivation transfers without modification – only the variable names change.

Understanding the problem carefully

Define $\text{sumDivisions}(\text{divisor}) = \sum_i \lceil \text{nums}[i] / \text{divisor} \rceil$. As *divisor* increases, each term $\lceil \text{nums}[i] / \text{divisor} \rceil$ can only decrease or stay the same, so *sumDivisions* is non-increasing in *divisor* – the same monotonicity structure as *hoursNeeded* in Section 2.17.

Candidate Space and Predicate

The candidate space is integer divisors in $[1, \max(\text{nums})]$ (any divisor at least as large as the largest element reduces every term to 1, so nothing larger is ever needed). The predicate is $P(\text{divisor}) = (\text{sumDivisions}(\text{divisor}) \leq \text{threshold})$, and we want the smallest divisor where P holds.

Why the derivation transfers without modification

There is no new reasoning required: this is a direct relabeling of Koko Eating Bananas, with $piles \rightarrow nums$, $k \rightarrow divisor$, and $h \rightarrow threshold$. This section exists specifically to demonstrate that once you recognize the abstract shape (*minimize a divisor/rate subject to a sum-of-ceiling-divisions budget*), you should reuse the exact same code template rather than rederiving it – pattern recognition, not just algorithmic cleverness, is the actual skill being exercised here.

Algorithm

Identical in structure to Algorithm 2.17's MINEATINGSPEED, with the feasibility check replaced by $sumDivisions(mid) \leq threshold$.

Dry run

Dry Run. Take $nums = [1, 2, 5, 9]$, $threshold = 6$.

- $lo = 1, hi = 9$: $mid = 5$. Sum: $\lceil 1/5 \rceil + \lceil 2/5 \rceil + \lceil 5/5 \rceil + \lceil 9/5 \rceil = 1 + 1 + 1 + 2 = 5 \leq 6$. Feasible. $hi = 4$.
- $lo = 1, hi = 4$: $mid = 2$. Sum: $\lceil 1/2 \rceil + \lceil 2/2 \rceil + \lceil 5/2 \rceil + \lceil 9/2 \rceil = 1 + 1 + 3 + 5 = 10 > 6$. Infeasible. $lo = 3$.
- $lo = 3, hi = 4$: $mid = 3$. Sum: $\lceil 1/3 \rceil + \lceil 2/3 \rceil + \lceil 5/3 \rceil + \lceil 9/3 \rceil = 1 + 1 + 2 + 3 = 7 > 6$. Infeasible. $lo = 4$.
- $lo = 4, hi = 4$: $mid = 4$. Sum: $\lceil 1/4 \rceil + \lceil 2/4 \rceil + \lceil 5/4 \rceil + \lceil 9/4 \rceil = 1 + 1 + 2 + 3 = 7 > 6$. Infeasible. $lo = 5$.
- $lo = 5 > hi = 4$: loop ends.

The answer is 5.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long sumDivisions(vector<int>& nums, int divisor) {
5      long long sum = 0;
6      for (int x : nums) {
7          sum += (x + divisor - 1) / divisor; // ceil(x / divisor)
8      }
9      return sum;
10 }
11
12 int smallestDivisor(vector<int>& nums, int threshold) {
13     int lo = 1, hi = *max_element(nums.begin(), nums.end());
14
15     while (lo <= hi) {
16         int mid = lo + (hi - lo) / 2;
17         if (sumDivisions(nums, mid) <= threshold) {
18             hi = mid - 1;
19         } else {
20             lo = mid + 1;
21         }
22     }

```

```

23     return lo;
24 }
25
26 int main() {
27     vector<int> nums = {1, 2, 5, 9};
28     cout << "Smallest divisor: "
29           << smallestDivisor(nums, 6) << endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity.

$$O(n \log(\max(nums))),$$

identical in structure to Section 2.17. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

Recognizing that two differently-worded problems are the same template underneath is just as valuable as deriving a template from scratch. Maintain a small mental (or literal) catalogue of solved binary-search-on-the-answer templates – “minimize rate subject to a ceiling-division sum budget” is now one entry in that catalogue, covering both this problem and Koko Eating Bananas.

2.20 Capacity to Ship Packages within D Days

Problem Statement. We are given an array *weights* of n package weights, which must be shipped **in the given order** (no reordering). Each day, the ship loads a contiguous run of packages, in order, whose total weight does not exceed the ship’s capacity, then sails; the next day starts loading from the next unshipped package. We must find the **minimum ship capacity** that allows all packages to be shipped within D days.

Pattern Recognition

“Minimum capacity such that all items can be processed within a day limit, items must stay in order” is another binary-search-on-the-answer problem, but the feasibility check itself is structurally different from Sections 2.17–2.19: rather than summing independent per-item costs, we must **greedily simulate** a day-by-day loading process, because the order constraint means packages cannot be considered independently of their neighbors. Whenever a binary-search-on-the-answer problem also mentions *order* or *contiguity* constraints on how items are grouped, expect the feasibility check itself to require a small greedy simulation rather than a single summation formula.

Understanding the problem carefully

For a fixed candidate capacity c , simulate the shipping process greedily: start a new day, and keep adding the next package’s weight to today’s load as long as it does not exceed c ; the moment adding the next package would exceed c , end the day and start a new one with that package. This greedy simulation is itself optimal for *counting the minimum days needed for a fixed capacity* – packing each day as full as possible before moving to the next day can never use *more* days than any other valid loading strategy, because deliberately under-loading a day only defers weight to later days without ever reducing the total weight that must still be shipped.

Define $daysNeeded(c)$ as the number of days this greedy simulation takes. As c increases, each day can carry at least as much as before, so $daysNeeded(c)$ is non-increasing in c .

Candidate Space and Predicate

The candidate space is integer capacities in $[\max(weights), \sum(weights)]$: the capacity must be at least the heaviest single package (otherwise that package alone could never be shipped), and capacity equal to the total weight always finishes in exactly 1 day. The predicate is $P(c) = (daysNeeded(c) \leq D)$, and we want the smallest c where P holds.

Why the greedy simulation inside the predicate is valid

This is a small but important instance of *nesting* a greedy algorithm inside a binary search predicate: the binary search handles *which* capacity to test, while a greedy simulation – packing each day as full as the fixed capacity allows before moving on – handles *whether* that capacity is feasible. The greedy simulation’s correctness rests on the exchange argument that ending a day early (loading less than the maximum the capacity allows) can never help and can only push more weight into later days, since the in-order constraint means no future package can be moved earlier to compensate. This nested structure – binary search over a monotonic outer property, with a greedy or simulation-based feasibility check inside – is common throughout the “binary search on the answer” family and is exactly what makes capacity c feasible or infeasible a well-defined, monotonic question.

Algorithm

1. Initialize $lo \leftarrow \max(weights)$, $hi \leftarrow \sum(weights)$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) Compute $daysNeeded(mid)$ by greedily simulating loading days.
 - (c) If $daysNeeded(mid) \leq D$: feasible; try smaller: $hi \leftarrow mid - 1$.
 - (d) Else: infeasible; try larger: $lo \leftarrow mid + 1$.
3. Return lo .

Algorithm 31 Minimum Capacity to Ship within D Days

```

1: procedure SHIPWITHINDAYS( $weights, D$ )
2:    $lo \leftarrow \max(weights)$ ,  $hi \leftarrow \sum(weights)$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:      $days \leftarrow \text{DAYSNEEDED}(weights, mid)$ 
6:     if  $days \leq D$  then
7:        $hi \leftarrow mid - 1$ 
8:     else
9:        $lo \leftarrow mid + 1$ 
10:    end if
11:  end while
12:  return  $lo$ 
13: end procedure

```

Dry run

Dry Run. Take $weights = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$, $D = 5$.
 $\max = 10$, $\sum = 55$.

- $lo = 10, hi = 55$: $mid = 32$. Simulating: day loads accumulate $1+2+3+4+5+6+7 = 28$ (next, 8, would make $36 > 32$, so day ends); day 2 loads $8+9+10 = 27$. Total $daysNeeded = 2 \leq 5$. Feasible. $hi = 31$.
- Binary search continues narrowing downward; eventually it converges to $lo = 15$, the minimum feasible capacity (with capacity 15: day loads $1+2+3+4+5 = 15$; day 2 loads $6+7 = 13$ (next, 8, would make $21 > 15$); day 3 loads 8 alone (next, 9, would make $17 > 15$); day 4 loads 9 alone (next, 10, would make $19 > 15$); day 5 loads 10. Total $daysNeeded = 5 \leq 5$ – exactly feasible, and capacity 14 can be checked to need 6 days, confirming 15 is minimal).

The answer is 15.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int daysNeeded(vector<int>& weights, int capacity) {
5     int days = 1;
6     long long currentLoad = 0;
7
8     for (int w : weights) {
9         if (currentLoad + w > capacity) {
10             days++;
11             currentLoad = 0;
12         }
13         currentLoad += w;
14     }
15     return days;
16 }
17
18 int shipWithinDays(vector<int>& weights, int D) {
19     int lo = *max_element(weights.begin(), weights.end());
20     int hi = accumulate(weights.begin(), weights.end(), 0);
21
22     while (lo <= hi) {
23         int mid = lo + (hi - lo) / 2;
24         if (daysNeeded(weights, mid) <= D) {
25             hi = mid - 1;
26         } else {
27             lo = mid + 1;
28         }
29     }
30     return lo;
31 }
32
33 int main() {
34     vector<int> weights = {1,2,3,4,5,6,7,8,9,10};
35     cout << "Minimum capacity: "
36          << shipWithinDays(weights, 5) << endl;
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. The candidate capacity range halves each iteration, costing $O(\log(\sum(weights)))$ iterations, and each feasibility check costs $O(n)$ for the greedy simulation, giving

$$O(n \log(\sum(weights))).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

When the items being processed must stay in a fixed order, the feasibility check for binary search on the answer is usually a greedy simulation (pack as much as possible into each “bucket” before starting a new one) rather than a simple per-item summation. Verify, separately, that the greedy simulation itself is optimal for the fixed-candidate sub-problem – here, via the exchange argument that under-loading a day never helps – before trusting it as the feasibility predicate.

2.21 Kth Missing Positive Number

Problem Statement. We are given a sorted array *arr* of *n* distinct positive integers, and an integer *k*. We must find the *k*-th positive integer that is **missing** from *arr*.

Pattern Recognition

There is no array index to “find *X* at” here – we are counting *missing* numbers, which is a different flavor of monotonic quantity than anything in Sections 2.2–2.14. Whenever a sorted-array problem asks about *missing* elements (gaps in an otherwise-expected sequence) and wants the *k*-th such gap, look for a formula that counts “how many integers are missing up to this point in the array,” and binary search for the boundary where that count first reaches *k*.

Understanding the problem carefully

Consider index *i* (0-indexed) in the array. If none of *arr*[0], ..., *arr*[*i*] were missing from the positive integers 1, ..., *arr*[*i*], there would be exactly *arr*[*i*] of them present, occupying *arr*[*i*] slots; but only *i* + 1 of them *are* actually present (the first *i* + 1 elements of the array), so

$$missingCount(i) = arr[i] - (i + 1)$$

counts exactly how many positive integers from 1 to *arr*[*i*] are missing from the array. Because *arr* is sorted and strictly increasing, *arr*[*i*] grows at least as fast as *i* does, so *missingCount*(*i*) is non-decreasing in *i* – exactly the monotonic structure we need.

Candidate Space and Predicate

The candidate space is indices 0, ..., *n* − 1 (with an implicit “beyond the array” fallback). The predicate is $P(i) = (missingCount(i) \geq k)$, and we want the smallest index where *P* holds.

Why *k* + *lo* is the final answer

Let *lo* be the smallest index where *missingCount*(*lo*) ≥ *k* (or *n*, if even the last element’s missing count is still below *k*, meaning the answer lies beyond the array entirely). Just before this index,

$missingCount(lo - 1) < k$, meaning fewer than k numbers are missing among $1, \dots, arr[lo - 1]$; from $arr[lo - 1] + 1$ onward (which is not yet present in the array, since $arr[lo]$ is strictly larger), missing numbers resume counting up consecutively until reaching $missingCount(lo) \geq k$. A clean way to see the final formula: exactly lo array elements (indices $0, \dots, lo - 1$) are known to be *smaller* than the answer (since the crossover into “enough missing numbers” has not yet happened at index $lo - 1$), so among the first $k + lo$ positive integers, exactly lo are present in the array and the remaining k are missing – meaning the k -th missing positive integer is precisely $k + lo$.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow n - 1$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) Compute $missingCount \leftarrow arr[mid] - (mid + 1)$.
 - (c) If $missingCount < k$: $lo \leftarrow mid + 1$.
 - (d) Else: $hi \leftarrow mid - 1$.
3. Return $k + lo$.

Algorithm 32 Kth Missing Positive Number

```

1: procedure KTHMISSING( $arr, k$ )
2:    $lo \leftarrow 0$ ,  $hi \leftarrow n - 1$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:      $missing \leftarrow arr[mid] - (mid + 1)$ 
6:     if  $missing < k$  then
7:        $lo \leftarrow mid + 1$ 
8:     else
9:        $hi \leftarrow mid - 1$ 
10:    end if
11:  end while
12:  return  $k + lo$ 
13: end procedure

```

Dry run

Dry Run. Take $arr = [2, 3, 4, 7, 11]$, $k = 5$.

- $lo = 0, hi = 4$: $mid = 2$, $missing = arr[2] - 3 = 4 - 3 = 1 < 5$. $lo = 3$.
- $lo = 3, hi = 4$: $mid = 3$, $missing = 7 - 4 = 3 < 5$. $lo = 4$.
- $lo = 4, hi = 4$: $mid = 4$, $missing = 11 - 5 = 6 \geq 5$. $hi = 3$.
- $lo = 4 > hi = 3$: loop ends.

The answer is $k + lo = 5 + 4 = 9$. Checking by hand: the missing positive integers, in order, are 1, 5, 6, 8, 9, 10, 12, $\dots$, and the 5-th one is indeed 9.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int findKthMissing(vector<int>& arr, int k) {
5     int n = arr.size();
6     int lo = 0, hi = n - 1;
7
8     while (lo <= hi) {
9         int mid = lo + (hi - lo) / 2;
10        int missing = arr[mid] - (mid + 1);
11        if (missing < k) {
12            lo = mid + 1;
13        } else {
14            hi = mid - 1;
15        }
16    }
17    return k + lo;
18 }
19
20 int main() {
21     vector<int> arr = {2, 3, 4, 7, 11};
22     cout << "5th missing positive number: "
23          << findKthMissing(arr, 5) << endl;
24     return 0;
25 }

```

Complexity Analysis

Time Complexity. The search window halves each iteration, giving

$$O(\log n).$$

This is a substantial improvement over the natural $O(n)$ linear-scan approach of walking through the array while counting missing numbers one at a time. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

When a problem asks about “missing” elements relative to an expected dense sequence (here, the positive integers), look for a closed-form counting function – “expected count minus actual count up to this point” – that is monotonic, and binary search on that function’s crossover rather than scanning and counting one gap at a time.

2.22 Aggressive Cows Problem

Problem Statement. We are given n stall positions $stalls[0..n-1]$ along a number line, and k cows to place into these stalls (at most one cow per stall). The cows are aggressive, so we want to place them to **maximize the minimum distance** between any two cows. Find this maximum possible minimum distance.

Pattern Recognition

“Maximize the minimum distance/gap between placed items” is the mirror image of every problem so far in this section: until now we *minimized* a candidate (speed, day, divisor, capacity) subject to a feasibility constraint that became *easier* as the candidate grew. Here, feasibility (“can we place all k cows with at least this much spacing”) becomes **harder** as the candidate distance grows, so we are finding the **largest** feasible candidate, not the smallest. Whenever you see “maximize the minimum ...”, expect to flip the binary search’s narrowing direction relative to the “minimize subject to feasibility” problems seen earlier.

Understanding the problem carefully

Sort the stall positions first – only relative order matters for spacing, and an unsorted input gives no useful structure to exploit. For a fixed candidate minimum distance d , ask: can we place all k cows in the sorted stalls such that every pair of adjacent placed cows is at least d apart? This is checked greedily: place the first cow at the first stall, then repeatedly place the next cow at the first stall that is at least d away from the previously placed cow. If we manage to place all k cows this way, d is feasible.

As d increases, this greedy placement can only place fewer or equal cows (every placement decision becomes at least as restrictive), so the number of cows placeable is non-increasing in d – meaning feasibility ($\geq k$ cows placed) is **true** for small d and **false** for large d , the mirror image of the earlier sections’ monotonicity direction.

Candidate Space and Predicate

The candidate space is distances $d \in [1, \max(stalls) - \min(stalls)]$. The predicate is $P(d) =$ (greedy placement with spacing $\geq d$ fits all k cows), and we want the **largest** d where P holds – since P is true for small d and false for large d , we record a successful candidate and narrow *right* (search for an even larger feasible d), the mirrored version of the “narrow left after success” template used for minimize-type problems.

Why the greedy feasibility check is optimal

For a *fixed* target spacing d , greedily placing each next cow as early as possible (at the first stall that is at least d from the previous placement) maximizes the number of cows we can fit using that spacing: placing a cow any later than necessary only makes it harder to fit subsequent cows, never easier, since stalls are processed in increasing order and no stall can be revisited. This is the same kind of exchange argument used for interval scheduling in the Greedy chapter – the earliest valid placement is always at least as good as any later valid placement for the purpose of fitting more items afterward.

Algorithm

1. Sort *stalls*.
2. Initialize $lo \leftarrow 1$, $hi \leftarrow stalls[n - 1] - stalls[0]$.
3. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) Greedily count how many cows fit with minimum spacing mid .
 - (c) If $count \geq k$: feasible; record mid , try larger: $lo \leftarrow mid + 1$.
 - (d) Else: infeasible; try smaller: $hi \leftarrow mid - 1$.

4. Return the recorded answer (equivalently hi once the loop ends).

Algorithm 33 Aggressive Cows

```

1: procedure AGGRESSIVECOWS( $stalls, k$ )
2:   sort  $stalls$ 
3:    $lo \leftarrow 1, hi \leftarrow stalls[n - 1] - stalls[0]$ 
4:   while  $lo \leq hi$  do
5:      $mid \leftarrow lo + (hi - lo)/2$ 
6:      $count \leftarrow COWSPLACEABLE(stalls, mid)$ 
7:     if  $count \geq k$  then
8:        $lo \leftarrow mid + 1$ 
9:     else
10:       $hi \leftarrow mid - 1$ 
11:    end if
12:  end while
13:  return  $hi$ 
14: end procedure

```

Dry run

Dry Run. Take $stalls = [0, 3, 4, 7, 10, 9]$, $k = 4$. Sorted: $[0, 3, 4, 7, 9, 10]$.

- $lo = 1, hi = 10$: $mid = 5$. Greedy: place at 0; next ≥ 5 is 7, place; next ≥ 12 – none. Only 2 cows placed < 4 . Infeasible. $hi = 4$.
- $lo = 1, hi = 4$: $mid = 2$. Greedy: place at 0; next ≥ 2 is 3, place; next ≥ 5 is 7, place; next ≥ 9 is 9, place. 4 cows placed ≥ 4 . Feasible. $lo = 3$.
- $lo = 3, hi = 4$: $mid = 3$. Greedy: place at 0; next ≥ 3 is 3, place; next ≥ 6 is 7, place; next ≥ 10 is 10, place. 4 cows placed ≥ 4 . Feasible. $lo = 4$.
- $lo = 4, hi = 4$: $mid = 4$. Greedy: place at 0; next ≥ 4 is 4, place; next ≥ 8 is 9, place; next ≥ 13 – none. Only 3 cows placed < 4 . Infeasible. $hi = 3$.
- $lo = 4 > hi = 3$: loop ends.

The answer is $hi = 3$: the maximum minimum distance is 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int cowsPlaceable(vector<int>& stalls, int d) {
5     int count = 1;
6     int lastPos = stalls[0];
7
8     for (int i = 1; i < (int)stalls.size(); i++) {
9         if (stalls[i] - lastPos >= d) {
10             count++;
11             lastPos = stalls[i];
12         }
13     }
14     return count;

```

```

15 }
16
17 int aggressiveCows(vector<int>& stalls, int k) {
18     sort(stalls.begin(), stalls.end());
19     int n = stalls.size();
20     int lo = 1, hi = stalls[n - 1] - stalls[0];
21
22     while (lo <= hi) {
23         int mid = lo + (hi - lo) / 2;
24         if (cowsPlaceable(stalls, mid) >= k) {
25             lo = mid + 1;
26         } else {
27             hi = mid - 1;
28         }
29     }
30     return hi;
31 }
32
33 int main() {
34     vector<int> stalls = {0, 3, 4, 7, 10, 9};
35     cout << "Maximum minimum distance: "
36          << aggressiveCows(stalls, 4) << endl;
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. Sorting costs $O(n \log n)$. The candidate distance range halves each iteration, costing $O(\log(\max(stalls) - \min(stalls)))$ iterations, and each feasibility check costs $O(n)$, giving an overall time complexity of

$$O(n \log n + n \log(\max(stalls))).$$

Space Complexity. $O(1)$ auxiliary space beyond sorting.

Key Takeaway

“Maximize the minimum” and “minimize the maximum” problems both binary search on the answer, but with the narrowing direction flipped relative to each other: for “maximize the minimum,” record a successful candidate and search for a *larger* one; for “minimize the maximum” (as in Sections 2.17–2.20), record a successful candidate and search for a *smaller* one. Always pause to identify which of the two flavors you are facing before writing the narrowing logic.

2.23 Book Allocation Problem

Problem Statement. We are given n books, where book i has $pages[i]$ pages, and m students. Each student must be allocated a **contiguous** range of books (in the given order), every book must be allocated to exactly one student, and every student must receive at least one book. We must choose the allocation to **minimize the maximum** number of pages assigned to any single student.

Pattern Recognition

“Minimize the maximum load, items assigned in contiguous order, divided among a fixed number of workers/students” is exactly the same shape as Capacity to Ship Packages within D Days (Section 2.20): there, we fixed the number of “buckets” indirectly through a day limit D and a per-bucket capacity candidate; here, the number of buckets is fixed directly as m students. Once you notice that “contiguous partition into a fixed number of groups, minimize the maximum group sum” is the same template regardless of whether it is phrased as books, packages, or paint sections (see Section 2.25), you can reuse the identical greedy feasibility check every time.

Understanding the problem carefully

For a fixed candidate maximum $maxPages$, greedily simulate assigning books to the current student until adding the next book’s pages would exceed $maxPages$; at that point, move to the next student. This yields $studentsNeeded(maxPages)$, the minimum number of students required if no student may hold more than $maxPages$ total pages. As $maxPages$ increases, each student can absorb more books before being forced to stop, so $studentsNeeded$ is non-increasing in $maxPages$.

Candidate Space and Predicate

The candidate space is integer page totals in $[\max(pages), \sum(pages)]$ (a single student must be able to hold at least the largest individual book, and a single student holding everything always works). The predicate is $P(maxPages) = (studentsNeeded(maxPages) \leq m)$, and we want the smallest $maxPages$ where P holds – this is a “minimize subject to feasibility” problem, narrowing the same direction as Sections 2.17–2.20, not the flipped direction of Aggressive Cows.

Why this is identical in structure to ship packages

The greedy feasibility check here is justified by precisely the same exchange argument as Section 2.20: greedily filling each student with as many books as the candidate maximum allows, before moving to the next student, never uses more students than any other valid contiguous allocation respecting that same maximum, because deferring a book to a later student only ever makes that later student’s job harder, never easier, and books cannot be reordered.

Algorithm

1. Initialize $lo \leftarrow \max(pages)$, $hi \leftarrow \sum(pages)$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) Compute $studentsNeeded(mid)$ via greedy simulation.
 - (c) If $studentsNeeded(mid) \leq m$: feasible; try smaller: $hi \leftarrow mid - 1$.
 - (d) Else: infeasible; try larger: $lo \leftarrow mid + 1$.
3. Return lo .

Algorithm 34 Book Allocation

```

1: procedure BOOKALLOCATION(pages, m)
2:   lo  $\leftarrow$  max(pages), hi  $\leftarrow$   $\sum$ (pages)
3:   while lo  $\leq$  hi do
4:     mid  $\leftarrow$  lo + (hi - lo)/2
5:     students  $\leftarrow$  STUDENTSNEEDED(pages, mid)
6:     if students  $\leq$  m then
7:       hi  $\leftarrow$  mid - 1
8:     else
9:       lo  $\leftarrow$  mid + 1
10:    end if
11:  end while
12:  return lo
13: end procedure

```

Dry run

Dry Run. Take *pages* = [12, 34, 67, 90], *m* = 2.
 max = 90, $\sum$ = 203.

- *lo* = 90, *hi* = 203: *mid* = 146. Greedy: student 1 takes 12+34+67 = 113 (next, 90, would make 203 > 146, stop); student 2 takes 90. *studentsNeeded* = 2 $\leq$ 2. Feasible. *hi* = 145.
- Narrowing continues; eventually the search converges to *lo* = 113: with maximum 113, student 1 takes 12+34+67 = 113 (next, 90, would make 203 > 113, stop); student 2 takes 90. Still 2 students. Checking 112: student 1 can only take 12+34 = 46 (next, 67, would make 113 > 112, stop); student 2 takes 67 (next, 90, would make 157 > 112, stop); student 3 takes 90 – that needs 3 students, exceeding *m* = 2. So 113 is indeed minimal.

The answer is 113.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int studentsNeeded(vector<int>& pages, int maxPages) {
5      int students = 1;
6      long long currentLoad = 0;
7
8      for (int p : pages) {
9          if (currentLoad + p > maxPages) {
10             students++;
11             currentLoad = 0;
12         }
13         currentLoad += p;
14     }
15     return students;
16 }
17
18 int bookAllocation(vector<int>& pages, int m) {
19     int lo = *max_element(pages.begin(), pages.end());
20     int hi = accumulate(pages.begin(), pages.end(), 0);
21

```

```

22     while (lo <= hi) {
23         int mid = lo + (hi - lo) / 2;
24         if (studentsNeeded(pages, mid) <= m) {
25             hi = mid - 1;
26         } else {
27             lo = mid + 1;
28         }
29     }
30     return lo;
31 }
32
33 int main() {
34     vector<int> pages = {12, 34, 67, 90};
35     cout << "Minimum maximum pages: "
36          << bookAllocation(pages, 2) << endl;
37     return 0;
38 }

```

Complexity Analysis

Time Complexity.

$$O(n \log(\sum(pages))),$$

identical in structure to Section 2.20. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

“Partition a sequence into a fixed number of contiguous groups to minimize the maximum group sum” is a single template with at least three names on this sheet: Book Allocation, Capacity to Ship Packages, and Painter’s Partition (Section 2.25). Learn the template once – greedy bucket-filling feasibility check nested inside a minimize-subject-to-feasibility binary search – and recognize its disguises rather than re-deriving it from scratch each time.

2.24 Split Array – Largest Sum

Problem Statement. We are given an array *nums* of *n* non-negative integers and an integer *m*. We must split *nums* into *m* **contiguous, non-empty** subarrays so as to **minimize the largest sum** among the *m* subarrays.

Pattern Recognition

This is, word for word, the Book Allocation Problem (Section 2.23) with “books/pages/students” relabeled as “array/elements/subarrays.” There is no new idea to extract here beyond confirming, explicitly, that you recognize the relabeling – which is itself the point of including this problem on the sheet immediately after book allocation: to test whether the pattern has truly been internalized, or only memorized under one specific cover story.

Understanding the problem carefully

Exactly as in Section 2.23, define *partsNeeded(maxSum)* as the minimum number of contiguous parts needed if no part’s sum may exceed *maxSum*, computed by the same greedy bucket-filling simulation. This is non-increasing in *maxSum*, giving the same monotonic structure.

Candidate Space and Predicate

Candidate space: integer sums in $[\max(nums), \sum(nums)]$. Predicate: $P(maxSum) = (partsNeeded(maxSum) \leq m)$. Want the smallest $maxSum$ where P holds.

Algorithm

Identical to Algorithm 2.23's BOOKALLOCATION, with *pages* relabeled *nums* and *m* meaning the number of parts directly.

Dry run

Dry Run. Take $nums = [7, 2, 5, 10, 8]$, $m = 2$.

$\max = 10$, $\sum = 32$. Binary search converges to $maxSum = 18$: one valid split is $[7, 2, 5]$ (sum 14) and $[10, 8]$ (sum 18), giving largest sum 18; checking 17 shows it forces a third part ($[7, 2, 5] = 14$, then $[10] = 10$ would leave $[8]$ separately, or $[7, 2, 5, 10] = 24 > 17$ forces an earlier break, ultimately requiring 3 parts), confirming 18 is minimal.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int partsNeeded(vector<int>& nums, int maxSum) {
5      int parts = 1;
6      long long currentSum = 0;
7
8      for (int x : nums) {
9          if (currentSum + x > maxSum) {
10             parts++;
11             currentSum = 0;
12         }
13         currentSum += x;
14     }
15     return parts;
16 }
17
18 int splitArray(vector<int>& nums, int m) {
19     int lo = *max_element(nums.begin(), nums.end());
20     int hi = accumulate(nums.begin(), nums.end(), 0);
21
22     while (lo <= hi) {
23         int mid = lo + (hi - lo) / 2;
24         if (partsNeeded(nums, mid) <= m) {
25             hi = mid - 1;
26         } else {
27             lo = mid + 1;
28         }
29     }
30     return lo;
31 }
32
33 int main() {
34     vector<int> nums = {7, 2, 5, 10, 8};
35     cout << "Minimized largest sum: "
36          << splitArray(nums, 2) << endl;

```

```

37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(n \log(\sum(nums)))$, identical to Section 2.23. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

The fastest way to solve this problem is to recognize it as Book Allocation in disguise. Building a small catalogue of templates – rather than a catalogue of individual problems – is what lets binary search on the answer scale to dozens of seemingly distinct sheet problems with only a handful of underlying ideas.

2.25 Painter’s Partition Problem

Problem Statement. We are given n boards with lengths $boards[0..n-1]$ and k painters. Each painter paints a **contiguous** set of boards, and painting one unit of board length takes one unit of time (painters work in parallel, each on their own assigned contiguous range). We must assign boards to painters so as to **minimize the time the slowest painter takes** (equivalently, minimize the maximum total board length assigned to any one painter).

Pattern Recognition

A third disguise of the same template as Sections 2.23–2.24: “boards/painters” plays the role of “pages/students” or “array elements/subarray parts.” By this third repetition, the goal is no longer to re-derive anything, but to confirm that the pattern – *partition a sequence into a fixed number of contiguous groups, minimize the maximum group sum, via binary search on the answer with a greedy bucket-filling feasibility check* – has become an immediately recognizable, reusable unit, regardless of the cover story a problem uses to disguise it.

Understanding the problem carefully

Define $paintersNeeded(maxTime)$ exactly as in Section 2.23: greedily assign boards to the current painter until adding the next board would exceed $maxTime$, then move to the next painter. This count is non-increasing in $maxTime$, by the same reasoning as the prior two sections.

Candidate Space and Predicate

Candidate space: integer times in $[\max(boards), \sum(boards)]$. Predicate: $P(maxTime) = (paintersNeeded(maxTime) \leq k)$. Want the smallest $maxTime$ where P holds.

Algorithm

Identical to Algorithm 2.23’s BOOKALLOCATION, with *pages* relabeled *boards* and *m* relabeled *k*.

Dry run

Dry Run. Take $boards = [10, 20, 30, 40]$, $k = 2$.

$\max = 40$, $\sum = 100$. Binary search converges to $maxTime = 60$: one painter takes $[10, 20, 30]$ (sum 60), the other takes $[40]$ (sum 40); checking 59 would force the first painter to

stop after $[10, 20] = 30$ (since adding 30 would make $60 > 59$), needing a third painter for $[30]$ and $[40]$ separately exceeding $k = 2$, so 60 is minimal.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int paintersNeeded(vector<int>& boards, int maxTime) {
5     int painters = 1;
6     long long currentTime = 0;
7
8     for (int len : boards) {
9         if (currentTime + len > maxTime) {
10             painters++;
11             currentTime = 0;
12         }
13         currentTime += len;
14     }
15     return painters;
16 }
17
18 int paintersPartition(vector<int>& boards, int k) {
19     int lo = *max_element(boards.begin(), boards.end());
20     int hi = accumulate(boards.begin(), boards.end(), 0);
21
22     while (lo <= hi) {
23         int mid = lo + (hi - lo) / 2;
24         if (paintersNeeded(boards, mid) <= k) {
25             hi = mid - 1;
26         } else {
27             lo = mid + 1;
28         }
29     }
30     return lo;
31 }
32
33 int main() {
34     vector<int> boards = {10, 20, 30, 40};
35     cout << "Minimum time: " << paintersPartition(boards, 2) << endl;
36     return 0;
37 }

```

Complexity Analysis

Time Complexity. $O(n \log(\sum(\text{boards})))$. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

Three problems, three cover stories, one template. If you find yourself about to write a fresh derivation for a “partition into k contiguous groups, minimize the maximum group sum” problem, stop and check the catalogue of Sections 2.23–2.25 first.

2.26 Minimize the Maximum Distance Between Gas Stations

Problem Statement. We are given n gas station positions, sorted along a number line, and an integer k . We may insert k additional gas stations **anywhere** (not restricted to integer positions) among the existing ones. We must choose where to insert them so as to **minimize the largest gap** between any two consecutive gas stations (existing or newly inserted), and report that minimized largest gap.

Pattern Recognition

This problem keeps the “minimize the maximum, subject to a budget” shape of Sections 2.23–2.25, but with one crucial new wrinkle: the answer is a **real number**, not an integer, because stations may be placed at fractional positions. Whenever a binary-search-on-the-answer problem allows continuous (real-valued) placement rather than discrete/integer choices, the `while(lo <= hi)` integer loop must be replaced with a fixed-precision loop that runs until hi and lo are within some small tolerance ε of each other.

Understanding the problem carefully

Consider the existing gaps between consecutive original stations: $gap_1, gap_2, \dots, gap_{n-1}$. For a fixed candidate maximum allowed gap d , the minimum number of new stations needed to bring gap gap_i down to segments no larger than d is

$$\left\lceil \frac{gap_i}{d} \right\rceil - 1$$

(inserting c points into a single gap of length L divides it into $c+1$ equal-or-smaller segments; we need the smallest c such that $L/(c+1) \leq d$, which rearranges to $c \geq L/d - 1$, i.e. $c = \lceil L/d \rceil - 1$ for integer c). Summing this over every existing gap gives $stationsNeeded(d)$, the total new stations required to guarantee no gap exceeds d . As d increases, fewer stations are needed per gap, so $stationsNeeded(d)$ is non-increasing in d .

Candidate Space and Predicate

The candidate space is real numbers $d \in (0, \max(gap_i)]$. The predicate is $P(d) = (stationsNeeded(d) \leq k)$, and we want the smallest d where P holds.

Why a fixed-precision loop replaces the integer loop

With a continuous candidate space, there is no notion of “the next integer candidate,” so the loop cannot terminate by lo and hi crossing as integers do. Instead, we run the loop a fixed number of times (or until $hi - lo$ shrinks below a chosen precision threshold ε , e.g. 10^{-6}), narrowing lo or hi to mid (not $mid \pm 1$, since there is no discrete step to add or subtract) at each iteration. Each iteration still halves the remaining interval, so a fixed number of iterations (around 50–100, since 2^{-100} is far smaller than any reasonable ε) is always enough regardless of the initial interval width.

Algorithm

1. Compute the existing gaps; $lo \leftarrow 0$, $hi \leftarrow \max(gap_i)$.
2. Repeat a fixed number of times (or while $hi - lo > \varepsilon$):
 - (a) $mid \leftarrow (lo + hi)/2$.
 - (b) Compute $stationsNeeded(mid) = \sum_i (\lceil gap_i / mid \rceil - 1)$.

(c) If $stationsNeeded(mid) \leq k$: $hi \leftarrow mid$ (feasible; try smaller).

(d) Else: $lo \leftarrow mid$ (infeasible; try larger).

3. Return hi (or lo ; they have converged to within ε).

Algorithm 35 Minimize Maximum Gas Station Distance

```

1: procedure MINIMIZE_MAX_DISTANCE( $stations, k$ )
2:   compute gaps  $gap_1, \dots, gap_{n-1}$ 
3:    $lo \leftarrow 0, hi \leftarrow \max_i gap_i$ 
4:   for  $iter \leftarrow 1$  to 100 do
5:      $mid \leftarrow (lo + hi)/2$ 
6:      $needed \leftarrow \sum_i (\lceil gap_i / mid \rceil - 1)$ 
7:     if  $needed \leq k$  then
8:        $hi \leftarrow mid$ 
9:     else
10:       $lo \leftarrow mid$ 
11:    end if
12:  end for
13:  return  $hi$ 
14: end procedure

```

Dry run

Dry Run. Take stations at $[1, 2, 3, 4, 5]$ (gaps all = 1), $k = 4$.

There are 4 gaps, each of length 1. With $k = 4$ new stations to distribute (one per gap), each gap of length 1 becomes two segments of length 0.5. So $stationsNeeded(0.5) = \sum (\lceil 1/0.5 \rceil - 1) = \sum (2 - 1) = 4 \cdot 1 = 4 \leq 4$ – feasible. Checking a slightly smaller candidate, say $d = 0.49$: $\lceil 1/0.49 \rceil - 1 = \lceil 2.04 \rceil - 1 = 3 - 1 = 2$ per gap, times 4 gaps = $8 > 4$ – infeasible. So the binary search converges to $d = 0.5$, the minimized maximum distance.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 double minimizeMaxDistance(vector<int>& stations, int k) {
5     int n = stations.size();
6     vector<double> gaps(n - 1);
7     for (int i = 0; i + 1 < n; i++) {
8         gaps[i] = stations[i + 1] - stations[i];
9     }
10
11     double lo = 0, hi = *max_element(gaps.begin(), gaps.end());
12
13     for (int iter = 0; iter < 100; iter++) {
14         double mid = (lo + hi) / 2.0;
15         long long needed = 0;
16         for (double g : gaps) {
17             needed += (long long)ceil(g / mid) - 1;
18         }
19
20         if (needed <= k) {
21             hi = mid;

```

```

22     } else {
23         lo = mid;
24     }
25 }
26 return hi;
27 }
28
29 int main() {
30     vector<int> stations = {1, 2, 3, 4, 5};
31     cout << fixed << setprecision(4);
32     cout << "Minimized maximum distance: "
33         << minimizeMaxDistance(stations, 4) << endl;
34     return 0;
35 }

```

Complexity Analysis

Time Complexity. The fixed-precision loop runs a constant number of iterations (e.g. 100), and each feasibility check costs $O(n)$, giving

$$O(n \cdot \text{iterations}) = O(n),$$

treating the iteration count as a constant determined by the desired precision (more precisely, $O(n \log(\text{range}/\varepsilon))$ if the precision ε is treated as a variable rather than fixed). **Space Complexity.** $O(n)$ to store the gaps.

Key Takeaway

Binary search on the answer is not limited to integer candidates. When the problem allows real-valued placement, keep every other part of the template (monotonic feasibility predicate, minimize/maximize narrowing direction) and simply replace the integer loop and its ± 1 adjustments with a fixed-iteration or epsilon-tolerance loop that narrows lo/hi directly to mid .

2.27 Median of Two Sorted Arrays

Problem Statement. We are given two sorted arrays *nums1* (size n_1) and *nums2* (size n_2). We must find the median of the combined sorted array of all $n_1 + n_2$ elements, **without** actually merging the two arrays, in $O(\log(\min(n_1, n_2)))$ time.

Pattern Recognition

The keyword combination “*median of two sorted arrays*” together with a required complexity of $O(\log(\min(n_1, n_2)))$ – far faster than the $O(n_1 + n_2)$ that an actual merge would cost – is the signal for a fundamentally different binary-search target than anything earlier in this chapter: we will binary search not over values, and not over a single array’s indices, but over **how the combined sorted sequence should be partitioned** between the two arrays. This “binary search on a partition point” technique is one of the most intricate patterns on the sheet, and is worth slowing down for.

Understanding the problem carefully

The median of a sorted sequence of length T splits it into a left half of size $\lceil T/2 \rceil$ and a right half of size $\lfloor T/2 \rfloor$, where every element in the left half is $\leq$ every element in the right half. Our

goal is to find *how many* elements of this left half come from *nums1* (call this *cut1*) and how many come from *nums2* (call this *cut2* = $\lceil T/2 \rceil - \text{cut1}$), *without* ever materializing the merged array.

For a candidate $\text{cut1} \in [0, n_1]$ (with *cut2* determined as above), define four boundary values:

$$l_1 = \text{nums1}[\text{cut1} - 1] \text{ (or } -\infty \text{ if } \text{cut1} = 0), \quad r_1 = \text{nums1}[\text{cut1}] \text{ (or } +\infty \text{ if } \text{cut1} = n_1),$$

$$l_2 = \text{nums2}[\text{cut2} - 1] \text{ (or } -\infty \text{ if } \text{cut2} = 0), \quad r_2 = \text{nums2}[\text{cut2}] \text{ (or } +\infty \text{ if } \text{cut2} = n_2).$$

A partition is **valid** exactly when $l_1 \leq r_2$ and $l_2 \leq r_1$: every element on the left side (l_1 and l_2) is at most every element on the right side (r_1 and r_2), which is precisely the condition needed for the left half (across both arrays combined) to contain only values $\leq$ every value in the right half.

Candidate Space and Predicate

The candidate space is $\text{cut1} \in [0, n_1]$ (binary searching on the *smaller* array keeps the range, and hence the iteration count, as small as possible). The predicate is $P(\text{cut1}) = (l_1 \leq r_2 \text{ and } l_2 \leq r_1)$. Because increasing *cut1* monotonically increases l_1 and r_1 while monotonically decreasing l_2 and r_2 (as *cut2* shrinks correspondingly), there is exactly one crossover region where the partition is valid, and binary search finds it directly.

Why exactly one valid partition exists, and how to search for it

If $l_1 > r_2$, the left part of *nums1* is reaching too far (taking values larger than what the right part of *nums2* can absorb), so *cut1* must decrease: search left, $hi \leftarrow \text{cut1} - 1$. If $l_2 > r_1$ (equivalently, *nums1*'s left part is not reaching far enough), *cut1* must increase: search right, $lo \leftarrow \text{cut1} + 1$. Because l_1, r_1 increase weakly with *cut1* and l_2, r_2 decrease weakly with *cut1* (equivalently increase weakly as *cut1* decreases), exactly one of these two failure conditions can hold at a time for an invalid partition, and the search converges to the unique boundary where neither failure condition holds.

Once a valid partition is found, the median is computed from $\max(l_1, l_2)$ and $\min(r_1, r_2)$: if T is odd, the median is $\max(l_1, l_2)$ (the largest element of the left half, which has one more element than the right half); if T is even, the median is the average of $\max(l_1, l_2)$ and $\min(r_1, r_2)$.

Algorithm

1. Ensure *nums1* is the smaller array (swap if necessary, so the binary search range is $[0, n_1]$ with $n_1 \leq n_2$).
2. $T \leftarrow n_1 + n_2$, $half \leftarrow \lceil T/2 \rceil$.
3. $lo \leftarrow 0$, $hi \leftarrow n_1$.
4. While $lo \leq hi$:
 - (a) $\text{cut1} \leftarrow lo + (hi - lo)/2$, $\text{cut2} \leftarrow half - \text{cut1}$.
 - (b) Compute l_1, r_1, l_2, r_2 as above.
 - (c) If $l_1 \leq r_2$ and $l_2 \leq r_1$: valid partition found; compute and return the median as described above.
 - (d) Else if $l_1 > r_2$: $hi \leftarrow \text{cut1} - 1$.
 - (e) Else: $lo \leftarrow \text{cut1} + 1$.

Algorithm 36 Median of Two Sorted Arrays

```

1: procedure FINDMEDIAN(nums1, nums2)
2:   if  $n_1 > n_2$  then swap nums1, nums2
3:   end if
4:    $T \leftarrow n_1 + n_2$ ,  $half \leftarrow \lceil T/2 \rceil$ 
5:    $lo \leftarrow 0$ ,  $hi \leftarrow n_1$ 
6:   while  $lo \leq hi$  do
7:      $cut1 \leftarrow lo + (hi - lo)/2$ 
8:      $cut2 \leftarrow half - cut1$ 
9:      $l_1 \leftarrow (cut1 = 0) ? -\infty : nums1[cut1 - 1]$ 
10:     $r_1 \leftarrow (cut1 = n_1) ? +\infty : nums1[cut1]$ 
11:     $l_2 \leftarrow (cut2 = 0) ? -\infty : nums2[cut2 - 1]$ 
12:     $r_2 \leftarrow (cut2 = n_2) ? +\infty : nums2[cut2]$ 
13:    if  $l_1 \leq r_2$  and  $l_2 \leq r_1$  then
14:      if  $T$  is odd then return  $\max(l_1, l_2)$ 
15:      elsereturn  $(\max(l_1, l_2) + \min(r_1, r_2))/2$ 
16:    end if
17:    else if  $l_1 > r_2$  then
18:       $hi \leftarrow cut1 - 1$ 
19:    else
20:       $lo \leftarrow cut1 + 1$ 
21:    end if
22:  end while
23: end procedure

```

Dry run

Dry Run. Take $nums1 = [1, 3]$, $nums2 = [2]$.

$n_1 = 2 \leq n_2 = 1$? No – swap so $nums1 = [2]$ ($n_1 = 1$), $nums2 = [1, 3]$ ($n_2 = 2$). $T = 3$, $half = \lceil 3/2 \rceil = 2$.

- $lo = 0, hi = 1$: $cut1 = 0$, $cut2 = 2 - 0 = 2$. $l_1 = -\infty$ ($cut1 = 0$), $r_1 = nums1[0] = 2$. $l_2 = nums2[1] = 3$ ($cut2 = 2 = n_2$, so $r_2 = +\infty$). Check: $l_1 = -\infty \leq r_2 = +\infty$? Yes. $l_2 = 3 \leq r_1 = 2$? No. So $l_2 > r_1$, $lo \leftarrow 1$.
- $lo = 1, hi = 1$: $cut1 = 1$, $cut2 = 2 - 1 = 1$. $l_1 = nums1[0] = 2$ ($cut1 = 1 = n_1$, so $r_1 = +\infty$). $l_2 = nums2[0] = 1$, $r_2 = nums2[1] = 3$. Check: $l_1 = 2 \leq r_2 = 3$? Yes. $l_2 = 1 \leq r_1 = +\infty$? Yes. Valid partition.

$T = 3$ is odd, so the median is $\max(l_1, l_2) = \max(2, 1) = 2$. Checking by hand: the merged array is $[1, 2, 3]$, whose median is indeed 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
5     if (nums1.size() > nums2.size()) swap(nums1, nums2);
6
7     int n1 = nums1.size(), n2 = nums2.size();
8     int total = n1 + n2;
9     int half = (total + 1) / 2;

```

```

10
11     int lo = 0, hi = n1;
12
13     while (lo <= hi) {
14         int cut1 = lo + (hi - lo) / 2;
15         int cut2 = half - cut1;
16
17         int l1 = (cut1 == 0) ? INT_MIN : nums1[cut1 - 1];
18         int r1 = (cut1 == n1) ? INT_MAX : nums1[cut1];
19         int l2 = (cut2 == 0) ? INT_MIN : nums2[cut2 - 1];
20         int r2 = (cut2 == n2) ? INT_MAX : nums2[cut2];
21
22         if (l1 <= r2 && l2 <= r1) {
23             if (total % 2 == 1) {
24                 return max(l1, l2);
25             } else {
26                 return (max(l1, l2) + min(r1, r2)) / 2.0;
27             }
28         } else if (l1 > r2) {
29             hi = cut1 - 1;
30         } else {
31             lo = cut1 + 1;
32         }
33     }
34     return -1.0; // unreachable for valid sorted input
35 }
36
37 int main() {
38     vector<int> nums1 = {1, 3};
39     vector<int> nums2 = {2};
40     cout << "Median: "
41          << findMedianSortedArrays(nums1, nums2) << endl;
42     return 0;
43 }

```

Complexity Analysis

Time Complexity. The binary search runs over $cut1 \in [0, n_1]$, where $n_1 = \min(n_1, n_2)$ (after the swap), giving

$$O(\log(\min(n_1, n_2))).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Binary search does not need to search over array *values* or even a single array's *indices* – here, it searches over a **split point** that simultaneously partitions two arrays, using a validity condition ($l_1 \leq r_2$ and $l_2 \leq r_1$) as the monotonic predicate. This “binary search on a partition” idea, once recognized, also unlocks Section 2.28, the general k -th element version of this same problem.

2.28 Kth Element of Two Sorted Arrays

Problem Statement. We are given two sorted arrays $nums1$ (size n_1) and $nums2$ (size n_2), and an integer k . We must find the k -th smallest element (1-indexed) of the combined sorted array, without merging, in $O(\log(\min(n_1, n_2)))$ time.

Pattern Recognition

This is the direct generalization of Section 2.27: the median is simply the special case $k = \lceil (n_1 + n_2)/2 \rceil$ (for odd total length) or the average of the k -th and $(k+1)$ -th elements (for even total length). The same “binary search on a partition split between two arrays” technique applies, with the partition sized to put exactly k elements on the left instead of exactly half.

Understanding the problem carefully

We again look for a split $(cut1, cut2)$ with $cut1 + cut2 = k$, $0 \leq cut1 \leq n_1$, and $0 \leq cut2 \leq n_2$, such that the left side (the first $cut1$ elements of *nums1* and first $cut2$ elements of *nums2*) contains exactly the k smallest elements overall. The same four boundary values l_1, r_1, l_2, r_2 and the same validity condition $l_1 \leq r_2$ and $l_2 \leq r_1$ from Section 2.27 apply unchanged.

Candidate Space and Predicate

The candidate space is $cut1 \in [\max(0, k - n_2), \min(k, n_1)]$ (the lower bound ensures $cut2 = k - cut1 \leq n_2$; the upper bound ensures $cut1 \leq n_1$ and $cut2 \geq 0$). The predicate is the same validity check as Section 2.27: $P(cut1) = (l_1 \leq r_2 \text{ and } l_2 \leq r_1)$.

Why only the target split size changes

Nothing about the underlying correctness argument changes from Section 2.27 – we are still looking for the unique partition where the left side’s largest element does not exceed the right side’s smallest element, guaranteeing the left side is exactly the k smallest elements of the combined array. Once a valid partition is found, the k -th smallest element is simply $\max(l_1, l_2)$ – the largest element among the k elements placed on the left.

Algorithm

1. $lo \leftarrow \max(0, k - n_2)$, $hi \leftarrow \min(k, n_1)$.
2. While $lo \leq hi$:
 - (a) $cut1 \leftarrow lo + (hi - lo)/2$, $cut2 \leftarrow k - cut1$.
 - (b) Compute l_1, r_1, l_2, r_2 exactly as in Section 2.27.
 - (c) If $l_1 \leq r_2$ and $l_2 \leq r_1$: return $\max(l_1, l_2)$.
 - (d) Else if $l_1 > r_2$: $hi \leftarrow cut1 - 1$.
 - (e) Else: $lo \leftarrow cut1 + 1$.

Algorithm 37 Kth Element of Two Sorted Arrays

```

1: procedure KTHELEMENT(nums1, nums2, k)
2:   lo  $\leftarrow$  max(0, k - n2), hi  $\leftarrow$  min(k, n1)
3:   while lo  $\leq$  hi do
4:     cut1  $\leftarrow$  lo + (hi - lo)/2
5:     cut2  $\leftarrow$  k - cut1
6:     l1  $\leftarrow$  (cut1 = 0) ?  $-\infty$  : nums1[cut1 - 1]
7:     r1  $\leftarrow$  (cut1 = n1) ?  $+\infty$  : nums1[cut1]
8:     l2  $\leftarrow$  (cut2 = 0) ?  $-\infty$  : nums2[cut2 - 1]
9:     r2  $\leftarrow$  (cut2 = n2) ?  $+\infty$  : nums2[cut2]
10:    if l1  $\leq$  r2 and l2  $\leq$  r1 then
11:      return max(l1, l2)
12:    else if l1 > r2 then
13:      hi  $\leftarrow$  cut1 - 1
14:    else
15:      lo  $\leftarrow$  cut1 + 1
16:    end if
17:  end while
18: end procedure

```

Dry run

Dry Run. Take *nums1* = [2, 3, 6, 7, 9], *nums2* = [1, 4, 8, 10], *k* = 5.

*n*₁ = 5, *n*₂ = 4. *lo* = max(0, 5 - 4) = 1, *hi* = min(5, 5) = 5.

- *lo* = 1, *hi* = 5: *cut1* = 3, *cut2* = 5 - 3 = 2. *l*₁ = *nums1*[2] = 6, *r*₁ = *nums1*[3] = 7. *l*₂ = *nums2*[1] = 4, *r*₂ = *nums2*[2] = 8. Check: *l*₁ = 6 $\leq$ *r*₂ = 8? Yes. *l*₂ = 4 $\leq$ *r*₁ = 7? Yes. Valid partition.

The answer is max(*l*₁, *l*₂) = max(6, 4) = 6. Checking by hand: the merged array is [1, 2, 3, 4, 6, 7, 8, 9, 10], whose 5-th smallest element (1-indexed) is indeed 6.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int kthElement(vector<int>& nums1, vector<int>& nums2, int k) {
5     int n1 = nums1.size(), n2 = nums2.size();
6
7     int lo = max(0, k - n2);
8     int hi = min(k, n1);
9
10    while (lo <= hi) {
11        int cut1 = lo + (hi - lo) / 2;
12        int cut2 = k - cut1;
13
14        int l1 = (cut1 == 0) ? INT_MIN : nums1[cut1 - 1];
15        int r1 = (cut1 == n1) ? INT_MAX : nums1[cut1];
16        int l2 = (cut2 == 0) ? INT_MIN : nums2[cut2 - 1];
17        int r2 = (cut2 == n2) ? INT_MAX : nums2[cut2];
18
19        if (l1 <= r2 && l2 <= r1) {
20            return max(l1, l2);

```

```

21     } else if (l1 > r2) {
22         hi = cut1 - 1;
23     } else {
24         lo = cut1 + 1;
25     }
26 }
27 return -1; // unreachable for valid input with 1 <= k <= n1+n2
28 }
29
30 int main() {
31     vector<int> nums1 = {2, 3, 6, 7, 9};
32     vector<int> nums2 = {1, 4, 8, 10};
33     cout << "5th smallest element: "
34          << kthElement(nums1, nums2, 5) << endl;
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. The binary search range has width at most $\min(k, n_1) - \max(0, k - n_2) \leq \min(n_1, n_2)$, giving

$$O(\log(\min(n_1, n_2))).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

Median-of-two-sorted-arrays and k -th-element-of-two-sorted-arrays are the same partition-search technique with only the target split size changed from “half” to “ k .” Whenever you solve one binary-search-on-a-partition problem, immediately check whether nearby sheet problems are just different target split sizes of the same idea.

2.29 Row with Maximum Number of 1s

We now begin the third and final group of problems in this chapter: **binary search on a 2D array**. The recurring theme is that each *row* (or column) of the matrix is independently sorted, which lets us reuse the 1D binary search templates from Sections 2.2–2.14 *per row or column*, or combine row-sortedness and column-sortedness to search the whole 2D index space in a single sweep.

Problem Statement. We are given an $n \times m$ binary matrix in which **every row is sorted** (all 0s appear before all 1s within that row). We must find the index of the row containing the **maximum number of 1s**.

Pattern Recognition

“Each row is sorted” (even though the matrix as a whole is not globally ordered in any simple way) is the keyword that tells us we can binary search *within each row independently*, rather than needing any 2D-aware search technique. Whenever a 2D-array problem’s sortedness guarantee is row-by-row (or column-by-column) rather than global, default first to “run a 1D binary search on each row/column,” since it is almost always the simplest correct approach, even if not always the asymptotically fastest one.

Understanding the problem carefully

Because each row is sorted with 0s before 1s, the number of 1s in a row equals $m - \text{lowerBound}(\text{row}, 1)$, where $\text{lowerBound}(\text{row}, 1)$ is the first index in that row where the value is ≥ 1 (Section 2.3). Computing this for every row and tracking the maximum gives the answer.

Reuse, Don't Re-derive

Apply Algorithm 2.3's LOWERBOUND procedure to each row independently (searching for the first 1), convert each result to a count of 1s, and track the row index achieving the maximum count.

A note on the alternative $O(n + m)$ staircase approach

A different, even faster technique exists for this specific problem: starting from the top-right corner of the matrix, move left whenever the current cell is 1 (recording this row as the best-so-far and shrinking the column boundary), and move down whenever the current cell is 0. This “staircase search” exploits row-and-column-sortedness jointly and runs in $O(n + m)$ total, faster than the $O(n \log m)$ of the per-row binary search. We mention it here because it foreshadows the 2D search technique used in Section 2.31, but the per-row binary search remains the more directly transferable technique for this chapter's purposes, since it requires no new derivation beyond what Section 2.3 already established.

Algorithm (Per-Row Binary Search)

1. Initialize $\text{bestRow} \leftarrow -1$, $\text{bestCount} \leftarrow 0$.
2. For each row i from 0 to $n - 1$:
 - (a) $\text{idx} \leftarrow \text{lowerBound}(\text{row } i, 1)$.
 - (b) $\text{count} \leftarrow m - \text{idx}$.
 - (c) If $\text{count} > \text{bestCount}$: update $\text{bestCount} \leftarrow \text{count}$, $\text{bestRow} \leftarrow i$.
3. Return bestRow .

Algorithm 38 Row with Maximum Number of 1s

```

1: procedure ROWWITHMAXONES(matrix)
2:    $\text{bestRow} \leftarrow -1$ ,  $\text{bestCount} \leftarrow 0$ 
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:      $\text{idx} \leftarrow \text{LOWERBOUND}(\text{matrix}[i], 1)$ 
5:      $\text{count} \leftarrow m - \text{idx}$ 
6:     if  $\text{count} > \text{bestCount}$  then
7:        $\text{bestCount} \leftarrow \text{count}$ ,  $\text{bestRow} \leftarrow i$ 
8:     end if
9:   end for
10:  return  $\text{bestRow}$ 
11: end procedure

```

Dry run

Dry Run. Take the matrix $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$.

- Row 0: [0,0,0]. Lower bound of 1 is index 3 (no 1 found). Count = $3 - 3 = 0$.
- Row 1: [0,1,1]. Lower bound of 1 is index 1. Count = $3 - 1 = 2$. Best so far: row 1, count 2.
- Row 2: [1,1,1]. Lower bound of 1 is index 0. Count = $3 - 0 = 3$. Best so far: row 2, count 3.
- Row 3: [0,0,0]. Count = 0.

The answer is row 2, with 3 ones.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lowerBoundRow(vector<int>& row, int X) {
5     int lo = 0, hi = row.size() - 1, ans = row.size();
6     while (lo <= hi) {
7         int mid = lo + (hi - lo) / 2;
8         if (row[mid] >= X) { ans = mid; hi = mid - 1; }
9         else lo = mid + 1;
10    }
11    return ans;
12 }
13
14 int rowWithMaxOnes(vector<vector<int>>& matrix) {
15     int n = matrix.size(), m = matrix[0].size();
16     int bestRow = -1, bestCount = 0;
17
18     for (int i = 0; i < n; i++) {
19         int idx = lowerBoundRow(matrix[i], 1);
20         int count = m - idx;
21         if (count > bestCount) {
22             bestCount = count;
23             bestRow = i;
24         }
25     }
26     return bestRow;
27 }
28
29 int main() {
30     vector<vector<int>> matrix = {
31         {0,0,0},
32         {0,1,1},
33         {1,1,1},
34         {0,0,0}
35     };
36     cout << "Row with max ones: " << rowWithMaxOnes(matrix) << endl;
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. For each of the n rows, a binary search costs $O(\log m)$, giving an

overall time complexity of

$$O(n \log m).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

The simplest way to handle a 2D problem where sortedness is guaranteed only *per row* (or per column) is to apply a 1D binary search template independently to each row, accepting an extra factor of n (or m) in the time complexity in exchange for needing no new derivation. Faster joint row-and-column techniques exist for some such problems (the staircase search above, and Section 2.31), but they require the matrix to be sorted in *both* directions simultaneously.

2.30 Search in a 2D Matrix

Problem Statement. We are given an $n \times m$ matrix with the following guarantees: every row is sorted in increasing order, and the first element of every row (except the first) is strictly greater than the last element of the previous row. Given a target X , determine whether X exists in the matrix, in $O(\log(nm))$ time.

Pattern Recognition

The extra guarantee here – *each row’s first element exceeds the previous row’s last element* – is what elevates this beyond “each row is sorted independently” (Section 2.29): it means that if you read the matrix row by row, left to right, the values you encounter are **globally non-decreasing**, exactly as if the matrix were a single flattened, sorted 1D array. Whenever a 2D matrix problem gives you this row-chaining guarantee, immediately think “flatten conceptually and run ordinary binary search,” rather than reaching for a more complex 2D technique.

Understanding the problem carefully

Because the matrix reads as a single sorted sequence when traversed row-major (row 0 left to right, then row 1 left to right, and so on), we can treat it as a virtual 1D array of length nm , where virtual index idx corresponds to matrix cell $(idx/m, idx \bmod m)$ (integer division for the row, remainder for the column). No actual flattening or copying is needed – we simply convert each binary search midpoint index into a (row, col) pair on the fly.

Candidate Space and Predicate

The candidate space is virtual indices $0, \dots, nm - 1$. The predicate is exactly the same as ordinary binary search (Section 2.2): compare the value at the virtual index to X and discard the appropriate half.

Why the row-chaining guarantee makes flattening valid

Ordinary binary search needs the entire candidate sequence (values, in virtual-index order) to be sorted. The row-chaining guarantee is precisely what makes this true here: within a row, sortedness gives increasing values left to right; across the row boundary, the guarantee that the next row’s first element exceeds the previous row’s last element ensures no drop in value occurs when virtual index crosses from the end of one row to the start of the next. So the entire virtual

sequence is non-decreasing, and ordinary binary search's discard argument (Section 2.2) applies without any modification beyond the index-mapping arithmetic.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow nm - 1$.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) $row \leftarrow mid/m$, $col \leftarrow mid \bmod m$.
 - (c) If $matrix[row][col] = X$: return **true**.
 - (d) If $matrix[row][col] < X$: $lo \leftarrow mid + 1$.
 - (e) Else: $hi \leftarrow mid - 1$.
3. If the loop ends without returning, return **false**.

Algorithm 39 Search in a 2D Matrix

```

1: procedure SEARCHMATRIX(matrix, X)
2:    $lo \leftarrow 0$ ,  $hi \leftarrow nm - 1$ 
3:   while  $lo \leq hi$  do
4:      $mid \leftarrow lo + (hi - lo)/2$ 
5:      $row \leftarrow mid/m$ ,  $col \leftarrow mid \bmod m$ 
6:     if  $matrix[row][col] = X$  then return true
7:     end if
8:     if  $matrix[row][col] < X$  then
9:        $lo \leftarrow mid + 1$ 
10:    else
11:       $hi \leftarrow mid - 1$ 
12:    end if
13:  end while
14:  return false
15: end procedure

```

Dry run

Dry Run. Take $matrix = \begin{bmatrix} 1 & 3 & 5 & 7 \\ 10 & 11 & 16 & 20 \\ 23 & 30 & 34 & 60 \end{bmatrix}$, $X = 3$. Here $n = 3$, $m = 4$, $nm = 12$.

- $lo = 0, hi = 11$: $mid = 5$, $row = 1$, $col = 1$, $matrix[1][1] = 11 > 3$. $hi = 4$.
- $lo = 0, hi = 4$: $mid = 2$, $row = 0$, $col = 2$, $matrix[0][2] = 5 > 3$. $hi = 1$.
- $lo = 0, hi = 1$: $mid = 0$, $row = 0$, $col = 0$, $matrix[0][0] = 1 < 3$. $lo = 1$.
- $lo = 1, hi = 1$: $mid = 1$, $row = 0$, $col = 1$, $matrix[0][1] = 3 = 3$. Return **true**.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool searchMatrix(vector<vector<int>>& matrix, int X) {
5     int n = matrix.size(), m = matrix[0].size();
6     int lo = 0, hi = n * m - 1;
7
8     while (lo <= hi) {
9         int mid = lo + (hi - lo) / 2;
10        int row = mid / m, col = mid % m;
11
12        if (matrix[row][col] == X) return true;
13        else if (matrix[row][col] < X) lo = mid + 1;
14        else hi = mid - 1;
15    }
16    return false;
17 }
18
19 int main() {
20     vector<vector<int>> matrix = {
21         {1, 3, 5, 7},
22         {10, 11, 16, 20},
23         {23, 30, 34, 60}
24     };
25     cout << (searchMatrix(matrix, 3) ? "true" : "false") << endl;
26     return 0;
27 }

```

Complexity Analysis

Time Complexity. The virtual candidate range of size nm halves each iteration, giving

$$O(\log(nm)).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

A 2D structure with a strong enough chaining guarantee between rows can be searched with completely unmodified 1D binary search, as long as you translate a single virtual index into a (row, col) pair via division and modulo. Always check first whether a 2D problem's sortedness guarantee is this strong before reaching for a more specialized 2D technique like the one needed in Section 2.31.

2.31 Search in a 2D Matrix II

Problem Statement. We are given an $n \times m$ matrix in which every row is sorted in increasing order **left to right**, and every column is sorted in increasing order **top to bottom** – but, unlike Section 2.30, there is **no** guarantee chaining one row to the next (the last element of a row may be larger than the first element of the next row). Given a target X , determine whether it exists in the matrix.

Pattern Recognition

The missing row-chaining guarantee is the keyword to notice by its *absence*: it means the matrix can **not** be treated as a flattened sorted 1D array (Section 2.30’s virtual-index trick would give wrong answers here, since the virtual sequence is not globally sorted). Instead, the two *independent* sortedness guarantees – rows increasing rightward, columns increasing downward – call for a different technique: starting from a corner where one direction increases and the other decreases the candidate’s relation to X , and walking a single staircase path.

Understanding the problem carefully

Start at the **top-right** corner, $(row, col) = (0, m-1)$. From this corner: moving **left** (decreasing col) only ever decreases the value (row sortedness), and moving **down** (increasing row) only ever increases the value (column sortedness). This gives us a clean three-way comparison at every step:

- If $matrix[row][col] = X$: found it.
- If $matrix[row][col] > X$: every entry further right in this row is even larger (row sortedness), so X cannot be in the rest of this row; move left: $col \leftarrow col - 1$.
- If $matrix[row][col] < X$: every entry further up in this column is even smaller (column sortedness), so X cannot be above in this column; move down: $row \leftarrow row + 1$.

Why the Top-Right Corner Is the Only Safe Starting Point

The top-left corner is the smallest element in the entire matrix (both row and column sortedness make every other element $\geq$ it), so both “move right” and “move down” would increase the value – a comparison there gives no way to discard a direction safely. The bottom-right corner is symmetric (the largest element, both directions only decrease). *Only* the top-right (or, symmetrically, bottom-left) corner has the property that one direction strictly increases the value while the other strictly decreases it, which is exactly what is needed to safely discard an entire row or column on each comparison.

Why this staircase walk is correct and terminates in $O(n + m)$

At each step, the walk either eliminates an entire remaining row’s worth of values (when it moves left, the eliminated value and everything to its right in that row are confirmed too large) or an entire remaining column’s worth (when it moves down, the eliminated value and everything above it in that column are confirmed too small) – but more precisely, each step moves the pointer one position left or one position down, and since col can decrease at most m times and row can increase at most n times before going out of bounds, the walk terminates after at most $n + m$ steps, either finding X or running off the matrix entirely (confirming X is absent).

Algorithm

1. Initialize $row \leftarrow 0$, $col \leftarrow m - 1$.
2. While $row < n$ and $col \geq 0$:
 - (a) If $matrix[row][col] = X$: return **true**.
 - (b) If $matrix[row][col] > X$: $col \leftarrow col - 1$.

(c) Else: $row \leftarrow row + 1$.

3. Return false.

Algorithm 40 Search in a 2D Matrix II (Staircase Search)

```

1: procedure SEARCHMATRIXII(matrix, X)
2:    $row \leftarrow 0$ ,  $col \leftarrow m - 1$ 
3:   while  $row < n$  and  $col \geq 0$  do
4:     if  $matrix[row][col] = X$  then return true
5:     end if
6:     if  $matrix[row][col] > X$  then
7:        $col \leftarrow col - 1$ 
8:     else
9:        $row \leftarrow row + 1$ 
10:    end if
11:  end while
12:  return false
13: end procedure

```

Dry run

Dry Run. Take $matrix = \begin{bmatrix} 1 & 4 & 7 & 11 \\ 2 & 5 & 8 & 12 \\ 3 & 6 & 9 & 16 \\ 10 & 13 & 14 & 17 \end{bmatrix}$, $X = 5$. ($n = m = 4$.)

- $row = 0, col = 3$: $matrix[0][3] = 11 > 5$. $col = 2$.
- $row = 0, col = 2$: $matrix[0][2] = 7 > 5$. $col = 1$.
- $row = 0, col = 1$: $matrix[0][1] = 4 < 5$. $row = 1$.
- $row = 1, col = 1$: $matrix[1][1] = 5 = 5$. Return true.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool searchMatrixII(vector<vector<int>>& matrix, int X) {
5     int n = matrix.size(), m = matrix[0].size();
6     int row = 0, col = m - 1;
7
8     while (row < n && col >= 0) {
9         if (matrix[row][col] == X) return true;
10        else if (matrix[row][col] > X) col--;
11        else row++;
12    }
13    return false;
14 }
15
16 int main() {
17     vector<vector<int>> matrix = {
18         {1, 4, 7, 11},

```

```

19         {2, 5, 8, 12},
20         {3, 6, 9, 16},
21         {10, 13, 14, 17}
22     };
23     cout << (searchMatrixII(matrix, 5) ? "true" : "false") << endl;
24     return 0;
25 }
```

Complexity Analysis

Time Complexity. Each step decreases *col* or increases *row*, and there are at most m decreases and n increases before the pointer exits the matrix, giving

$$O(n + m).$$

Note this is not $O(\log(nm))$ – the lack of row-chaining genuinely costs us the logarithmic complexity available in Section 2.30, even though this is still a vast improvement over the $O(nm)$ brute-force scan. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

When a 2D matrix is sorted along both rows and columns independently but not chained into one global order, look for a corner where the two sortedness directions act as *opposite* comparisons (one direction increasing, the other decreasing) relative to the target – that corner is always the unique safe starting point for a linear “staircase” walk, even though it does not give the $O(\log(nm))$ that full chaining would allow.

2.32 Find a Peak Element in a 2D Matrix

Problem Statement. We are given an $n \times m$ matrix of distinct integers, where no two horizontally or vertically adjacent cells are equal. A cell is a **peak** if it is strictly greater than all of its (up to four) horizontal and vertical neighbors; cells outside the matrix are treated as $-\infty$. Find the position of any one peak.

Pattern Recognition

This is the 2D generalization of Section 2.14, and the keyword pattern is identical: no sortedness, just a local-comparison rule plus an implicit $-\infty$ border. The new idea needed is how to binary search over *one* dimension (say, columns) while using an $O(n)$ linear scan over the *other* dimension (rows) as part of each predicate evaluation – a “binary search over one axis, brute-force scan over the other” technique that is common whenever a 2D problem reduces to a 1D binary search plus a per-step linear helper.

Understanding the problem carefully

Fix a candidate middle column *midCol*. Scan down that column to find the row *maxRow* containing the largest value in that column – call it $v = \text{matrix}[\text{maxRow}][\text{midCol}]$. Compare v to its left neighbor $\text{matrix}[\text{maxRow}][\text{midCol}-1]$ and right neighbor $\text{matrix}[\text{maxRow}][\text{midCol}+1]$ (treating out-of-range neighbors as $-\infty$).

- If v is greater than both neighbors: v is greater than everything above and below it in its own column (by choice of *maxRow* as the column’s maximum) *and* greater than its left and right neighbors – so v is a peak. Done.

- If the left neighbor is greater than v : since that left neighbor is, in particular, greater than the column maximum at $midCol$, the column $midCol - 1$ (or further left) must contain an even larger local structure – a peak is guaranteed to exist somewhere in the columns $[lo, midCol - 1]$, by the same $-\infty$ -border existence argument as Section 2.14, applied along the column axis.
- Symmetrically, if the right neighbor is greater, a peak is guaranteed in columns $[midCol + 1, hi]$.

Candidate Space and Predicate

The candidate space is columns $0, \dots, m - 1$. At each candidate column, an $O(n)$ scan finds that column's maximum value's row, and a constant-time comparison to its horizontal neighbors decides whether to stop (peak found) or which half of the remaining columns to search next – the same “climbing toward a guaranteed peak” argument as Section 2.14, just with an entire column's maximum standing in for a single array element.

Why restricting attention to each column's maximum is valid

The reason we only need to examine the *maximum* of each candidate column, rather than every cell in it, is that if a peak exists in this column, it is impossible for it to be at a position other than the column's maximum: a peak must be greater than its upper and lower neighbors within the same column, which already forces it to be at least as large as both of those, and chaining this reasoning shows the only cell in a column that could possibly satisfy “greater than both vertical neighbors” even before checking horizontal neighbors is necessarily a local maximum along that column. Examining the column's global maximum is therefore sufficient to either confirm a peak or correctly decide which direction to continue the binary search.

Algorithm

1. Initialize $lo \leftarrow 0$, $hi \leftarrow m - 1$.
2. While $lo \leq hi$:
 - (a) $midCol \leftarrow lo + (hi - lo)/2$.
 - (b) Scan column $midCol$ to find $maxRow$, the row of its maximum value v .
 - (c) Compare v to its left and right neighbors.
 - (d) If v exceeds both: return $(maxRow, midCol)$.
 - (e) If the left neighbor exceeds v : $hi \leftarrow midCol - 1$.
 - (f) Else (right neighbor exceeds v): $lo \leftarrow midCol + 1$.

Algorithm 41 Find Peak Element in a 2D Matrix

```

1: procedure FINDPEAK2D(matrix)
2:    $lo \leftarrow 0, hi \leftarrow m - 1$ 
3:   while  $lo \leq hi$  do
4:      $midCol \leftarrow lo + (hi - lo) / 2$ 
5:      $maxRow \leftarrow \arg \max_i matrix[i][midCol]$ 
6:      $v \leftarrow matrix[maxRow][midCol]$ 
7:      $left \leftarrow (midCol = 0) ? -\infty : matrix[maxRow][midCol - 1]$ 
8:      $right \leftarrow (midCol = m - 1) ? -\infty : matrix[maxRow][midCol + 1]$ 
9:     if  $v > left$  and  $v > right$  then
10:      return ( $maxRow, midCol$ )
11:     else if  $left > v$  then
12:        $hi \leftarrow midCol - 1$ 
13:     else
14:        $lo \leftarrow midCol + 1$ 
15:     end if
16:   end while
17: end procedure

```

Dry run

Dry Run. Take $matrix = \begin{bmatrix} 10 & 8 & 10 & 10 \\ 14 & 13 & 12 & 11 \\ 15 & 9 & 11 & 21 \\ 16 & 17 & 19 & 20 \end{bmatrix}$ ($n = m = 4$).

- $lo = 0, hi = 3$: $midCol = 1$. Column 1: $[8, 13, 9, 17]$; max is 17 at row 3. Left neighbor $matrix[3][0] = 16$, right neighbor $matrix[3][2] = 19$. Is $17 > 16$ and $17 > 19$? No ($17 < 19$). Right neighbor is greater: $lo = 2$.
- $lo = 2, hi = 3$: $midCol = 2$. Column 2: $[10, 12, 11, 19]$; max is 19 at row 3. Left neighbor $matrix[3][1] = 17$, right neighbor $matrix[3][3] = 20$. Is $19 > 17$ and $19 > 20$? No ($19 < 20$). Right neighbor greater: $lo = 3$.
- $lo = 3, hi = 3$: $midCol = 3$. Column 3: $[10, 11, 21, 20]$; max is 21 at row 2. Left neighbor $matrix[2][2] = 11$. No right neighbor (out of range, $-\infty$). Is $21 > 11$ and $21 > -\infty$? Yes. Peak found at $(2, 3)$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 pair<int,int> findPeak2D(vector<vector<int>>& matrix) {
5     int n = matrix.size(), m = matrix[0].size();
6     int lo = 0, hi = m - 1;
7
8     while (lo <= hi) {
9         int midCol = lo + (hi - lo) / 2;
10        int maxRow = 0;
11        for (int i = 1; i < n; i++) {
12            if (matrix[i][midCol] > matrix[maxRow][midCol]) maxRow = i;
13        }

```

```

14
15     int v = matrix[maxRow][midCol];
16     int left  = (midCol == 0)      ? INT_MIN : matrix[maxRow][midCol
17                                     - 1];
18     int right = (midCol == m - 1) ? INT_MIN : matrix[maxRow][midCol
19                                     + 1];
20
21     if (v > left && v > right) {
22         return {maxRow, midCol};
23     } else if (left > v) {
24         hi = midCol - 1;
25     } else {
26         lo = midCol + 1;
27     }
28 }
29
30 int main() {
31     vector<vector<int>> matrix = {
32         {10, 8, 10, 10},
33         {14, 13, 12, 11},
34         {15, 9, 11, 21},
35         {16, 17, 19, 20}
36     };
37     auto [r, c] = findPeak2D(matrix);
38     cout << "Peak at (" << r << ", " << c << ")" << endl;
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. The column range halves each iteration ($O(\log m)$ iterations), and each iteration scans a full column in $O(n)$, giving an overall time complexity of

$$O(n \log m).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

When a 2D problem's local-comparison structure generalizes a 1D pattern you already know (here, peak-finding from Section 2.14), look for a way to binary search one axis while reducing each candidate to a single representative value via a linear scan along the other axis – the representative value's own local comparisons then drive the binary search exactly as in the 1D version.

2.33 Median of a Row-wise Sorted Matrix

Problem Statement. We are given an $n \times m$ matrix in which every row is sorted in increasing order (with no chaining guarantee between rows), and nm is odd. Find the median of all nm elements.

Pattern Recognition

This closing problem of the chapter combines two ideas from earlier sections: *binary search on the answer* (Sections 2.15–2.28), where the candidate space is a range of *values* rather than indices, and *per-row binary search* (Section 2.29), where row-sortedness is exploited independently per row inside a helper function. The keyword combination “*median*” plus “*each row sorted*” plus a request for better than $O(nm \log(nm))$ (which a full sort-and-pick would cost) should trigger exactly this combination.

Understanding the problem carefully

The median of nm elements (with nm odd) is the value v such that exactly $(nm - 1)/2$ elements are strictly smaller than v , and v itself is present in the matrix. Equivalently, v is the smallest value for which $\text{countLessOrEqual}(v) \geq \lceil nm/2 \rceil$, where $\text{countLessOrEqual}(v)$ counts how many matrix elements are $\leq v$. We do not need to merge or sort the matrix to compute this count: for each row, since the row is independently sorted, the count of elements $\leq v$ in that row is exactly $\text{upperBound}(\text{row}, v)$ (Section 2.4, the first index strictly greater than v , which equals the count of elements $\leq v$). Summing this across all n rows gives $\text{countLessOrEqual}(v)$ for the whole matrix in $O(n \log m)$.

Candidate Space and Predicate

The candidate space is integer values in $[\min(\text{matrix}), \max(\text{matrix})]$ (the minimum and maximum single elements across the whole matrix, found by checking the first and last elements of each row, since rows are sorted). The predicate is $P(v) = (\text{countLessOrEqual}(v) \geq \lceil nm/2 \rceil)$, and we want the smallest v where P holds.

Why this nested binary search correctly finds the median

As v increases, $\text{countLessOrEqual}(v)$ can only increase or stay the same (a larger threshold never excludes an element that a smaller threshold included), so P is monotonic, justifying the outer binary search over v . The smallest v satisfying P is, by definition, the smallest value such that at least half of the matrix’s elements do not exceed it – which is exactly the definition of the median for an odd total count. Crucially, v itself does not need to be searched for separately within the matrix: because countLessOrEqual jumps by at least 1 at v if and only if v is actually present in the matrix (if v were absent, the count at v and at $v - 1$ would be identical, so the crossover could not happen exactly at an absent value), the binary search automatically lands on a value that is genuinely in the matrix.

Algorithm

1. $lo \leftarrow$ min over all rows’ first elements; $hi \leftarrow$ max over all rows’ last elements.
2. While $lo \leq hi$:
 - (a) $mid \leftarrow lo + (hi - lo)/2$.
 - (b) For each row, add $\text{upperBound}(\text{row}, mid)$ to get $count$.
 - (c) If $count < \lceil nm/2 \rceil$: $lo \leftarrow mid + 1$.
 - (d) Else: $hi \leftarrow mid - 1$.
3. Return lo .

Algorithm 42 Median of a Row-wise Sorted Matrix

```

1: procedure MATRIXMEDIAN(matrix)
2:    $lo \leftarrow \min_i \text{matrix}[i][0], hi \leftarrow \max_i \text{matrix}[i][m-1]$ 
3:    $half \leftarrow \lceil nm/2 \rceil$ 
4:   while  $lo \leq hi$  do
5:      $mid \leftarrow lo + (hi - lo)/2$ 
6:      $count \leftarrow \sum_i \text{UPPERBOUND}(\text{matrix}[i], mid)$ 
7:     if  $count < half$  then
8:        $lo \leftarrow mid + 1$ 
9:     else
10:       $hi \leftarrow mid - 1$ 
11:    end if
12:  end while
13:  return  $lo$ 
14: end procedure

```

Dry run

Dry Run. Take $\text{matrix} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 6 & 9 \\ 3 & 6 & 9 \end{bmatrix}$ ($n = m = 3, nm = 9$).

$half = \lceil 9/2 \rceil = 5$. $lo = \min(1, 2, 3) = 1$, $hi = \max(5, 9, 9) = 9$.

- $lo = 1, hi = 9$: $mid = 5$. Counts ≤ 5 : row 1: $[1, 3, 5]$, upper bound of 5 is index 3 (all 3 qualify). Row 2: $[2, 6, 9]$, upper bound of 5 is index 1 (just the 2). Row 3: $[3, 6, 9]$, upper bound of 5 is index 1 (just the 3). Total = $3 + 1 + 1 = 5 \geq 5$. $hi = 4$.
- $lo = 1, hi = 4$: $mid = 2$. Counts ≤ 2 : row 1: upper bound of 2 is index 1 (just the 1). Row 2: upper bound of 2 is index 1 (just the 2). Row 3: upper bound of 2 is index 0 (none). Total = $1 + 1 + 0 = 2 < 5$. $lo = 3$.
- $lo = 3, hi = 4$: $mid = 3$. Counts ≤ 3 : row 1: index 2 (the 1, 3). Row 2: index 1 (just the 2). Row 3: index 1 (just the 3). Total = $2 + 1 + 1 = 4 < 5$. $lo = 4$.
- $lo = 4, hi = 4$: $mid = 4$. Counts ≤ 4 : same as ≤ 3 for these rows (no element equals 4), so total = $4 < 5$. $lo = 5$.
- $lo = 5 > hi = 4$: loop ends.

The answer is $lo = 5$. Checking by hand: sorting all 9 elements gives $[1, 2, 3, 3, 5, 6, 6, 9, 9]$, and the middle (5th) element is indeed 5.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int upperBoundRow(vector<int>& row, int X) {
5   int lo = 0, hi = row.size() - 1, ans = row.size();
6   while (lo <= hi) {
7     int mid = lo + (hi - lo) / 2;
8     if (row[mid] > X) { ans = mid; hi = mid - 1; }
9     else lo = mid + 1;
10  }

```

```

11     return ans;
12 }
13
14 int matrixMedian(vector<vector<int>>& matrix) {
15     int n = matrix.size(), m = matrix[0].size();
16     int lo = INT_MAX, hi = INT_MIN;
17     for (auto& row : matrix) {
18         lo = min(lo, row[0]);
19         hi = max(hi, row[m - 1]);
20     }
21
22     int half = (n * m) / 2 + 1; // ceil(nm/2) for odd nm
23
24     while (lo <= hi) {
25         int mid = lo + (hi - lo) / 2;
26         int count = 0;
27         for (auto& row : matrix) {
28             count += upperBoundRow(row, mid);
29         }
30
31         if (count < half) {
32             lo = mid + 1;
33         } else {
34             hi = mid - 1;
35         }
36     }
37     return lo;
38 }
39
40 int main() {
41     vector<vector<int>> matrix = {
42         {1, 3, 5},
43         {2, 6, 9},
44         {3, 6, 9}
45     };
46     cout << "Median: " << matrixMedian(matrix) << endl;
47     return 0;
48 }

```

Complexity Analysis

Time Complexity. The candidate value range halves each iteration ($O(\log(\max - \min))$ iterations), and each feasibility check sums a per-row binary search costing $O(\log m)$ across n rows, giving an overall time complexity of

$$O(n \log m \log(\max - \min)),$$

substantially better than sorting all nm elements directly. **Space Complexity.** $O(1)$ auxiliary space.

Key Takeaway

This closing problem is a deliberate capstone: it requires recognizing that “binary search on the answer” (search over a value range using a monotonic count predicate) and “per-row binary search” (exploiting row-sortedness inside the predicate) are not mutually exclusive techniques – they nest naturally, with the outer search handling *which* value to test and the inner per-row searches handling *how cheaply* to test it. Recognizing when two patterns from

different parts of this chapter can be combined, rather than always reaching for a single isolated template, is the most advanced skill this chapter has to offer.